Appunti di Java

Table of contents

1 Learning Stack — Java **9**

I Introduzione **10**

2 Introduzione a Java **11**
2.1 Perché Java? . . . 11
2.2 Compilatori e interpreti . . . 11
2.3 La JVM e il bytecode . . . 12
2.4 Perché si chiama Java? . . . 13
2.5 File e programmi: Java vs C . . . 13
2.6 Il primo programma Java . . . 14
2.7 Il metodo main — riga per riga . . . 14
2.8 `System.out.println` — OOP in azione . . . 15
2.9 Compilare ed eseguire da terminale . . . 15
2.10 Riepilogo . . . 16

3 Tipi, stringhe e input **17**
3.1 Tipi primitivi e oggetti . . . 17
3.2 Gli 8 tipi primitivi . . . 17
3.3 Valori di default . . . 18
3.4 Casting: convertire tra tipi . . . 19
3.5 La classe String . . . 20
3.6 Confrontare le stringhe: `==` vs `equals()` . . . 20
3.7 Il package `java.lang` . . . 21
3.8 Le classi wrapper . . . 22
3.9 Input da tastiera con Scanner . . . 23
3.10 Esempio completo: scheda personaggio . . . 24
3.11 Riepilogo . . . 25

II OOP **26**

4 Paradigmi e oggetti **27**
4.1 Un nuovo modo di pensare . . . 27
4.2 Il paradigma imperativo . . . 27

4.3 I limiti del paradigma imperativo . . . 28
4.4 Il paradigma orientato agli oggetti . . . 29
4.5 I tre pilastri dell'OOP . . . 29
4.6 Classi e oggetti . . . 30
4.7 Il primo oggetto in Java . . . 30
4.8 Creare e usare oggetti . . . 31
4.9 Confronto diretto: C vs Java . . . 32
4.10 Riepilogo . . . 32

5 Creare una classe **33**
5.1 Da struct a class . . . 33
5.2 Struttura di una classe Java . . . 34
5.3 Attributi e incapsulamento . . . 34
5.4 Il costruttore . . . 35
5.5 La parola chiave `this` . . . 36
5.6 Tipi di costruttori . . . 36
5.6.1 Overloading . . . 37
5.7 Getter e setter . . . 38
5.8 La classe completa . . . 39
5.9 Usare la classe . . . 42
5.10 Riepilogo . . . 43

6 Incapsulamento e ADT **44**
6.1 Riprendiamo da dove eravamo . . . 44
6.2 Il problema degli attributi pubblici . . . 44
6.3 Incapsulamento . . . 45
6.4 Information hiding . . . 46
6.5 Tipi di dato astratti (ADT) . . . 46
6.6 Il Personaggio come ADT . . . 47
6.7 Un secondo esempio: l'arma come ADT . . . 47
6.8 Incapsulamento in azione: simulazione completa . . . 49
6.9 Riepilogo . . . 50

7 Ereditarietà **51**
7.1 Il problema della duplicazione . . . 51
7.2 La soluzione: ereditarietà . . . 52
7.3 Costruiamo la gerarchia . . . 52
7.4 Usare la gerarchia . . . 55
7.5 La parola chiave `super` . . . 56
7.6 Il principio "è un" . . . 56
7.7 La gerarchia completa . . . 57
7.8 Riepilogo . . . 57

8 Polimorfismo, upcasting e downcasting **59**
8.1 Riprendiamo dal polimorfismo . . . 59
8.2 Override: ridefinire un metodo . . . 59
8.3 Riferimenti polimorfici e il vantaggio reale . . . 61
8.4 Upcasting . . . 62
8.5 Downcasting . . . 62
8.6 L'operatore `instanceof` . . . 63
8.7 `==` vs `equals()` . . . 63
8.8 Implementare `equals()` in una classe . . . 64
8.9 Riepilogo . . . 65

9 Classi astratte e interfacce **66**
9.1 Il problema del metodo "generico" . . . 66
9.2 Classi astratte . . . 67
9.3 Metodi astratti . . . 68
9.4 Le sottoclassi concrete . . . 69
9.5 Interfacce . . . 70
9.6 Dichiarare un'interfaccia . . . 71
9.7 Implementare un'interfaccia . . . 71
9.8 Classe astratta vs interfaccia . . . 73
9.9 Esempio completo . . . 73
9.10 Riepilogo . . . 74

III Eccezioni **75**

10 Eccezioni **76**
10.1 Cosa succede quando qualcosa va storto? . . . 76
10.2 Cos'è un'eccezione . . . 76
10.3 La gerarchia delle eccezioni . . . 77
10.4 Le eccezioni più comuni . . . 77
10.5 Try-catch: gestire le eccezioni . . . 78
10.6 Blocchi multipli catch . . . 79
10.7 Il blocco finally . . . 80
10.8 Lanciare eccezioni con throw . . . 81
10.9 Creare eccezioni personalizzate . . . 82
10.10 Propagare eccezioni con throws . . . 82
10.11 Esempio completo: input robusto . . . 83
10.12 Riepilogo . . . 84

IV Collections e Stream 86

11 Java Collections 87
11.1 Il problema degli array . . . 87
11.2 La gerarchia delle Collections . . . 87
11.3 Generics: collezioni tipizzate . . . 88
11.4 ArrayList: la lista dinamica . . . 89
11.5 Iterare su una lista . . . 90
11.6 HashMap: la mappa chiave-valore . . . 90
11.7 Iterare su una HashMap . . . 91
11.8 Collections con oggetti personalizzati . . . 92
11.9 `equals()` e `hashCode()`: il contratto fondamentale . . . 93
11.10 La classe Collections (utility) . . . 94
11.11 Comparable: definire l'ordine naturale . . . 95
11.11.1 Implementare Comparable . . . 96
11.12 Array vs ArrayList — quando usare cosa . . . 98
11.13 Esempio completo: gestione inventario . . . 98
11.14 Riepilogo . . . 99

12 List, Set e Queue 101
12.1 Tre strutture, tre problemi diversi . . . 101
12.2 List: sequenza ordinata . . . 101
12.2.1 ArrayList vs LinkedList . . . 101
12.2.2 Metodi aggiuntivi di List . . . 102
12.3 Set: nessun duplicato . . . 103
12.3.1 HashSet . . . 103
12.3.2 TreeSet . . . 103
12.3.3 Operazioni tra insiemi . . . 104
12.4 Queue: la coda . . . 105
12.4.1 Metodi principali di Queue . . . 105
12.4.2 PriorityQueue: coda con priorità . . . 106
12.5 Confronto finale: quale struttura scegliere? . . . 107
12.6 Esempio completo: sistema di turni . . . 108
12.7 Riepilogo . . . 109

13 Stream e pipeline 110
13.1 Un nuovo modo di pensare alle collections . . . 110
13.2 Cos'è uno Stream . . . 111
13.3 Creare uno Stream . . . 111
13.4 Lambda e method reference . . . 112
13.5 Operazioni intermedie . . . 112
13.5.1 filter() . . . 113
13.5.2 map() . . . 113

13.5.3 sorted() . . . 114
13.5.4 limit() e skip() . . . 114
13.5.5 distinct() e peek() . . . 115
13.6 Operazioni terminali . . . 115
13.6.1 collect() . . . 115
13.6.2 forEach() . . . 116
13.6.3 count(), sum(), average(), min(), max() . . . 116
13.6.4 anyMatch(), allMatch(), noneMatch() . . . 117
13.6.5 findFirst() e findAny() . . . 117
13.7 Optional: gestire l'assenza di valore . . . 118
13.8 Pipeline complete: esempi reali . . . 118
13.8.1 Classifica dei sopravvissuti . . . 118
13.8.2 Statistiche della squadra . . . 119
13.8.3 Dal file al report . . . 119
13.9 Riepilogo . . . 120

V Stringhe avanzate **121**

14 Stringhe avanzate **122**
14.1 String è immutabile . . . 122
14.2 Il problema delle concatenazioni in ciclo . . . 122
14.3 StringBuilder: il buffer delle stringhe . . . 123
14.4 Metodi principali di StringBuilder . . . 123
14.5 Lo String Pool . . . 124
14.6 Stack, Heap e String Pool: dove vivono gli oggetti . . . 125
14.7 Simulazione della memoria . . . 126
14.8 Esperimento guidato . . . 127
14.9 Il metodo `intern()` . . . 128
14.10 Metodi avanzati di String . . . 128
14.10.1 Concatenare con join e formatted . . . 129
14.11 Espressioni regolari . . . 129
14.11.1 I caratteri speciali principali . . . 129
14.11.2 matches(), replaceAll(), split() . . . 130
14.11.3 Pattern e Matcher: ricerca avanzata . . . 131
14.12 Esempio completo: parser di un log di gioco . . . 131
14.13 Riepilogo . . . 133

VI Gestione file **134**

15 Path, Files e Log **135**
15.1 Lavorare con i file: una premessa . . . 135

15.2 Path: non è una stringa . . . 135
15.3 Percorsi relativi e assoluti . . . 136
15.4 Path vs Files: separazione delle responsabilità . . . 137
15.5 Operazioni di base su Path . . . 137
15.6 Verificare esistenza e tipo . . . 138
15.7 Leggere un file: tre approcci . . . 138
15.7.1 `readAllLines()` — tutto in memoria . . . 138
15.7.2 `readString()` — come unica stringa . . . 138
15.7.3 `Files.lines()` — stream lazy . . . 139
15.8 Scrivere un file . . . 139
15.9 Cartelle di output: buona pratica . . . 140
15.10 Esempio completo: analisi log di accessi . . . 141
15.11 Log: registrare gli eventi . . . 143
15.12 Gestione delle date con `java.time` . . . 144
15.13 Riepilogo . . . 145

16 Buffer e scrittura **146**
16.1 Il principio del buffer . . . 146
16.2 Buffer impliciti: le API di alto livello . . . 146
16.3 Il primo passo verso il buffer: `Files.lines()` . . . 147
16.4 Buffer espliciti su testo: BufferedReader e BufferedWriter . . . 147
16.4.1 BufferedReader: leggere riga per riga . . . 147
16.4.2 Gestione delle eccezioni dentro il try . . . 148
16.4.3 Due risorse nello stesso try . . . 148
16.4.4 BufferedWriter: scrivere con buffer . . . 149
16.4.5 Append con BufferedWriter . . . 149
16.4.6 PrintWriter: testo formattato . . . 150
16.5 L'analogia tra file e stringhe . . . 150
16.6 Buffer su flussi binari . . . 150
16.7 Buffer specializzati: DataInputStream e DataOutputStream . . . 151
16.7.1 Quando usarli . . . 152
16.8 Il quadro completo dei buffer . . . 153
16.9 La codifica dei caratteri . . . 153
16.10 Flush: svuotare il buffer manualmente . . . 154
16.11 Il quadro completo: quando usare cosa . . . 154
16.12 Esempio completo: logger con buffer e rotazione giornaliera . . . 155
16.13 Riepilogo . . . 157

VII Qualità del codice **158**

17 Testing con JUnit **159**
17.1 Perché testare il codice? . . . 159

17.2 JUnit 5: il framework di testing . . . 159
17.3 Struttura di una classe di test . . . 160
17.4 Le asserzioni principali . . . 161
17.5 Scrivere test significativi . . . 162
17.6 Test delle eccezioni . . . 165
17.7 Test parametrizzati . . . 165
17.8 Piano di test . . . 166
17.9 Test-Driven Development (TDD) . . . 167
17.10 Esempio completo: test della classe Arma . . . 168
17.11 Riepilogo . . . 170

18 SOLID **171**
18.1 Cos'è SOLID . . . 171
18.2 S - Single Responsibility Principle . . . 171
18.2.1 Il problema . . . 172
18.2.2 La soluzione . . . 172
18.3 O - Open/Closed Principle . . . 173
18.3.1 Il problema . . . 173
18.3.2 La soluzione . . . 174
18.4 L - Liskov Substitution Principle . . . 175
18.4.1 Il problema . . . 175
18.4.2 La soluzione . . . 176
18.5 I - Interface Segregation Principle . . . 176
18.5.1 Il problema . . . 176
18.5.2 La soluzione . . . 177
18.6 D - Dependency Inversion Principle . . . 178
18.6.1 Il problema . . . 178
18.6.2 La soluzione . . . 179
18.7 I cinque principi insieme . . . 180
18.8 Riepilogo . . . 181

1 Learning Stack — Java

Appunti di programmazione in Java.

Part I

Introduzione

2 Introduzione a Java

2.1 Perché Java?

Prima di scrivere una sola riga di codice, vale la pena capire **perché esiste Java** e cosa lo rende diverso dai linguaggi che conosci già.

Con il C hai imparato a ragionare in modo imperativo: istruzioni, variabili, funzioni. Con il c il codice compilato funziona **solo** sulla macchina per cui è stato compilato. Un programma compilato su Linux non gira su Windows, e viceversa.

Java nasce negli anni '90 con un obiettivo preciso:

> *"Write once, run anywhere."* Scrivi il codice una volta, eseguilo ovunque.

2.2 Compilatori e interpreti

Per capire come Java risolve il problema della portabilità, dobbiamo fare un passo indietro.

Esistono due modi fondamentali per eseguire un programma:

Il compilatore traduce l'intero sorgente in codice macchina *prima* dell'esecuzione. Il risultato è un file eseguibile velocissimo, ma legato a uno specifico sistema operativo e processore. È quello che fa `gcc` con il C.

L'interprete legge ed esegue il sorgente riga per riga, senza compilare prima. È più flessibile — lo stesso file gira su qualsiasi macchina che ha l'interprete installato — ma in genere più lento. Python funziona così.

Compilatore vs interprete in sintesi

	Compilatore	Interprete
Quando traduce	Prima dell'esecuzione	Durante l'esecuzione
Velocità	Più veloce	Più lenta
Portabilità	Limitata	Alta
Esempio	C, C++	Python, JavaScript

Java fa qualcosa di più intelligente: **unisce i vantaggi di entrambi**.

2.3 La JVM e il bytecode

Ecco come funziona il ciclo di vita di un programma Java:

1. Tu scrivi il codice sorgente in un file `.java`.

2. Il compilatore `javac` traduce il sorgente non in codice macchina, ma in **bytecode** — un linguaggio intermedio, portabile e standardizzato, contenuto in file `.class`.

3. La Java Virtual Machine (JVM) legge il bytecode ed esegue il programma sulla macchina reale. Ogni sistema operativo ha la sua JVM, ma il bytecode è sempre lo stesso.

```
Codice sorgente (.java)
        ↓  javac
    Bytecode (.class)
        ↓  JVM
   Esecuzione sul sistema
```

Il compilatore JIT

La JVM non si limita a interpretare il bytecode riga per riga. Al suo interno c'è anche un compilatore **Just-In-Time (JIT)** che, mentre il programma gira, traduce le parti più usate direttamente in codice macchina nativo. Risultato: la velocità si avvicina a quella del C, mantenendo la portabilità.

Il bytecode non è codice macchina e non può girare direttamente sul processore. Ma è uguale su tutte le piattaforme. Basta avere la JVM installata, e il programma gira.

2.4 Perché si chiama Java?

Il nome originale del linguaggio era **Oak** ("quercia") ma esisteva già un altro linguaggio con quel nome e serviva un'alternativa.

Durante una riunione che stava diventando molto lunga, il team di sviluppatori stava bevendo grandi quantità di caffè. Qualcuno propose **Java** — slang americano per "caffè", che prende il nome dall'isola indonesiana famosa per le sue piantagioni.

Il nome piacque e rimase. Da qui il logo della tazzina fumante che ancora oggi identifica Java.

> **JDK, JRE e JVM — le sigle ricorrenti**
>
> - **JVM** (Java Virtual Machine) — esegue il bytecode
> - **JRE** (Java Runtime Environment) — JVM + librerie standard, per chi vuole solo *eseguire* programmi Java
> - **JDK** (Java Development Kit) — JRE + compilatore + strumenti di sviluppo, per chi vuole *scrivere* programmi Java
>
> Se stai sviluppando, ti serve il JDK.

2.5 File e programmi: Java vs C

In C i programmi erano divisi in file `.h` (intestazioni con le dichiarazioni) e file `.c` (implementazioni). Per collegare tutto si usava `#include`, che funziona in modo molto meccanico: il compilatore copia il contenuto del file incluso dentro al programma.

Java funziona diversamente:

- Ogni classe sta in un **file separato** chiamato esattamente come la classe: `NomeClasse.java`
- Non esistono file header `.h`
- Per usare classi già pronte si usa `import`

```
import java.util.Scanner;  // importa la classe Scanner
```

> **`import` non è `#include`**
>
> Quando scrivi `import`, non stai copiando del codice nel tuo file. Stai solo dicendo al compilatore **dove trovare** quella classe.

2.6 Il primo programma Java

In Java, anche il programma più semplice deve avere almeno una **classe** e un metodo **`main`**. Il `main` è il punto di ingresso: è da lì che la JVM comincia a eseguire le istruzioni.

```java
public class PrimoProgramma {
    public static void main(String[] args) {
        System.out.println("Ciao, Java!");
    }
}
```

Tre regole fondamentali:

- Tutto il codice sta **dentro una classe**
- L'esecuzione parte sempre dal **`main`**
- Il file deve chiamarsi **esattamente come la classe**: `PrimoProgramma.java`

2.7 Il metodo main — riga per riga

La firma del main è sempre questa:

```java
public static void main(String[] args)
```

Analizziamola pezzo per pezzo:

Parola	Significato
`public`	Visibile alla JVM — deve poterlo chiamare dall'esterno
`static`	Non serve creare un oggetto per chiamarlo — la JVM lo chiama direttamente
`void`	Non restituisce nessun valore
`String[] args`	Può ricevere argomenti da riga di comando

> **Il main non va in tutte le classi**
>
> Il `main` serve **solo nella classe da cui parte il programma**. Tutte le altre classi non hanno un `main` — hanno solo attributi e metodi.

2.8 `System.out.println` — OOP in azione

La prima istruzione che scrivi in Java è già un esempio perfetto di programmazione a oggetti:

```
System.out.println("Ciao, Java!");
```

Scomponila così:

- `System` — una **classe** della libreria standard Java
- `out` — un **oggetto** di tipo `PrintStream` che gestisce l'output
- `println()` — un **metodo** che stampa il testo e va a capo

In C usavi `printf()` — una funzione globale. In Java invece stai chiamando un **metodo su un oggetto**. Anche per stampare una riga, Java ragiona in termini di oggetti che collaborano.

```
System.out.println("Testo con a capo");  // va a capo
System.out.print("Testo senza a capo"); // non va a capo
System.out.printf("Formattato: %d\n", 42); // come printf in C
```

2.9 Compilare ed eseguire da terminale

Se vuoi fare tutto da terminale senza IDE:

```
# Compila il file sorgente → genera PrimoProgramma.class
javac PrimoProgramma.java

# Esegui il bytecode con la JVM
java PrimoProgramma
```

Attenzione al nome

Il comando `java` vuole il **nome della classe**, non il nome del file. Quindi `java PrimoProgramma`, non `java PrimoProgramma.class`.

2.10 Riepilogo

I concetti chiave

- Java compila il sorgente `.java` in **bytecode** `.class`, non in codice macchina.
- La **JVM** esegue il bytecode su qualsiasi sistema operativo — *write once, run anywhere*.
- Il compilatore **JIT** ottimizza l'esecuzione traducendo il bytecode in codice nativo al volo.
- Ogni classe sta in un file `.java` con lo stesso nome della classe.
- `import` indica dove trovare una classe, non copia il suo codice.
- Ogni programma ha un punto di ingresso: il metodo `main`.
- `System.out.println()` è già OOP: classe, oggetto e metodo.

3 Tipi, stringhe e input

3.1 Tipi primitivi e oggetti

Java fa una distinzione netta tra due categorie di dati: i **tipi primitivi** e gli **oggetti**.

I **tipi primitivi** sono i mattoni fondamentali del linguaggio: numeri, caratteri, valori logici. Sono scritti in minuscolo, occupano una quantità fissa di memoria e contengono direttamente il loro valore.

Le **classi** descrivono oggetti complessi con dati e metodi. Si scrivono con l'iniziale maiuscola — proprio per segnare la differenza visiva rispetto ai tipi primitivi.

> **La differenza fondamentale in memoria**
>
> Quando assegni un valore a una variabile primitiva, memorizzi **direttamente quel dato**. Quando assegni un oggetto, memorizzi un **riferimento** — un indirizzo che punta a dove l'oggetto si trova in memoria.
>
> ```
> int vita = 100;          // il valore 100 è direttamente nella variabile
> Personaggio eroe = ...;  // eroe contiene un indirizzo, non l'oggetto
> ```

3.2 Gli 8 tipi primitivi

Java ha **8 tipi primitivi**, divisi in quattro categorie:

Categoria	Tipo	Dimensione	Intervallo / Note
Interi	`byte`	8 bit	da -128 a 127
	`short`	16 bit	da -32.768 a 32.767

Categoria	Tipo	Dimensione	Intervallo / Note
	`int`	32 bit	da -2 miliardi a +2 miliardi — il più usato
	`long`	64 bit	numeri molto grandi, si scrive con `L` finale
Reali	`float`	32 bit	precisione singola, si scrive con `f` finale
	`double`	64 bit	precisione doppia — il più usato per i decimali
Caratteri	`char`	16 bit	un singolo carattere Unicode, tra apici singoli
Booleani	`boolean`	—	solo `true` o `false`

```java
int livello = 5;
long punteggio = 9_999_999_999L;  // L obbligatorio per i long grandi
double velocita = 3.14;
float peso = 72.5f;               // f obbligatorio per i float
char iniziale = 'A';
boolean inVita = true;
```

> **Quando usare int e quando long?**
>
> Nella maggior parte dei casi `int` è sufficiente. Usa `long` solo quando sai che il numero può superare circa 2 miliardi — ad esempio un punteggio globale online, un timestamp in millisecondi, o l'ID di un database.

3.3 Valori di default

Quando dichiari un attributo di una classe senza assegnare un valore, Java lo inizializza automaticamente:

Tipo	Valore di default
`int`, `short`, `byte`, `long`	`0`

Tipo	Valore di default
`float`, `double`	`0.0`
`char`	`'\u0000'` (carattere vuoto)
`boolean`	`false`
Qualsiasi oggetto	`null`

Attenzione alle variabili locali

I valori di default valgono solo per gli **attributi di una classe**. Le variabili locali (dentro un metodo) **non** vengono inizializzate automaticamente — il compilatore ti dà errore se provi a usarle senza averle prima assegnato.

```
int x;
System.out.println(x); // ERRORE: variable x might not have been initialized
```

3.4 Casting: convertire tra tipi

A volte devi convertire un tipo in un altro. Java distingue due casi:

Casting implicito da un tipo "piccolo" a uno "grande", senza perdita di dati. Java lo fa automaticamente:

```
int n = 42;
double d = n;  // automatico: int → double, nessun problema
```

Casting esplicito da un tipo "grande" a uno "piccolo", con possibile perdita di dati. Devi dirlo esplicitamente con le parentesi:

```
double punteggio = 97.8;
int puntiInteri = (int) punteggio;  // diventa 97 - la parte decimale viene troncata!
```

Il casting tronca, non arrotonda

`(int) 97.8` dà `97`, non `98`. Se vuoi arrotondare usa `Math.round()`:

```
long arrotondato = Math.round(97.8);  // dà 98
```

3.5 La classe String

`String` non è un tipo primitivo — è una **classe**. Si scrive con la maiuscola e ogni stringa è un oggetto a tutti gli effetti, con i propri metodi.

```
String nomeGiocatore = "Arthas";
String messaggio = "Livello " + 5;  // concatenazione con +
```

Puoi chiedere a una stringa di fare cose:

```
String s = "Hello, World!";

int lunghezza = s.length();           // 13
String maiuscolo = s.toUpperCase();   // "HELLO, WORLD!"
String minuscolo = s.toLowerCase();   // "hello, world!"
boolean inizia = s.startsWith("He"); // true
boolean contiene = s.contains("World"); // true
String rimpiazza = s.replace("World", "Java"); // "Hello, Java!"
String parte = s.substring(7, 12);   // "World"
int posizione = s.indexOf("World");  // 7
String pulita = "  spazi  ".trim();  // "spazi"
```

> ℹ Perché String con la maiuscola?
>
> Per distinguerla visivamente dai tipi primitivi. In Java la convenzione è: - **minuscolo** → tipo primitivo (`int`, `double`, `boolean`…) - **Maiuscola** → classe o oggetto (`String`, `Scanner`, `Personaggio`…)

3.6 Confrontare le stringhe: `==` vs `equals()`

Questo è uno degli errori più comuni per chi viene dal C.

```
String a = "ciao";
String b = "ciao";

// NON fare così:
if (a == b) { ... }      // confronta i riferimenti, non il contenuto!
```

```
// Fare così:
if (a.equals(b)) { ... } // confronta il contenuto - questo è corretto
```

`==` confronta se due variabili puntano **allo stesso oggetto in memoria**. `equals()` confronta se il **contenuto** delle due stringhe è uguale.

> ⚠ Regola d'oro per le stringhe
>
> Per confrontare stringhe usa **sempre** `equals()`, mai `==`.
> Se vuoi confrontare ignorando maiuscole/minuscole:
>
> ```
> if (a.equalsIgnoreCase(b)) { ... }  // "Ciao" == "ciao" → true
> ```

3.7 Il package `java.lang`

Java ha una libreria standard enorme. Il package **`java.lang`** è quello di base — viene importato automaticamente in ogni programma, senza bisogno di scrivere `import`.

Le classi più utili:

Classe	A cosa serve
`String`	Gestione testi
`Math`	Funzioni matematiche
`Integer`, `Double`	Convertire tipi primitivi in oggetti (wrapper)
`Object`	La superclasse di tutte le classi Java

Qualche metodo utile di `Math`:

```
Math.abs(-5)          // valore assoluto → 5
Math.max(10, 20)      // massimo → 20
Math.min(10, 20)      // minimo → 10
Math.pow(2, 10)       // potenza → 1024.0
Math.sqrt(16)         // radice quadrata → 4.0
Math.round(3.7)       // arrotondamento → 4
Math.random()         // numero casuale tra 0.0 e 1.0
```

Per generare un numero intero casuale tra 0 e N:

```
int casuale = (int)(Math.random() * N);
```

3.8 Le classi wrapper

Ogni tipo primitivo ha una corrispondente **classe wrapper** con la maiuscola:

Primitivo	Wrapper
`int`	`Integer`
`double`	`Double`
`boolean`	`Boolean`
`char`	`Character`

Le wrapper sono utili soprattutto per convertire tra tipi:

```
// Da stringa a numero
int n = Integer.parseInt("42");
double d = Double.parseDouble("3.14");

// Da numero a stringa
String s = Integer.toString(42);
String s2 = String.valueOf(3.14);

// Costanti utili
int max = Integer.MAX_VALUE;  // 2147483647
int min = Integer.MIN_VALUE;  // -2147483648
```

Autoboxing

Java converte automaticamente tra primitivi e wrapper quando necessario — questa si chiama **autoboxing**:

```
Integer x = 42;   // autoboxing: int → Integer
int y = x;        // unboxing: Integer → int
```

Non devi farlo a mano, Java lo gestisce da solo.

3.9 Input da tastiera con Scanner

Per leggere input dall'utente si usa la classe `Scanner`, che si trova nel package `java.util` — quindi va importata:

```
import java.util.Scanner;

public class InputEsempio {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Inserisci il tuo nome: ");
        String nome = sc.nextLine();

        System.out.print("Inserisci la tua età: ");
        int eta = sc.nextInt();

        System.out.println("Ciao " + nome + ", hai " + eta + " anni.");

        sc.close();
    }
}
```

I metodi principali di `Scanner`:

Metodo	Legge
`nextLine()`	Una riga intera (String)
`next()`	Una parola (String)
`nextInt()`	Un intero
`nextDouble()`	Un decimale
`nextBoolean()`	Un booleano

Il problema del `nextLine()` dopo `nextInt()`

`nextInt()` legge il numero ma lascia nel buffer il carattere `\n` (invio). Se subito dopo chiami `nextLine()`, legge quella riga vuota invece di aspettare l'input.
La soluzione: aggiungi un `sc.nextLine()` extra per "consumare" il carattere rimasto.

```
int eta = sc.nextInt();
sc.nextLine();          // consuma il \n rimasto nel buffer
String nome = sc.nextLine();  // ora legge correttamente
```

3.10 Esempio completo: scheda personaggio

Mettiamo tutto insieme con un esempio concreto:

```
import java.util.Scanner;

public class Scheda {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("=== CREA IL TUO PERSONAGGIO ===\n");

        System.out.print("Nome: ");
        String nome = sc.nextLine();

        System.out.print("Classe (guerriero/mago/arciere): ");
        String classe = sc.nextLine();

        System.out.print("Livello iniziale: ");
        int livello = sc.nextInt();

        // Calcola vita iniziale in base al livello
        int vita = livello * 20 + 80;

        System.out.println("\n--- SCHEDA PERSONAGGIO ---");
        System.out.println("Nome:    " + nome.toUpperCase());
        System.out.println("Classe:  " + classe);
        System.out.println("Livello: " + livello);
        System.out.println("Vita:    " + vita);
        System.out.println("Pronto per l'avventura!");

        sc.close();
```

```
    }
}
```

3.11 Riepilogo

I concetti chiave

- Java ha **8 tipi primitivi**: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`.
- I tipi primitivi contengono direttamente il valore; gli oggetti contengono un **riferimento**.
- Il **widening casting** è automatico; il **narrowing casting** richiede le parentesi esplicite.
- `String` è una **classe**, non un tipo primitivo — si scrive con la maiuscola e ha metodi propri.
- Per confrontare stringhe usa sempre **`equals()`**, mai `==`.
- Il package **`java.lang`** (String, Math, Integer…) è importato automaticamente.
- Per leggere input da tastiera si usa **`Scanner`** da `java.util`.

Part II

OOP

4 Paradigmi e oggetti

4.1 Un nuovo modo di pensare

Passare dal C a Java non è solo imparare una nuova sintassi. È un **cambio di mentalità**.

Nel C il focus è sulla macchina:

> *"Cosa deve fare il computer, passo dopo passo?"*

Nella programmazione orientata agli oggetti (OOP) il focus si sposta sul problema:

> *"Quali entità esistono nel mio problema? Come interagiscono?"*

Un programma non è più una lista di comandi, ma un modo per **modellare la realtà**.

4.2 Il paradigma imperativo

Un programma imperativo è una **sequenza di istruzioni** che modificano lo stato della memoria. Il C che hai usato finora funziona così:

```c
// Gestione di un personaggio - approccio imperativo in C
char nome[] = "Kratos";
int vita = 100;
int attacco = 25;
int livello = 1;

vita -= 30;    // subisce danno
livello++;     // sale di livello
attacco += 5;  // l'attacco aumenta

printf("%s - Vita: %d, Livello: %d\n", nome, vita, livello);
```

Il programma dice alla macchina **esattamente cosa fare**, in quale ordine, su quali variabili.

> **Focus del paradigma imperativo**
>
> Il programmatore pensa in termini di **stato** (i valori in memoria) e **trasformazioni** (le operazioni che li modificano). Il computer esegue tutto fedelmente, passo dopo passo.

4.3 I limiti del paradigma imperativo

Su programmi piccoli tutto funziona. Ma appena la complessità cresce, emergono problemi seri.

Complessità difficile da gestire. Con decine di personaggi, nemici, oggetti e livelli, capire "chi modifica cosa" diventa un incubo.

Poco riuso del codice. Le funzioni sono spesso scritte per un caso specifico e difficili da riutilizzare.

Manutenzione complicata. Cambiare una parte del codice può romperne un'altra in modo imprevedibile.

Il risultato è lo **spaghetti code**: variabili globali accessibili da chiunque, funzioni sparse, nessuna organizzazione.

> **Il problema dello spaghetti code**
>
> ```
> int vita_giocatore = 100;
> int vita_nemico = 80;
> int punteggio = 0;
>
> void attacca() {
>  vita_nemico -= 25;  // chiunque può modificare vita_nemico!
>  punteggio += 10;
> }
>
> void subisci_danno() {
>  vita_giocatore -= 30; // nessun controllo: vita può diventare negativa
> }
> ```
>
> Niente impedisce a una funzione qualsiasi di portare `vita_giocatore` a -500. Con decine di variabili globali il codice diventa fragile e impossibile da mantenere.

4.4 Il paradigma orientato agli oggetti

L'OOP propone una soluzione diversa: modellare il software come un insieme di **oggetti** che interagiscono tra loro.

Ogni oggetto:

- rappresenta un'**entità** del problema reale
- ha dei **dati** che lo descrivono — chiamati *attributi*
- sa compiere certe **azioni** — chiamate *metodi*
- comunica con gli altri oggetti scambiandosi **messaggi**

> Un'analogia utile
>
> Pensa a un videogioco multiplayer come un **mondo abitato da oggetti**: ogni giocatore, ogni nemico, ogni arma, ogni livello è un oggetto. Ognuno ha le sue caratteristiche e sa fare certe cose. Interagiscono tra loro, ma ognuno gestisce il proprio stato in autonomia.

4.5 I tre pilastri dell'OOP

L'OOP si regge su tre concetti fondamentali che esploreremo nelle prossime lezioni:

> 1 — Incapsulamento
>
> Dati e metodi sono racchiusi dentro l'oggetto. I dettagli interni sono nascosti; si interagisce solo attraverso l'interfaccia pubblica.

> 2 — Ereditarietà
>
> Una classe può derivare da un'altra, ereditandone attributi e metodi. Permette di riusare il codice e organizzare le classi in gerarchie.

> 3 — Polimorfismo
>
> La stessa azione può comportarsi in modo diverso a seconda dell'oggetto che la esegue. Stessa interfaccia, comportamenti differenti.

4.6 Classi e oggetti

Gli oggetti vengono creati a partire da uno **schema** chiamato **classe**.

Una classe è come la **scheda tipo di un oggetto**. Nell'esempio del gioco la scheda di un personaggio descrive quali statistiche ha ogni personaggio di quel tipo e cosa sa fare. La scheda non è il personaggio — è il modello da cui si creano tanti personaggi diversi, ognuno con i propri valori.

	Descrizione	Esempio
Classe	Il modello, lo schema generale	`Personaggio`
Oggetto	La realizzazione concreta	`eroe`, `nemico`, `boss`

> **Definizione**
>
> Una **classe** è un modello che descrive attributi e metodi di un tipo di oggetto. Un **oggetto** è un'istanza concreta di una classe, con i propri valori.

4.7 Il primo oggetto in Java

Vediamo come si traduce tutto questo in codice. Modelliamo un personaggio di un videogioco.

Cosa descrive un personaggio? `nome`, `vita`, `attacco`, `livello`

Cosa sa fare un personaggio? Subire danno, attaccare, salire di livello.

```
public class Personaggio {

    // Attributi - descrivono lo stato dell'oggetto
    String nome;
    int vita;
    int attacco;
    int livello;

    // Metodi - descrivono il comportamento
```

```java
    void subirDanno(int danno) {
        vita -= danno;
        if (vita < 0) vita = 0;
        System.out.println(nome + " subisce " + danno +
            " danni. Vita rimasta: " + vita);
    }

    void attacca(Personaggio bersaglio) {
        System.out.println(nome + " attacca " + bersaglio.nome + "!");
        bersaglio.subirDanno(attacco);
    }

    void salaDiLivello() {
        livello++;
        attacco += 5;
        vita += 20;
        System.out.println(nome + " sale al livello " + livello + "!");
    }
}
```

4.8 Creare e usare oggetti

Per creare un oggetto si usa la parola chiave `new`:

```java
Personaggio eroe = new Personaggio();
eroe.nome = "Kratos";
eroe.vita = 100;
eroe.attacco = 25;
eroe.livello = 1;

Personaggio nemico = new Personaggio();
nemico.nome = "Drago";
nemico.vita = 200;
nemico.attacco = 40;
nemico.livello = 5;
```

`new` fa tre cose: 1. **Alloca** la memoria necessaria per l'oggetto 2. **Inizializza** gli attributi con valori di default 3. **Restituisce** un riferimento all'oggetto appena creato

Per accedere ad attributi e metodi si usa la **notazione punto**:

```
eroe.attacca(nemico);   // "Kratos attacca Drago! Drago subisce 25 danni."
nemico.attacca(eroe);   // "Drago attacca Kratos! Kratos subisce 40 danni."
eroe.salaDiLivello();   // "Kratos sale al livello 2!"
```

I due oggetti sono **indipendenti**: modificare la vita di uno non influenza l'altro.

4.9 Confronto diretto: C vs Java

	Paradigma imperativo (C)	Paradigma OOP (Java)
Focus	Come fa la macchina	Quali entità esistono
Unità base	Funzione + variabile	Oggetto (dati + metodi)
Dati	Spesso globali	Incapsulati nell'oggetto
Organizzazione	Funzioni sparse	Classi e gerarchie
Riuso	Difficile	Tramite ereditarietà
Scalabilità	Limitata	Alta

4.10 Riepilogo

I concetti chiave di questa lezione

- Il paradigma **imperativo** si concentra su *come* fare le cose passo per passo.
- Il paradigma **OOP** si concentra su *quali entità* esistono e come interagiscono.
- Una **classe** è il modello; un **oggetto** è la realizzazione concreta.
- Ogni oggetto ha **attributi** (dati) e **metodi** (comportamenti).
- `new` crea un oggetto in memoria e restituisce un riferimento ad esso.
- La **notazione punto** permette di accedere ad attributi e metodi: `oggetto.metodo()`.
- I tre pilastri dell'OOP sono **incapsulamento**, **ereditarietà** e **polimorfismo**.

5 Creare una classe

5.1 Da struct a class

In C avevi le **struct** per raggruppare dati correlati. Una struct è un contenitore di variabili — niente di più. Le funzioni che operano su quei dati stanno altrove, separate.

```
// In C
struct Personaggio {
    char nome[50];
    int vita;
    int attacco;
};

// La funzione sta fuori dalla struct
void subirDanno(struct Personaggio* p, int danno) {
    p->vita -= danno;
}
```

In Java la **classe** fa un passo avanti decisivo: raccoglie insieme i dati **e** i metodi che li gestiscono. Dati e comportamenti sono un'unica cosa coerente.

	C (`struct`)	Java (`class`)
Dati		
Metodi	(separati)	(dentro la classe)
Accesso controllato		(public/private)
Costruttore		

5.2 Struttura di una classe Java

Una classe ben scritta ha sempre questa struttura:

```
public class NomeClasse {

    // 1. Attributi (privati)
    private tipo attributo1;
    private tipo attributo2;

    // 2. Costruttore/i
    public NomeClasse(parametri) {
        // inizializza gli attributi
    }

    // 3. Getter e setter
    public tipo getAttributo1() { return attributo1; }
    public void setAttributo1(tipo valore) { attributo1 = valore; }

    // 4. Altri metodi
    public void metodo() { ... }
}
```

Vediamo ogni parte nel dettaglio.

5.3 Attributi e incapsulamento

Gli **attributi** sono le variabili che descrivono lo stato dell'oggetto. Per convenzione si dichiarano sempre **`private`** — non possono essere modificati direttamente dall'esterno.

```
public class Personaggio {
    private String nome;
    private int vita;
    private int vitaMax;
    private int attacco;
    private int livello;
}
```

Perché `private`? Perché senza controllo, chiunque potrebbe scrivere:

```
eroe.vita = -9999;   // vita negativa - stato non valido!
eroe.livello = 999;  // livello impossibile
```

Con gli attributi privati, questi accessi diretti sono **vietati dal compilatore**. I dati sono al sicuro dentro l'oggetto.

> **La regola d'oro**
>
> **Attributi `private`, metodi `public`.**
> I dati sono nascosti. Il comportamento è pubblico. Chi usa la classe interagisce solo attraverso i metodi — non tocca mai i dati direttamente.

5.4 Il costruttore

Il **costruttore** è un metodo speciale che viene chiamato automaticamente quando si crea un oggetto con `new`. Il suo compito è **inizializzare gli attributi** in modo che l'oggetto parta sempre da uno stato valido.

Caratteristiche del costruttore: - Ha lo **stesso nome della classe** - **Non ha tipo di ritorno** (nemmeno `void`) - Viene chiamato automaticamente da `new`

```
public class Personaggio {
    private String nome;
    private int vita;
    private int vitaMax;
    private int attacco;
    private int livello;

    // Costruttore
    public Personaggio(String nome, int vitaMax, int attacco, int livello) {
        this.nome = nome;
        this.vitaMax = vitaMax;
        this.vita = vitaMax;   // parte con vita piena
        this.attacco = attacco;
        this.livello = livello;
    }
}
```

Ora creare un oggetto è immediato e sicuro:

```
Personaggio eroe = new Personaggio("Kratos", 100, 25, 1);
Personaggio nemico = new Personaggio("Drago", 200, 40, 5);
```

5.5 La parola chiave `this`

Nel costruttore hai visto `this.nome = nome`. A cosa serve `this`?

`this` è un riferimento all'**oggetto corrente** — quello su cui il metodo sta operando. Serve per distinguere tra il parametro del costruttore e l'attributo della classe quando hanno lo stesso nome.

```
public Personaggio(String nome, int vitaMax) {
    this.nome = nome;       // this.nome = attributo della classe
                            //       nome = parametro del costruttore
    this.vitaMax = vitaMax;
    this.vita = vitaMax;
}
```

Senza `this`, Java non saprebbe se `nome` si riferisce all'attributo o al parametro. Con `this` è esplicito: stai parlando dell'attributo dell'oggetto corrente.

> **Quando usare `this`**
>
> `this` è obbligatorio quando il parametro ha lo stesso nome dell'attributo. Molti sviluppatori lo usano sempre per chiarezza, anche quando non strettamente necessario — rende immediatamente chiaro che stai operando su un attributo dell'oggetto.

5.6 Tipi di costruttori

Una classe può avere **più costruttori** con parametri diversi — è l'overloading che abbiamo già visto per i metodi. Il compilatore sceglie quale usare in base agli argomenti passati.

Costruttore vuoto — inizializza con valori di default:

```java
public Personaggio() {
    this.nome = "Sconosciuto";
    this.vitaMax = 100;
    this.vita = 100;
    this.attacco = 10;
    this.livello = 1;
}
```

Costruttore con parametri — i valori vengono scelti al momento della creazione:

```java
public Personaggio(String nome, int vitaMax, int attacco, int livello) {
    this.nome = nome;
    this.vitaMax = vitaMax;
    this.vita = vitaMax;
    this.attacco = (attacco > 0) ? attacco : 10;
    this.livello = (livello > 0) ? livello : 1;
}
```

Costruttore di copia — crea un nuovo oggetto identico a uno esistente:

```java
public Personaggio(Personaggio altro) {
    this.nome = altro.nome;
    this.vitaMax = altro.vitaMax;
    this.vita = altro.vita;
    this.attacco = altro.attacco;
    this.livello = altro.livello;
}
```

Uso dei tre costruttori:

```java
Personaggio p1 = new Personaggio();                        // vuoto
Personaggio p2 = new Personaggio("Arthas", 150, 35, 3);    // con parametri
Personaggio p3 = new Personaggio(p2);                      // copia di p2
```

5.6.1 Overloading

Hai appena visto tre costruttori con lo stesso nome Personaggio ma parametri diversi. Questo meccanismo si chiama overloading — e non vale solo per i costruttori, ma per qualsiasi metodo. L'overloading permette di definire più metodi con lo stesso nome nella stessa classe, a patto che abbiano firme diverse — cioè numero o tipo di parametri diversi. Il compilatore sceglie quale versione chiamare in base agli argomenti passati.

> **Note**
>
> Le regole dell'overloading
> I metodi devono avere lo stesso nome Devono differire per numero e/o tipo di parametri Il tipo di ritorno da solo non basta — due metodi con stessa firma e tipi di ritorno diversi danno errore di compilazione L'overloading è risolto a compile time — il compilatore sa già quale versione chiamare

---

5.7 Getter e setter

Con gli attributi privati, come si leggono e modificano i dati dall'esterno? Tramite metodi appositi:

- **Getter** — restituisce il valore di un attributo (`get` + nome attributo)
- **Setter** — modifica il valore, con eventuali controlli (`set` + nome attributo)

```java
public class Personaggio {
    private String nome;
    private int vita;
    private int vitaMax;
    private int attacco;
    private int livello;

    // Getter
    public String getNome()   { return nome; }
    public int getVita()      { return vita; }
    public int getVitaMax()   { return vitaMax; }
    public int getAttacco()   { return attacco; }
    public int getLivello()   { return livello; }

    // Setter con controlli
    public void setNome(String nome) {
        if (nome != null && !nome.isEmpty()) {
            this.nome = nome;
        }
    }

    public void setVita(int vita) {
```

```
        if (vita < 0) vita = 0;
        if (vita > vitaMax) vita = vitaMax;
        this.vita = vita;
    }

    public void setAttacco(int attacco) {
        if (attacco > 0) this.attacco = attacco;
    }
}
```

I setter sono le **porte d'ingresso controllate** ai dati. Possono contenere regole — ad esempio impedire vita negativa o attacco nullo — garantendo che l'oggetto mantenga sempre uno stato valido.

> **!** Non tutti gli attributi hanno bisogno di un setter
>
> Se un attributo non deve mai cambiare dopo la creazione (ad esempio il nome di un personaggio in certi giochi), puoi fornire solo il getter e nessun setter. Così l'attributo diventa di sola lettura dall'esterno.

5.8 La classe completa

Mettiamo tutto insieme: attributi privati, costruttori, getter, setter e metodi di comportamento.

```
public class Personaggio {

    // Attributi
    private String nome;
    private int vita;
    private int vitaMax;
    private int attacco;
    private int livello;

    // Costruttore vuoto
    public Personaggio() {
        this.nome = "Sconosciuto";
        this.vitaMax = 100;
```

```java
        this.vita = 100;
        this.attacco = 10;
        this.livello = 1;
    }

    // Costruttore con parametri
    public Personaggio(String nome, int vitaMax, int attacco, int livello) {
        this.nome = nome;
        this.vitaMax = (vitaMax > 0) ? vitaMax : 100;
        this.vita = this.vitaMax;
        this.attacco = (attacco > 0) ? attacco : 10;
        this.livello = (livello > 0) ? livello : 1;
    }

    // Costruttore di copia
    public Personaggio(Personaggio altro) {
        this.nome = altro.nome;
        this.vitaMax = altro.vitaMax;
        this.vita = altro.vita;
        this.attacco = altro.attacco;
        this.livello = altro.livello;
    }

    // Getter
    public String getNome()  { return nome; }
    public int getVita()     { return vita; }
    public int getVitaMax()  { return vitaMax; }
    public int getAttacco()  { return attacco; }
    public int getLivello()  { return livello; }

    // Setter
    public void setNome(String nome) {
        if (nome != null && !nome.isEmpty()) this.nome = nome;
    }

    public void setVita(int vita) {
        if (vita < 0) vita = 0;
        if (vita > vitaMax) vita = vitaMax;
        this.vita = vita;
    }

    public void setAttacco(int attacco) {
```

```java
        if (attacco > 0) this.attacco = attacco;
    }

    // Metodi di comportamento
    public void subirDanno(int danno) {
        setVita(vita - danno);
        System.out.println(nome + " subisce " + danno +
            " danni. Vita: " + vita + "/" + vitaMax);
    }

    public void attacca(Personaggio bersaglio) {
        System.out.println(nome + " attacca " + bersaglio.getNome() + "!");
        bersaglio.subirDanno(attacco);
    }

    public void salaDiLivello() {
        livello++;
        vitaMax += 20;
        setVita(vitaMax);  // vita piena al cambio livello
        attacco += 5;
        System.out.println(nome + " sale al livello " + livello +
            "! Vita: " + vita + " | Attacco: " + attacco);
    }

    public boolean isMorto() {
        return vita == 0;
    }

    public void stampaScheda() {
        System.out.println("=== " + nome + " ===");
        System.out.println("Livello:  " + livello);
        System.out.println("Vita:     " + vita + "/" + vitaMax);
        System.out.println("Attacco:  " + attacco);
        System.out.println("Stato:    " + (isMorto() ? "morto" : "in vita"));
    }
}
```

5.9 Usare la classe

```java
public class Main {
    public static void main(String[] args) {

        // Creo due personaggi con il costruttore con parametri
        Personaggio eroe = new Personaggio("Arthas", 120, 30, 1);
        Personaggio boss = new Personaggio("Lich King", 300, 50, 10);

        eroe.stampaScheda();
        System.out.println();
        boss.stampaScheda();

        System.out.println("\n--- COMBATTIMENTO ---\n");

        eroe.attacca(boss);
        boss.attacca(eroe);
        eroe.salaDiLivello();
        eroe.attacca(boss);

        System.out.println();
        eroe.stampaScheda();
        System.out.println();
        boss.stampaScheda();
    }
}
```

Output previsto:

```
=== Arthas ===
Livello:  1
Vita:     120/120
Attacco:  30
Stato:    in vita

=== Lich King ===
Livello:  10
Vita:     300/300
Attacco:  50
Stato:    in vita
```

```
--- COMBATTIMENTO ---

Arthas attacca Lich King!
Lich King subisce 30 danni. Vita: 270/300
Lich King attacca Arthas!
Arthas subisce 50 danni. Vita: 70/120
Arthas sale al livello 2! Vita: 140 | Attacco: 35
Arthas attacca Lich King!
Lich King subisce 35 danni. Vita: 235/300

=== Arthas ===
Livello:  2
Vita:     140/140
Attacco:  35
Stato:    in vita

=== Lich King ===
Livello:  10
Vita:     235/300
Attacco:  50
Stato:    in vita
```

5.10 Riepilogo

I concetti chiave

- In Java la **classe** raccoglie insieme dati (attributi) e comportamenti (metodi) — a differenza della `struct` del C.
- Gli attributi si dichiarano **`private`** per proteggerli da accessi non controllati.
- Il **costruttore** inizializza l'oggetto al momento della creazione con `new`. Ha lo stesso nome della classe e nessun tipo di ritorno.
- **`this`** è un riferimento all'oggetto corrente, utile per distinguere attributi e parametri con lo stesso nome.
- Una classe può avere **più costruttori** con parametri diversi (overloading).
- I **getter** leggono gli attributi; i **setter** li modificano con eventuali controlli.
- La regola d'oro: **attributi `private`, metodi `public`**.

6 Incapsulamento e ADT

6.1 Riprendiamo da dove eravamo

Abbiamo visto che dichiarare gli attributi `private` impedisce accessi non controllati dall'esterno.

Ma perché è così importante? E c'è un nome per questo principio?

Sì: si chiama **incapsulamento**, ed è uno dei tre pilastri dell'OOP.

6.2 Il problema degli attributi pubblici

Prima di parlare della soluzione, guardiamo ancora una volta il problema. Se gli attributi fossero pubblici:

```
public class Personaggio {
    public String nome;
    public int vita;
    public int vitaMax;
    public int attacco;
    public int livello;
}
```

Chiunque potrebbe scrivere queste cose dall'esterno:

```
eroe.vita = -9999;      // vita negativa - il personaggio è "morto" ma non lo sa
eroe.vitaMax = 0;       // vitaMax a zero - nessun senso
eroe.livello = -5;      // livello negativo - impossibile
eroe.attacco = 0;       // personaggio inutile
```

L'oggetto entra in uno **stato inconsistente** — una configurazione che non ha senso nel contesto del problema. E nessuno lo impedisce.

> ⚠ Stato inconsistente
>
> Uno stato inconsistente è quando un oggetto contiene valori che violano le regole del dominio: vita negativa, livello zero, nome vuoto. Con attributi pubblici è impossibile garantire che queste situazioni non accadano.

6.3 Incapsulamento

L'**incapsulamento** è il principio per cui i dati interni di un oggetto sono **nascosti** e accessibili solo attraverso metodi controllati.

In pratica: - Gli attributi sono `private` — invisibili dall'esterno - I metodi sono `public` — l'interfaccia attraverso cui si interagisce - I setter contengono **controlli** che garantiscono la validità dei dati

```java
public void setVita(int vita) {
    if (vita < 0) vita = 0;
    if (vita > vitaMax) vita = vitaMax;
    this.vita = vita;
}

public void setLivello(int livello) {
    if (livello > 0) this.livello = livello;
}
```

Ora è impossibile portare la vita sotto zero o il livello a un valore negativo — il setter lo impedisce silenziosamente.

> 💡 L'analogia del controller
>
> Pensa a un controller di una console: hai dei pulsanti (l'interfaccia pubblica) e non ti interessa cosa succede dentro i circuiti quando li premi. Sai solo che premendo X salti, premendo B attacchi.
> Allo stesso modo, chi usa un oggetto `Personaggio` non deve sapere come sono gestiti vita e livello internamente, usa i metodi pubblici e ottiene i risultati.

6.4 Information hiding

L'incapsulamento realizza un principio più generale chiamato **information hiding**: l'oggetto espone solo ciò che serve all'esterno e nasconde tutti i dettagli interni.

I vantaggi sono concreti:

Sicurezza — i dati non possono essere corrotti dall'esterno. Nessuno può portare `vita` a -9999.

Flessibilità — puoi cambiare completamente l'implementazione interna senza che il resto del programma se ne accorga. Se decidi di calcolare la vita in modo diverso, basta modificare i metodi interni — chi usa la classe non deve cambiare nulla.

Leggibilità — chi usa la classe sa esattamente cosa può e non può fare. L'interfaccia pubblica è il contratto.

6.5 Tipi di dato astratti (ADT)

Arriviamo ora al concetto teorico che sta dietro a tutto questo: il **tipo di dato astratto** (ADT — Abstract Data Type).

Finora hai usato tipi già pronti: `int`, `double`, `String`, `boolean`. Ma con le classi puoi **definire i tuoi tipi**, costruiti su misura per il problema che vuoi risolvere.

Un ADT è definito da due cose:

- i **valori possibili** — le configurazioni valide per quel tipo
- le **operazioni disponibili** — le azioni che hanno senso su quei valori

> **Definizione**
>
> Un tipo di dato astratto descrive **cosa** un tipo rappresenta e **cosa** si può fare con esso, senza specificare **come** è implementato internamente.

L'idea chiave è la separazione tra **interfaccia** e **implementazione**: chi usa un ADT non deve sapere come funziona dentro, gli basta sapere cosa può fare.

6.6 Il Personaggio come ADT

Pensiamo al nostro `Personaggio` come ADT.

Valori possibili: Un personaggio valido ha un nome non vuoto, una vita tra 0 e `vitaMax`, un livello positivo, un attacco positivo.

Operazioni disponibili: - creare un personaggio con nome e statistiche iniziali - farlo subire danno - farlo attaccare un bersaglio - farlo salire di livello - leggere le sue statistiche

Non ci interessa *come* queste operazioni sono implementate. Ci interessa *cosa* fanno e *quali valori accettano*.

```
// Chi usa la classe sa solo questo - l'interfaccia:
Personaggio eroe = new Personaggio("Kratos", 120, 30, 1);
eroe.subirDanno(25);
eroe.attacca(nemico);
eroe.salaDiLivello();
int v = eroe.getVita();
```

I dettagli interni ad esempio come viene calcolata la vita o come funziona il setter, sono irrilevanti per chi usa la classe.

> **ADT e cambiamento**
>
> Possiamo cambiare completamente il modo in cui calcoliamo il danno o gestiamo la vita, senza che il resto del programma se ne accorga, purché l'interfaccia pubblica rimanga la stessa. Questa è la potenza dell'astrazione.

6.7 Un secondo esempio: l'arma come ADT

Modelliamo un'arma che si consuma con l'uso.

Valori possibili: nome non vuoto, danno positivo, durabilità tra 0 e 100.

Operazioni: usare l'arma, ripararla, leggere il danno attuale.

```java
public class Arma {

    private String nome;
    private int dannoBase;
    private int durabilita;

    public Arma(String nome, int dannoBase) {
        this.nome = nome;
        this.dannoBase = (dannoBase > 0) ? dannoBase : 1;
        this.durabilita = 100;
    }

    public String getNome()      { return nome; }
    public int getDurabilita()   { return durabilita; }

    // Il danno cala proporzionalmente alla durabilità
    public int getDanno() {
        return (dannoBase * durabilita) / 100;
    }

    public void usa() {
        if (durabilita > 0) {
            durabilita--;
            System.out.println(nome + " usata. Durabilità: " + durabilita + "/100");
        } else {
            System.out.println(nome + " è rotta! Ripararla prima di usarla.");
        }
    }

    public void ripara() {
        durabilita = 100;
        System.out.println(nome + " riparata! Durabilità: 100/100");
    }

    public boolean isRotta() {
        return durabilita == 0;
    }
}
```

Chi usa `Arma` non sa come viene calcolato `getDanno()`. Sa solo che il danno dipende dalla durabilità e questo è tutto ciò che gli serve.

6.8 Incapsulamento in azione: simulazione completa

Vediamo tutto insieme — personaggio e arma che collaborano:

```
public class Main {
    public static void main(String[] args) {

        Personaggio eroe = new Personaggio("Arthas", 150, 0, 1);
        Arma spada = new Arma("Spada del Caos", 50);

        Personaggio nemico = new Personaggio("Lich King", 300, 45, 10);

        System.out.println("=== INIZIO COMBATTIMENTO ===\n");

        // Turno 1
        System.out.println("--- Turno 1 ---");
        int danno = spada.getDanno();
        System.out.println(eroe.getNome() + " colpisce con " +
            spada.getNome() + " per " + danno + " danni!");
        nemico.subirDanno(danno);
        spada.usa();
        nemico.subirDanno(nemico.getAttacco()); // il nemico contrattacca

        System.out.println();

        // Turno 2
        System.out.println("--- Turno 2 ---");
        danno = spada.getDanno();
        System.out.println(eroe.getNome() + " colpisce con " +
            spada.getNome() + " per " + danno + " danni!");
        nemico.subirDanno(danno);
        spada.usa();

        System.out.println();
        System.out.println("Vita di " + nemico.getNome() +
            ": " + nemico.getVita() + "/" + nemico.getVitaMax());
        System.out.println("Vita di " + eroe.getNome() +
            ": " + eroe.getVita() + "/" + eroe.getVitaMax());
        System.out.println("Durabilità " + spada.getNome() +
```

```
            ": " + spada.getDurabilita() + "/100");
    }
}
```

! Osserva cosa non vedi

Nel `main` non c'è nessun accesso diretto agli attributi. Tutto passa attraverso i metodi pubblici. L'implementazione interna di `Personaggio` e `Arma` è completamente nascosta: questo è l'incapsulamento in azione.

6.9 Riepilogo

i I concetti chiave

- L'**incapsulamento** nasconde gli attributi (`private`) e li espone solo tramite metodi controllati (`public`).
- L'**information hiding** garantisce che i dettagli implementativi restino nascosti: si espone solo l'interfaccia necessaria.
- Uno **stato inconsistente** è quando un oggetto contiene valori non validi, l'incapsulamento lo previene.
- Un **ADT** definisce un tipo tramite i suoi valori possibili e le sue operazioni, senza specificare l'implementazione.
- La separazione tra **interfaccia** e **implementazione** permette di cambiare il codice interno senza impatto su chi usa la classe.
- La regola rimane: **attributi `private`, metodi `public`**.

7 Ereditarietà

7.1 Il problema della duplicazione

Immagina di dover modellare i personaggi di un videogioco. Hai diversi tipi: guerriero, mago, arciere. Ognuno ha caratteristiche proprie, ma tutti condividono alcune cose di base: un nome, dei punti vita, un livello, la capacità di subire danno e di salire di livello.

Senza ereditarietà, dovresti riscrivere queste cose **in ogni classe**:

```
public class Guerriero {
    private String nome;
    private int vita;
    private int livello;
    // ... stesse cose del Mago e dell'Arciere
    private int forza;      // specifico del Guerriero
    private int armatura;   // specifico del Guerriero
}

public class Mago {
    private String nome;    // duplicato!
    private int vita;       // duplicato!
    private int livello;    // duplicato!
    // ...
    private int mana;       // specifico del Mago
    private int intelligenza; // specifico del Mago
}
```

Codice duplicato è codice difficile da mantenere: se vuoi cambiare come funziona `subirDanno()`, devi modificarlo in ogni classe separatamente.

Il problema della duplicazione

Copiare e incollare codice tra classi è uno dei peggiori errori di progettazione. Se trovi un bug in `subirDanno()`, devi ricordarti di correggerlo in `Guerriero`, `Mago`, `Arciere`… e in ogni altro personaggio che hai creato.

7.2 La soluzione: ereditarietà

L'**ereditarietà** permette di creare una nuova classe a partire da una già esistente. La nuova classe — chiamata **classe figlia** o sottoclasse — eredita automaticamente tutti gli attributi e i metodi della **classe madre** o superclasse, e può aggiungerne di nuovi.

In Java si usa la parola chiave `extends`:

```
public class ClasseFiglia extends ClasseMadre {
    // eredita tutto dalla madre
    // può aggiungere nuovi attributi e metodi
}
```

Definizione

L'ereditarietà è un meccanismo che permette a una classe di **derivare** da un'altra, acquisendone attributi e metodi senza doverli riscrivere.

7.3 Costruiamo la gerarchia

Partiamo dalla classe madre `Personaggio`, che contiene tutto ciò che è comune a ogni tipo di personaggio:

```
public class Personaggio {

    private String nome;
    private int vita;
    private int vitaMax;
```

```java
    private int livello;

    // Getter
    public String getNome()  { return nome; }
    public int getVita()     { return vita; }
    public int getVitaMax()  { return vitaMax; }
    public int getLivello()  { return livello; }

    // Setter con controlli
    public void setNome(String n)    { nome = n; }
    public void setLivello(int l)    { if (l > 0) livello = l; }
    public void setVitaMax(int vm)   { if (vm > 0) vitaMax = vm; }
    public void setVita(int v) {
        if (v < 0) v = 0;
        if (v > vitaMax) v = vitaMax;
        vita = v;
    }

    // Metodi comuni
    public void subirDanno(int danno) {
        setVita(vita - danno);
        System.out.println(nome + " subisce " + danno +
            " danni. Vita: " + vita + "/" + vitaMax);
    }

    public boolean isMorto() {
        return vita == 0;
    }

    public void salaDiLivello() {
        livello++;
        vitaMax += 20;
        setVita(vitaMax);
        System.out.println(nome + " sale al livello " + livello + "!");
    }
}
```

Ora creiamo le classi figlie. Il `Guerriero` aggiunge forza e armatura:

```java
public class Guerriero extends Personaggio {

    private int forza;
```

```java
    private int armatura;

    public int getForza()    { return forza; }
    public int getArmatura() { return armatura; }
    public void setForza(int f)    { if (f > 0) forza = f; }
    public void setArmatura(int a) { if (a >= 0) armatura = a; }

    // Metodo specifico del Guerriero
    public void colpoPoderoso(Personaggio bersaglio) {
        int danno = forza * 2;
        System.out.println(getNome() + " usa Colpo Poderoso!");
        bersaglio.subirDanno(danno);
    }
}
```

Il `Mago` aggiunge mana e intelligenza:

```java
public class Mago extends Personaggio {

    private int mana;
    private int manaMax;
    private int intelligenza;

    public int getMana()         { return mana; }
    public int getIntelligenza() { return intelligenza; }
    public void setMana(int m)   { if (m >= 0 && m <= manaMax) mana = m; }
    public void setManaMax(int mm) { if (mm > 0) manaMax = mm; }
    public void setIntelligenza(int i) { if (i > 0) intelligenza = i; }

    // Metodo specifico del Mago
    public void lanciaFirball(Personaggio bersaglio) {
        int costomana = 30;
        if (mana >= costomana) {
            int danno = intelligenza * 3;
            System.out.println(getNome() + " lancia Fireball!");
            bersaglio.subirDanno(danno);
            setMana(mana - costomana);
        } else {
            System.out.println(getNome() + " non ha abbastanza mana!");
        }
    }
}
```

7.4 Usare la gerarchia

Creiamo i personaggi e usiamoli:

```
Guerriero tank = new Guerriero();
tank.setNome("Arthas");
tank.setVitaMax(200);
tank.setVita(200);
tank.setLivello(5);
tank.setForza(40);
tank.setArmatura(20);

Mago mago = new Mago();
mago.setNome("Gandalf");
mago.setVitaMax(100);
mago.setVita(100);
mago.setLivello(8);
mago.setManaMax(150);
mago.setMana(150);
mago.setIntelligenza(60);

Guerriero nemico = new Guerriero();
nemico.setNome("Lich King");
nemico.setVitaMax(300);
nemico.setVita(300);
nemico.setForza(50);

// Arthas usa un metodo ereditato da Personaggio
tank.subirDanno(30);

// Arthas usa un metodo specifico del Guerriero
tank.colpoPoderoso(nemico);

// Gandalf usa un metodo specifico del Mago
mago.lanciaFirball(nemico);
```

Cosa ha ereditato il Guerriero?

`Guerriero` non ha un metodo `subirDanno()` — lo eredita da `Personaggio`. Idem per `isMorto()`, `salaDiLivello()`, tutti i getter e setter di base. Non è stato necessario riscriverli.

7.5 La parola chiave `super`

A volte la classe figlia vuole **estendere** il comportamento di un metodo della madre, non sostituirlo. Si usa la parola chiave `super` per chiamare il metodo della classe madre:

```
public class Guerriero extends Personaggio {

    private int armatura;

    // Il Guerriero riduce il danno grazie all'armatura
    @Override
    public void subirDanno(int danno) {
        int dannoRidotto = Math.max(0, danno - armatura);
        System.out.println(getNome() + " blocca " +
            (danno - dannoRidotto) + " danni con l'armatura.");
        super.subirDanno(dannoRidotto); // chiama il metodo della madre
    }
}
```

Ora quando un `Guerriero` subisce danno, prima calcola la riduzione dell'armatura, poi delega il resto a `Personaggio.subirDanno()`.

7.6 Il principio "è un"

Un modo semplice per capire se l'ereditarietà ha senso è chiedersi: *"X è un Y?"*

- Un `Guerriero` **è un** `Personaggio` → ereditarietà corretta
- Un `Mago` **è un** `Personaggio` → ereditarietà corretta
- Un `Guerriero` **è una** `Arma` → ereditarietà sbagliata

Se la frase non ha senso, l'ereditarietà non va usata.

Ereditarietà non è composizione

Se un `Guerriero` **ha una** `Arma`, non deve estendere `Arma` — deve avere un attributo di tipo `Arma`. L'ereditarietà modella una relazione "è un", non "ha un".

```
public class Guerriero extends Personaggio {
    private Arma armaPrincipale;  // "ha una" Arma → attributo
}
```

7.7 La gerarchia completa

Ecco la struttura che abbiamo costruito:

```
Personaggio
  Guerriero
        metodi aggiuntivi: colpoPoderoso()
  Mago
        metodi aggiuntivi: lanciaFireball()
```

`Personaggio` definisce il contratto comune. `Guerriero` e `Mago` lo specializzano aggiungendo comportamenti propri — senza duplicare nemmeno una riga di codice comune.

7.8 Riepilogo

I concetti chiave di questa lezione

- L'**ereditarietà** permette a una classe figlia di estendere una classe madre con `extends`.
- La classe figlia eredita tutti gli attributi e metodi della madre.
- La classe figlia può aggiungere nuovi attributi e metodi specifici.
- `super` permette di chiamare i metodi della classe madre dalla figlia.
- Il test **"è un"** aiuta a capire se l'ereditarietà è appropriata.

- Non confondere ereditarietà ("è un") con composizione ("ha un").

8 Polimorfismo, upcasting e downcasting

8.1 Riprendiamo dal polimorfismo

Con l'ereditarietà abbiamo costruito la gerarchia `Personaggio` → `Guerriero`, `Mago`, `Arciere`. Ogni sottoclasse ridefinisce `attacca()` con il proprio comportamento.

Il **polimorfismo** è la capacità di oggetti di classi diverse di rispondere alla stessa chiamata in modo diverso, ognuno secondo la propria natura.

> **Definizione**
>
> Il polimorfismo permette di trattare oggetti di classi diverse attraverso un'unica interfaccia comune, lasciando che ogni oggetto risponda a modo suo.

8.2 Override: ridefinire un metodo

L'override è il meccanismo con cui una sottoclasse **ridefinisce** un metodo ereditato dalla madre. L'annotazione `@Override` dice al compilatore che stiamo intenzionalmente ridefinendo un metodo — e ci avvisa se sbagliamo il nome.

```java
public class Guerriero extends Personaggio {
    private int forza;

    @Override
    public void attacca(Personaggio bersaglio) {
        int danno = forza + 15;
        System.out.println(getNome() + " colpisce con la spada!");
        bersaglio.subirDanno(danno);
    }
```

```java
}

public class Mago extends Personaggio {
    private int intelligenza;
    private int mana;

    @Override
    public void attacca(Personaggio bersaglio) {
        if (mana >= 20) {
            int danno = intelligenza * 2;
            System.out.println(getNome() + " lancia Fireball!");
            bersaglio.subirDanno(danno);
            mana -= 20;
        } else {
            System.out.println(getNome() + " non ha mana! Attacco fisico.");
            super.attacca(bersaglio);
        }
    }
}

public class Arciere extends Personaggio {
    private int destrezza;
    private int frecce;

    @Override
    public void attacca(Personaggio bersaglio) {
        if (frecce > 0) {
            int danno = destrezza + 10;
            System.out.println(getNome() + " scocca una freccia!");
            bersaglio.subirDanno(danno);
            frecce--;
        } else {
            System.out.println(getNome() + " ha esaurito le frecce!");
            super.attacca(bersaglio);
        }
    }
}
```

8.3 Riferimenti polimorfici e il vantaggio reale

In Java una variabile di tipo `Personaggio` può contenere qualsiasi oggetto di una sua sottoclasse — perché un `Guerriero` **è un** `Personaggio`:

```
Personaggio p1 = new Guerriero(...);
Personaggio p2 = new Mago(...);
Personaggio p3 = new Arciere(...);
```

Quando chiami `attacca()` su queste variabili, Java esegue il metodo della **classe reale dell'oggetto**, non quello della variabile. Questo si chiama **binding dinamico**: la scelta avviene a runtime.

```
Personaggio p = new Mago(...);
p.attacca(nemico);  // chiama Mago.attacca(), non Personaggio.attacca()!
```

Il vero vantaggio: puoi mettere tutti i personaggi in un unico array e gestirli con un solo ciclo:

```
Personaggio[] squadra = {
    new Guerriero("Arthas", 150, 30, 1, 15),
    new Mago("Gandalf", 100, 20, 3, 120),
    new Arciere("Legolas", 120, 25, 2, 20)
};

Personaggio boss = new Guerriero("Sauron", 500, 50, 10, 0);

for (Personaggio p : squadra) {
    p.attacca(boss);  // ognuno attacca a modo suo!
}
```

> **Perché è potente**
>
> Se domani aggiungi `Paladino extends Personaggio`, non tocchi nulla nel ciclo di combattimento. Aggiungi il `Paladino` all'array e il codice lo gestisce automaticamente. Questo è il vero vantaggio del polimorfismo.

8.4 Upcasting

L'**upcasting** è la conversione di un oggetto da un tipo figlio a un tipo padre. È sempre **sicuro** e avviene **automaticamente** — Java lo fa senza bisogno di parentesi esplicite.

```
Guerriero guerriero = new Guerriero("Arthas", 150, 30, 1, 15);

// Upcasting implicito - automatico
Personaggio p = guerriero;
```

Dopo l'upcasting, attraverso la variabile `p` puoi accedere solo ai metodi di `Personaggio` — anche se l'oggetto reale è un `Guerriero`. I metodi specifici del `Guerriero` non sono visibili:

```
p.attacca(nemico);    // OK - metodo di Personaggio (eseguito come Guerriero)
p.subirDanno(10);     // OK - metodo di Personaggio
// p.colpoPoderoso(); // ERRORE - metodo specifico di Guerriero, non visibile!
```

> **Cosa succede con l'upcasting**
>
> L'oggetto in memoria è e rimane un `Guerriero`. L'upcasting cambia solo il **tipo della variabile** — la "lente" attraverso cui lo guardi. Il binding dinamico garantisce che i metodi override vengano comunque eseguiti nella versione corretta.

Questo è esattamente quello che abbiamo fatto con l'array di `Personaggio[]` — tutti gli elementi erano tecnicamente upcastati a `Personaggio`.

8.5 Downcasting

Il **downcasting** è l'operazione inversa: da tipo padre a tipo figlio. Non è automatico — richiede le **parentesi esplicite** — e può fallire a runtime se l'oggetto reale non è del tipo che ci aspettiamo.

```
Personaggio p = new Guerriero("Arthas", 150, 30, 1, 15);

// Downcasting esplicito
Guerriero g = (Guerriero) p;
g.colpoPoderoso(nemico);  // ora posso accedere ai metodi specifici!
```

Ma cosa succede se l'oggetto reale non è un `Guerriero`?

```
Personaggio p = new Mago("Gandalf", 100, 20, 3, 120);
Guerriero g = (Guerriero) p;  // ERRORE a runtime: ClassCastException!
```

Il compilatore non si accorge dell'errore: `p` è di tipo `Personaggio` e un `Guerriero` è un `Personaggio`, quindi la sintassi è valida. Ma a runtime Java controlla il tipo reale dell'oggetto e lancia un'eccezione.

> ⚠ Il downcasting è rischioso
>
> Prima di fare un downcasting, verifica sempre il tipo reale dell'oggetto con **`instanceof`**.

8.6 L'operatore `instanceof`

`instanceof` controlla se un oggetto è un'istanza di una certa classe (o di una sua sottoclasse):

```
Personaggio p = new Mago("Gandalf", 100, 20, 3, 120);

if (p instanceof Mago) {
    Mago m = (Mago) p;
    System.out.println("Mana: " + m.getMana());
} else if (p instanceof Guerriero) {
    Guerriero g = (Guerriero) p;
    g.colpoPoderoso(nemico);
}
```

8.7 `==` vs `equals()`

Già visto per le stringhe, ma vale per tutti gli oggetti. È un errore classico che vale la pena approfondire.

`==` confronta i **riferimenti**, controlla se due variabili puntano allo stesso oggetto in memoria

`equals()` confronta il **contenuto**, controlla se due oggetti sono logicamente equivalenti.

```java
String a = new String("ciao");
String b = new String("ciao");

System.out.println(a == b);        // false - sono due oggetti diversi in memoria
System.out.println(a.equals(b));   // true  - il contenuto è uguale
```

Con i tipi primitivi `==` funziona correttamente perché confronta direttamente i valori:

```java
int x = 5;
int y = 5;
System.out.println(x == y);  // true - confronto di valori, non riferimenti
```

> **!** Regola
>
> - Per i **tipi primitivi** (`int`, `double`, `boolean`...) usa sempre `==`
> - Per gli **oggetti** (`String`, `Personaggio`, qualsiasi classe) usa sempre `equals()`
> - L'unica eccezione accettabile con `==` sugli oggetti è il confronto con `null`: `if (oggetto == null)`

8.8 Implementare `equals()` in una classe

Quando scrivi le tue classi, puoi decidere cosa significa "uguale" ridefinendo `equals()`:

```java
public class Personaggio {
    private String nome;
    private int livello;
    // ...

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;           // stesso riferimento
        if (obj == null) return false;          // null non è uguale a niente
        if (!(obj instanceof Personaggio)) return false; // tipi diversi

        Personaggio altro = (Personaggio) obj;
        return this.nome.equals(altro.nome) && this.livello == altro.livello;
    }
}
```

```java
Personaggio p1 = new Personaggio("Arthas", 150, 30, 1);
Personaggio p2 = new Personaggio("Arthas", 150, 30, 1);

System.out.println(p1 == p2);        // false - oggetti diversi in memoria
System.out.println(p1.equals(p2));   // true  - stesso nome e livello
```

8.9 Riepilogo

> **I concetti chiave di questa lezione**
>
> - Il **polimorfismo** permette a oggetti di classi diverse di rispondere alla stessa chiamata in modo diverso.
> - L'**override** (`@Override`) ridefinisce un metodo ereditato nella sottoclasse.
> - Il **binding dinamico** sceglie a runtime quale versione del metodo eseguire.
> - L'**upcasting** converte da tipo figlio a tipo padre — automatico e sempre sicuro.
> - Il **downcasting** converte da tipo padre a tipo figlio — esplicito e potenzialmente rischioso.
> - Usa sempre `instanceof` prima di fare un downcasting.
> - `==` confronta i **riferimenti**; `equals()` confronta il **contenuto** — per gli oggetti usa sempre `equals()`.

9 Classi astratte e interfacce

9.1 Il problema del metodo "generico"

Nella lezione sull'ereditarietà abbiamo costruito questa gerarchia:

```
Personaggio
  Guerriero
  Mago
  Arciere
```

Ogni sottoclasse ridefinisce `attacca()` con il proprio comportamento. Ma la versione di `attacca()` in `Personaggio` (quella base) non ha molto senso da sola:

```java
public void attacca(Personaggio bersaglio) {
    System.out.println(getNome() + " attacca con un colpo generico.");
    bersaglio.subirDanno(10);
}
```

Un "attacco generico" non esiste nel nostro gioco. `Personaggio` è un concetto astratto — nella realtà esistono solo guerrieri, maghi, arcieri. Nessuno crea mai un `Personaggio` puro.

Come impediamo che qualcuno scriva `new Personaggio()`? E come forziamo ogni sottoclasse a implementare `attacca()`?

La risposta è la **classe astratta**.

9.2 Classi astratte

Una **classe astratta** è una classe che non può essere istanziata direttamente, esiste solo per essere estesa. Si dichiara con la parola chiave `abstract`.

```
public abstract class Personaggio {

    private String nome;
    private int vita;
    private int vitaMax;
    private int attacco;
    private int livello;

    public Personaggio(String nome, int vitaMax, int attacco, int livello) {
        this.nome = nome;
        this.vitaMax = (vitaMax > 0) ? vitaMax : 100;
        this.vita = this.vitaMax;
        this.attacco = (attacco > 0) ? attacco : 10;
        this.livello = (livello > 0) ? livello : 1;
    }

    // Getter
    public String getNome()  { return nome; }
    public int getVita()     { return vita; }
    public int getVitaMax()  { return vitaMax; }
    public int getAttacco()  { return attacco; }
    public int getLivello()  { return livello; }

    // Setter
    public void setVita(int vita) {
        if (vita < 0) vita = 0;
        if (vita > vitaMax) vita = vitaMax;
        this.vita = vita;
    }

    // Metodi concreti - ereditati da tutte le sottoclassi
    public void subirDanno(int danno) {
        setVita(vita - danno);
        System.out.println(nome + " subisce " + danno +
            " danni. Vita: " + vita + "/" + vitaMax);
    }
```

```java
    public boolean isMorto() { return vita == 0; }

    public void salaDiLivello() {
        livello++;
        vitaMax += 20;
        setVita(vitaMax);
        attacco += 5;
        System.out.println(nome + " sale al livello " + livello + "!");
    }

    // Metodo astratto - ogni sottoclasse DEVE implementarlo
    public abstract void attacca(Personaggio bersaglio);

    public void stampaScheda() {
        System.out.println("=== " + nome + " [" + getClass().getSimpleName() + "] ===");
        System.out.println("Livello: " + livello + " | Vita: " +
            vita + "/" + vitaMax + " | Attacco: " + attacco);
    }
}
```

> **Regole delle classi astratte**
>
> - Si dichiarano con `abstract class`
> - **Non possono essere istanziate** direttamente: `new Personaggio()` dà errore
> - Possono avere sia metodi **concreti** (con implementazione) sia metodi **astratti** (senza)
> - Ogni classe figlia **deve** implementare tutti i metodi astratti, altrimenti deve essere astratta anche lei

---

9.3 Metodi astratti

Un **metodo astratto** è un metodo senza corpo con solo la firma, seguita da `;`. Dice alle sottoclassi: *"questo metodo deve esistere, ma tocca a te decidere come funziona"*.

```java
public abstract void attacca(Personaggio bersaglio);  // nessun corpo
```

Se una sottoclasse non implementa tutti i metodi astratti, il compilatore dà errore. È un modo per **forzare un contratto**: ogni tipo concreto di personaggio deve saper attaccare.

9.4 Le sottoclassi concrete

Ora le sottoclassi non possono più ignorare `attacca()`, devono implementarlo:

```
public class Guerriero extends Personaggio {

    private int armatura;

    public Guerriero(String nome, int vitaMax, int attacco, int livello, int armatura) {
        super(nome, vitaMax, attacco, livello);
        this.armatura = (armatura >= 0) ? armatura : 0;
    }

    @Override
    public void attacca(Personaggio bersaglio) {
        int danno = getAttacco() + 10;
        System.out.println(getNome() + " colpisce con la spada!");
        bersaglio.subirDanno(danno);
    }

    @Override
    public void subirDanno(int danno) {
        int dannoRidotto = Math.max(0, danno - armatura);
        System.out.println(getNome() + " blocca " +
            (danno - dannoRidotto) + " danni con l'armatura.");
        super.subirDanno(dannoRidotto);
    }
}
```

```
public class Mago extends Personaggio {

    private int mana;
    private int manaMax;

    public Mago(String nome, int vitaMax, int attacco, int livello, int manaMax) {
        super(nome, vitaMax, attacco, livello);
        this.manaMax = manaMax;
        this.mana = manaMax;
    }
```

```java
    public int getMana() { return mana; }

    @Override
    public void attacca(Personaggio bersaglio) {
        int costoMana = 25;
        if (mana >= costoMana) {
            int danno = getAttacco() * 2;
            System.out.println(getNome() + " lancia Fireball!");
            bersaglio.subirDanno(danno);
            mana -= costoMana;
            System.out.println("Mana rimasto: " + mana + "/" + manaMax);
        } else {
            System.out.println(getNome() + " non ha mana! Attacco fisico.");
            bersaglio.subirDanno(getAttacco());
        }
    }
}
```

9.5 Interfacce

Le classi astratte risolvono il problema dell'astrazione nella gerarchia. Ma Java non supporta l'**ereditarietà multipla** — una classe può estendere una sola classe madre.

E se volessimo aggiungere capacità trasversali? Ad esempio, alcune entità del gioco possono essere **curate**, altre possono essere **salvate** su file, altre ancora possono essere **osservate** da altri oggetti.

Queste capacità non appartengono alla gerarchia `Personaggio`, sono comportamenti che classi diverse possono condividere indipendentemente.

Per questo esistono le **interfacce**.

> **Definizione**
>
> Un'interfaccia è un **contratto**: dichiara quali metodi una classe deve implementare, senza fornire nessuna implementazione. Una classe può implementare **quante interfacce vuole**.

9.6 Dichiarare un'interfaccia

```
public interface Curabile {
    void cura(int quantita);
    int getVita();
    int getVitaMax();
}
```

```
public interface Salvabile {
    String toStringSalvataggio();
    void caricaDaSalvataggio(String dati);
}
```

Un'interfaccia può anche avere **metodi default**: metodi con un'implementazione di base che le classi possono sovrascrivere o usare così com'è:

```
public interface Curabile {
    void cura(int quantita);
    int getVita();
    int getVitaMax();

    // Metodo default - implementazione di base
    default boolean isFerito() {
        return getVita() < getVitaMax();
    }
}
```

9.7 Implementare un'interfaccia

Una classe implementa un'interfaccia con la parola chiave `implements`:

```
public class Guerriero extends Personaggio implements Curabile, Salvabile {

    private int armatura;

    public Guerriero(String nome, int vitaMax, int attacco, int livello, int armatura) {
        super(nome, vitaMax, attacco, livello);
```

```java
        this.armatura = (armatura >= 0) ? armatura : 0;
    }

    @Override
    public void attacca(Personaggio bersaglio) {
        bersaglio.subirDanno(getAttacco() + 10);
    }

    // Implementazione di Curabile
    @Override
    public void cura(int quantita) {
        setVita(getVita() + quantita);
        System.out.println(getNome() + " recupera " + quantita +
            " punti vita. Vita: " + getVita() + "/" + getVitaMax());
    }

    // Implementazione di Salvabile
    @Override
    public String toStringSalvataggio() {
        return getNome() + "," + getVita() + "," +
            getVitaMax() + "," + getAttacco() + "," + getLivello();
    }

    @Override
    public void caricaDaSalvataggio(String dati) {
        String[] parti = dati.split(",");
        // parsing e ripristino dello stato...
        System.out.println("Personaggio caricato: " + parti[0]);
    }
}
```

! Una classe può implementare più interfacce

```java
public class Guerriero extends Personaggio implements Curabile, Salvabile {
//                     ↑ una sola classe madre    ↑ quante interfacce si vuole
```

Questo è il modo in cui Java simula l'ereditarietà multipla in modo sicuro.

9.8 Classe astratta vs interfaccia

Ecco le differenze principali:

	Classe astratta	Interfaccia
Ereditarietà	Una sola (`extends`)	Illimitata (`implements`)
Metodi concreti	Sì	Solo `default`
Attributi	Sì	Solo costanti
Costruttore	Sì	No
Quando usarla	Gerarchia con codice condiviso	Comportamento trasversale

> La regola pratica
>
> Usa una **classe astratta** quando le classi condividono codice e appartengono alla stessa famiglia (`Personaggio` → `Guerriero`, `Mago`).
> Usa un'**interfaccia** quando vuoi definire una capacità che classi diverse e indipendenti possono avere (`Curabile`, `Salvabile`, `Comparabile`).

9.9 Esempio completo

```java
public class Main {
    public static void main(String[] args) {

        // Non posso scrivere: new Personaggio(...) → ERRORE!
        Guerriero eroe = new Guerriero("Arthas", 150, 30, 1, 10);
        Mago mago = new Mago("Gandalf", 100, 25, 3, 120);

        eroe.stampaScheda();
        mago.stampaScheda();

        System.out.println("\n--- COMBATTIMENTO ---\n");

        mago.attacca(eroe);  // Fireball
        eroe.attacca(mago);  // Colpo con spada
```

```
        System.out.println("\n--- CURA ---\n");

        // Posso usare il tipo interfaccia come variabile
        Curabile paziente = eroe;
        if (paziente.isFerito()) {
            paziente.cura(50);
        }

        System.out.println("\n--- SALVATAGGIO ---\n");

        Salvabile salvataggio = eroe;
        String dati = salvataggio.toStringSalvataggio();
        System.out.println("Dati salvati: " + dati);
    }
}
```

Nota come `eroe` può essere trattato come `Guerriero`, come `Curabile` e come 'Salvabile: è lo stesso oggetto visto attraverso interfacce diverse. Questo è il polimorfismo applicato alle interfacce.

9.10 Riepilogo

I concetti chiave

- Una **classe astratta** non può essere istanziata, esiste solo per essere estesa.
- Un **metodo astratto** non ha corpo: ogni sottoclasse concreta deve implementarlo.
- Una **interfaccia** è un contratto puro: dichiara metodi senza implementazione (salvo i `default`).
- Una classe può estendere **una sola** classe madre ma implementare **quante interfacce vuole**.
- Usa la classe astratta per **gerarchie con codice condiviso**; usa l'interfaccia per **capacità trasversali**.
- Le interfacce permettono il **polimorfismo** anche tra classi non imparentate.

Part III

Eccezioni

10 Eccezioni

10.1 Cosa succede quando qualcosa va storto?

Fino ad ora abbiamo scritto codice che funziona, assumendo che i dati in ingresso siano sempre validi, che i file esistano, che la rete sia disponibile. Ma nella realtà non è così.

Un utente inserisce una lettera dove si aspetta un numero. Un file non esiste. La memoria finisce. La connessione cade.

Questi sono **errori a runtime**, situazioni anomale che si verificano durante l'esecuzione del programma, non durante la compilazione. Java gestisce queste situazioni con un meccanismo chiamato **gestione delle eccezioni**.

10.2 Cos'è un'eccezione

Un'**eccezione** è un oggetto che rappresenta una situazione anomala verificatasi durante l'esecuzione. Quando si verifica un errore, Java **lancia** (throws) un'eccezione e interrompe il flusso normale e cerca qualcuno che la gestisca.

Se nessuno la gestisce, il programma termina stampando un messaggio di errore chiamato **stack trace**:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Main.divide(Main.java:8)
    at Main.main(Main.java:3)
```

Lo stack trace dice: cosa è successo (`ArithmeticException: / by zero`), dove è successo (`Main.java:8`), e come ci siamo arrivati (la sequenza di chiamate).

10.3 La gerarchia delle eccezioni

In Java le eccezioni sono classi organizzate in una gerarchia, Error ed Exception derivano entrambe dalla classe Throwable

```
Throwable
  Error                    → errori gravi della JVM (non gestire)
     OutOfMemoryError
     StackOverflowError
  Exception                → situazioni anomale gestibili
      RuntimeException     → eccezioni non verificate (unchecked)
         NullPointerException
         ArrayIndexOutOfBoundsException
         ClassCastException
         ArithmeticException
         NumberFormatException
      IOException          → eccezioni verificate (checked)
          FileNotFoundException
          ...
```

> **Checked vs Unchecked**
>
> **Unchecked** (RuntimeException e sottoclassi): il compilatore non ti obbliga a gestirle. Spesso indicano bug nel codice: accesso a un indice fuori range, divisione per zero, cast errato.
> **Checked** (tutto il resto di Exception) — il compilatore ti **obbliga** a gestirle o a dichiararle. Tipicamente sono situazioni esterne al controllo del programma: file non trovato, errore di rete.

10.4 Le eccezioni più comuni

Prima di vedere come gestirle, impariamo a riconoscerle:

`NullPointerException` — accesso a un riferimento `null`:

```java
String s = null;
s.length();  // NullPointerException!
```

ArrayIndexOutOfBoundsException — indice fuori dai limiti dell'array:

```
int[] arr = new int[3];
arr[5] = 10;  // ArrayIndexOutOfBoundsException!
```

ClassCastException — downcasting errato:

```
Personaggio p = new Mago(...);
Guerriero g = (Guerriero) p;  // ClassCastException!
```

ArithmeticException — operazione aritmetica illegale:

```
int risultato = 10 / 0;  // ArithmeticException: / by zero
```

NumberFormatException — conversione di stringa non valida:

```
int n = Integer.parseInt("abc");  // NumberFormatException!
```

StackOverflowError — ricorsione infinita:

```
void infinita() { infinita(); }  // StackOverflowError!
```

10.5 Try-catch: gestire le eccezioni

Il blocco **try-catch** permette di intercettare un'eccezione e gestirla senza che il programma si blocchi:

```
try {
    // codice che potrebbe lanciare un'eccezione
} catch (TipoEccezione e) {
    // cosa fare se quell'eccezione si verifica
}
```

Esempio concreto — divisione sicura:

```java
public static int dividi(int a, int b) {
    try {
        return a / b;
    } catch (ArithmeticException e) {
        System.out.println("Errore: divisione per zero!");
        return 0;
    }
}
```

Esempio con input utente:

```java
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Inserisci un numero: ");

try {
    int n = Integer.parseInt(sc.nextLine());
    System.out.println("Hai inserito: " + n);
} catch (NumberFormatException e) {
    System.out.println("Quello non è un numero valido!");
}
```

10.6 Blocchi multipli catch

Puoi gestire più tipi di eccezione con più blocchi `catch`:

```java
try {
    String[] nomi = {"Arthas", "Gandalf"};
    String input = sc.nextLine();
    int indice = Integer.parseInt(input);
    System.out.println("Personaggio: " + nomi[indice]);

} catch (NumberFormatException e) {
    System.out.println("Inserisci un numero intero.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Indice fuori range! Scegli tra 0 e 1.");
} catch (Exception e) {
```

```
    // cattura qualsiasi altra eccezione
    System.out.println("Errore inaspettato: " + e.getMessage());
}
```

Ordine dei catch

I blocchi `catch` vengono valutati dall'alto verso il basso — il primo che corrisponde viene eseguito. Metti sempre i tipi più specifici prima di quelli più generici. Se metti `Exception` per primo, cattura tutto e i blocchi successivi non vengono mai eseguiti.

10.7 Il blocco finally

Il blocco **`finally`** viene eseguito **sempre** — sia che l'eccezione si verifichi, sia che non si verifichi. È utile per operazioni di pulizia: chiudere file, connessioni, scanner.

```
Scanner sc = new Scanner(System.in);

try {
    System.out.print("Vita del personaggio: ");
    int vita = Integer.parseInt(sc.nextLine());
    System.out.println("Vita impostata: " + vita);

} catch (NumberFormatException e) {
    System.out.println("Valore non valido!");

} finally {
    sc.close();  // viene eseguito sempre, eccezione o no
    System.out.println("Scanner chiuso.");
}
```

Try-with-resources

Da Java 7 in poi puoi usare il **try-with-resources** per chiudere automaticamente le risorse senza `finally`:

```java
try (Scanner sc = new Scanner(System.in)) {
    int n = Integer.parseInt(sc.nextLine());
    System.out.println("Numero: " + n);
} catch (NumberFormatException e) {
    System.out.println("Non è un numero!");
}
// sc viene chiuso automaticamente
```

10.8 Lanciare eccezioni con throw

Non solo puoi gestire le eccezioni, puoi anche **lanciarle** tu stesso con `throw`. Questo è utile nei setter e nei costruttori quando ricevi valori non validi:

```java
public void setVita(int vita) {
    if (vita < 0) {
        throw new IllegalArgumentException("La vita non può essere negativa: " + vita);
    }
    if (vita > vitaMax) {
        throw new IllegalArgumentException("La vita non può superare vitaMax: " + vita);
    }
    this.vita = vita;
}
```

Chi chiama questo metodo deve gestire l'eccezione (o lasciarla propagare):

```java
try {
    eroe.setVita(-100);
} catch (IllegalArgumentException e) {
    System.out.println("Errore: " + e.getMessage());
}
```

10.9 Creare eccezioni personalizzate

Puoi creare le tue eccezioni estendendo `Exception` (checked) o `RuntimeException` (unchecked):

```
// Eccezione personalizzata per il gioco
public class PersonaggioMortoException extends RuntimeException {

    public PersonaggioMortoException(String nomePersonaggio) {
        super("Il personaggio " + nomePersonaggio + " è morto e non può agire!");
    }
}
```

Usarla:

```
public void attacca(Personaggio bersaglio) {
    if (isMorto()) {
        throw new PersonaggioMortoException(getNome());
    }
    bersaglio.subirDanno(attacco);
}
```

```
try {
    fantasma.attacca(eroe);
} catch (PersonaggioMortoException e) {
    System.out.println(e.getMessage());
}
```

> Quando creare eccezioni personalizzate
>
> Crea eccezioni personalizzate quando vuoi dare un nome significativo a un errore specifico del tuo dominio. `PersonaggioMortoException` è molto più chiara di un generico `IllegalStateException`.

10.10 Propagare eccezioni con throws

Se un metodo può lanciare una **checked exception** ma non la gestisce internamente, deve propagarla nella firma con `throws`:

```java
public void caricaDaFile(String percorso) throws IOException {
    // questo metodo può lanciare IOException
    // chi lo chiama deve gestirla
}
```

Chi chiama il metodo è obbligato a gestirla:

```java
try {
    caricaDaFile("salvataggio.txt");
} catch (IOException e) {
    System.out.println("File non trovato: " + e.getMessage());
}
```

> **`throw` vs `throws`**
>
> - **`throw`** lancia effettivamente un'eccezione all'interno del codice
> - **`throws`** dichiara nella firma del metodo che potrebbe lanciare quell'eccezione

---

10.11 Esempio completo: input robusto

Mettiamo tutto insieme con un esempio realistico: un programma che chiede la vita del personaggio e non si blocca su input errati:

```java
import java.util.Scanner;

public class InputRobusto {

    public static int chiediInteroPositivo(Scanner sc, String messaggio) {
        while (true) {
            System.out.print(messaggio);
            try {
                int valore = Integer.parseInt(sc.nextLine().trim());
                if (valore <= 0) {
                    System.out.println("Il valore deve essere positivo. Riprova.");
                    continue;
                }
                return valore;
```

```
            } catch (NumberFormatException e) {
                System.out.println("Inserisci un numero intero valido. Riprova.");
            }
        }
    }

    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {

            System.out.println("=== CREA IL TUO PERSONAGGIO ===\n");

            System.out.print("Nome: ");
            String nome = sc.nextLine().trim();

            int vitaMax  = chiediInteroPositivo(sc, "Vita massima: ");
            int attacco  = chiediInteroPositivo(sc, "Attacco: ");
            int livello  = chiediInteroPositivo(sc, "Livello: ");

            System.out.println("\nPersonaggio creato:");
            System.out.println("Nome:    " + nome);
            System.out.println("Vita:    " + vitaMax);
            System.out.println("Attacco: " + attacco);
            System.out.println("Livello: " + livello);
        }
    }
}
```

10.12 Riepilogo

> **I concetti chiave**
>
> - Un'**eccezione** è un oggetto che rappresenta una situazione anomala a runtime.
> - Le eccezioni **unchecked** (`RuntimeException`) indicano spesso bug nel codice — il compilatore non obbliga a gestirle.
> - Le eccezioni **checked** (`Exception`) riguardano situazioni esterne — il compilatore obbliga a gestirle.
> - Il blocco **`try-catch`** intercetta le eccezioni e permette di gestirle.
> - Il blocco **`finally`** viene eseguito sempre — utile per chiudere risorse.

- Il **try-with-resources** chiude automaticamente le risorse al termine del blocco.
- **`throw`** lancia un'eccezione; **`throws`** la dichiara nella firma del metodo.
- Puoi creare **eccezioni personalizzate** estendendo `Exception` o `RuntimeException`.

Part IV

Collections e Stream

11 Java Collections

11.1 Il problema degli array

Gli array che conosci dal C hanno un limite fondamentale: la **dimensione è fissa**. Devi decidere quanti elementi ci sono prima di creare l'array, e non puoi cambiare idea.

```
int[] punteggi = new int[10];  // esattamente 10 elementi, né più né meno
```

Nella realtà i dati sono dinamici: una lista di giocatori connessi cresce e diminuisce, un inventario si riempie e si svuota, i risultati di una ricerca variano. Serve qualcosa di più flessibile.

Java risponde con il **Collections Framework**: un insieme di classi e interfacce per gestire collezioni di oggetti in modo dinamico, sicuro ed efficiente.

11.2 La gerarchia delle Collections

Il Collections Framework è organizzato attorno a interfacce che definiscono i comportamenti, e classi che le implementano:

```
Collection
  List          → sequenza ordinata, elementi duplicati ammessi
      ArrayList
      LinkedList
  Set           → nessun duplicato
      HashSet
      TreeSet
  Queue         → coda FIFO
       LinkedList
```

```
        PriorityQueue

Map                 → coppie chiave-valore (non estende Collection)
        HashMap
        TreeMap
```

> **Interfaccia vs implementazione**
>
> Nel Collections Framework programmi sempre verso l'**interfaccia** (`List`, `Set`, `Map`) e istanzi l'**implementazione** (`ArrayList`, `HashSet`, `HashMap`). Questo ti permette di cambiare implementazione senza modificare il resto del codice.
>
> ```
> List<String> nomi = new ArrayList<>();  // tipo dichiarato: List; tipo reale: ArrayList
> ```

11.3 Generics: collezioni tipizzate

Prima di usare le collections, capiamo i **generics**: la sintassi con le parentesi angolari `<>`.

I generics permettono di specificare il **tipo degli elementi** che una collezione può contenere. Il compilatore verifica che si inseriscano solo elementi del tipo corretto.

```
List<String> nomi = new ArrayList<>();           // solo String
List<Integer> punteggi = new ArrayList<>();      // solo Integer
List<Personaggio> squadra = new ArrayList<>();   // solo Personaggio
```

Senza generics dovresti fare cast manuale ogni volta che estrai un elemento — con generics il compilatore lo fa per te e ti avvisa subito se inserisci il tipo sbagliato.

> **Tipi primitivi e generics**
>
> I generics funzionano solo con **oggetti**, non con tipi primitivi. Usa le classi wrapper: - `List<int>` → - `List<Integer>` →
> Java si occupa dell'autoboxing automaticamente — puoi aggiungere `int` e li converte in `Integer` da solo.

11.4 ArrayList: la lista dinamica

`ArrayList` è la collezione più usata in Java. È come un array che si ridimensiona automaticamente.

```
import java.util.ArrayList;
import java.util.List;

List<String> inventario = new ArrayList<>();

// Aggiungere elementi
inventario.add("Spada del Caos");
inventario.add("Scudo di Ferro");
inventario.add("Pozione di Vita");

// Dimensione
System.out.println("Oggetti: " + inventario.size()); // 3

// Accedere a un elemento per indice
String primo = inventario.get(0);  // "Spada del Caos"

// Modificare un elemento
inventario.set(1, "Scudo Leggendario");

// Rimuovere per indice
inventario.remove(2);  // rimuove "Pozione di Vita"

// Rimuovere per valore
inventario.remove("Spada del Caos");

// Verificare se contiene un elemento
boolean haSpada = inventario.contains("Spada del Caos");  // false

// Svuotare la lista
inventario.clear();

System.out.println("Vuota? " + inventario.isEmpty());  // true
```

11.5 Iterare su una lista

Ci sono tre modi principali per scorrere una lista:

For-each — il più comune e leggibile:

```
List<String> squadra = new ArrayList<>();
squadra.add("Arthas");
squadra.add("Gandalf");
squadra.add("Legolas");

for (String nome : squadra) {
    System.out.println("- " + nome);
}
```

For classico con indice — quando hai bisogno della posizione:

```
for (int i = 0; i < squadra.size(); i++) {
    System.out.println(i + ": " + squadra.get(i));
}
```

> ⚠ Non rimuovere con for-each
>
> Non rimuovere elementi da una lista mentre la stai scorrendo con un for-each — lancia `ConcurrentModificationException`. Accumula gli elementi da rimuovere e cancellali dopo.

11.6 HashMap: la mappa chiave-valore

`HashMap` associa a ogni **chiave** un **valore**. È perfetta quando vuoi cercare dati rapidamente per un identificatore.

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> punteggi = new HashMap<>();

// Inserire coppie chiave-valore
```

```java
punteggi.put("Arthas", 1500);
punteggi.put("Gandalf", 2300);
punteggi.put("Legolas", 1800);

// Leggere un valore
int p = punteggi.get("Gandalf");  // 2300

// Verificare se una chiave esiste
boolean c = punteggi.containsKey("Arthas");  // true

// Aggiornare un valore
punteggi.put("Arthas", 1750);  // sovrascrive il valore esistente

// Rimuovere una coppia
punteggi.remove("Legolas");

// Valore di default se la chiave non esiste
int valore = punteggi.getOrDefault("Sconosciuto", 0);  // 0

// Dimensione
System.out.println("Giocatori: " + punteggi.size());
```

11.7 Iterare su una HashMap

```java
Map<String, Integer> punteggi = new HashMap<>();
punteggi.put("Arthas", 1500);
punteggi.put("Gandalf", 2300);

// Iterare su tutte le coppie
for (Map.Entry<String, Integer> entry : punteggi.entrySet()) {
    System.out.println(entry.getKey() + " → " + entry.getValue());
}

// Iterare solo sulle chiavi
for (String nome : punteggi.keySet()) {
    System.out.println(nome);
}
```

```java
// Iterare solo sui valori
for (int punti : punteggi.values()) {
    System.out.println(punti);
}
```

11.8 Collections con oggetti personalizzati

Le collections funzionano benissimo con le tue classi:

```java
List<Personaggio> squadra = new ArrayList<>();

squadra.add(new Personaggio("Arthas", 150, 30, 1));
squadra.add(new Personaggio("Gandalf", 100, 20, 3));
squadra.add(new Personaggio("Legolas", 120, 25, 2));

// Stampa tutti i personaggi
for (Personaggio p : squadra) {
    p.stampaScheda();
}

// Trova i personaggi ancora in vita
List<Personaggio> inVita = new ArrayList<>();
for (Personaggio p : squadra) {
    if (!p.isMorto()) {
        inVita.add(p);
    }
}
System.out.println("Personaggi in vita: " + inVita.size());

// Cerca un personaggio per nome
Personaggio trovato = null;
for (Personaggio p : squadra) {
    if (p.getNome().equals("Gandalf")) {
        trovato = p;
        break;
    }
}
```

11.9 `equals()` e `hashCode()`: il contratto fondamentale

Nella lezione sul polimorfismo hai visto che per confrontare oggetti si usa `equals()` invece di `==`. Ma c'è una regola fondamentale in Java che spesso viene dimenticata:

> **Se ridefinisci `equals()`, devi ridefinire anche `hashCode()`.**

Perché? Perché `HashMap` e `HashSet` usano `hashCode()` per organizzare gli oggetti in "bucket" — celle di memoria indicizzate. Quando cerchi un oggetto, Java prima calcola il suo `hashCode()` per trovare il bucket giusto, poi usa `equals()` per trovare l'oggetto esatto dentro quel bucket.

Se `equals()` dice che due oggetti sono uguali ma `hashCode()` restituisce valori diversi, `HashMap` e `HashSet` li mettono in bucket diversi e non riescono a trovarli correttamente.

```
// Senza hashCode ridefinito - comportamento SBAGLIATO
Map<Personaggio, Integer> punteggi = new HashMap<>();
Personaggio p1 = new Personaggio("Arthas", 150, 30, 1);
punteggi.put(p1, 500);

Personaggio p2 = new Personaggio("Arthas", 150, 30, 1);
// p1.equals(p2) → true (stesso nome e livello)
// ma hashCode diversi → HashMap non trova p2!
System.out.println(punteggi.get(p2));  // null - non trovato!
```

La soluzione è ridefinire entrambi in modo coerente:

```
public class Personaggio {

    private String nome;
    private int livello;
    // ...

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        if (!(obj instanceof Personaggio)) return false;

        Personaggio altro = (Personaggio) obj;
        return this.livello == altro.livello &&
```

```
                this.nome.equals(altro.nome);
    }

    @Override
    public int hashCode() {
        int risultato = nome.hashCode();
        risultato = 31 * risultato + livello;
        return risultato;
    }
}
```

Oppure, da Java 7 in poi, puoi usare `Objects.hash()` che fa il lavoro sporco per te:

```
import java.util.Objects;

@Override
public int hashCode() {
    return Objects.hash(nome, livello);  // usa gli stessi campi di equals()
}
```

> ❗ Il contratto equals/hashCode
>
> - Se `a.equals(b)` è `true`, allora `a.hashCode()` deve essere uguale a `b.hashCode()`
> - Il contrario non è obbligatorio: due oggetti con lo stesso `hashCode` possono non essere uguali (collisione)
> - Usa **sempre gli stessi campi** in `equals()` e `hashCode()`
> - Se usi `Objects.hash()`, passa gli stessi campi che controlli in `equals()`

Ora con `hashCode()` ridefinito correttamente:

```
Map<Personaggio, Integer> punteggi = new HashMap<>();
Personaggio p1 = new Personaggio("Arthas", 150, 30, 1);
punteggi.put(p1, 500);

Personaggio p2 = new Personaggio("Arthas", 150, 30, 1);
System.out.println(punteggi.get(p2));  // 500 - trovato correttamente!
```

11.10 La classe Collections (utility)

`java.util.Collections` è una classe di utilità con metodi statici per operare sulle collezioni:

```java
import java.util.Collections;

List<Integer> numeri = new ArrayList<>();
numeri.add(5);
numeri.add(2);
numeri.add(8);
numeri.add(1);

Collections.sort(numeri);          // ordina in modo crescente → [1, 2, 5, 8]
Collections.reverse(numeri);       // inverte → [8, 5, 2, 1]
Collections.shuffle(numeri);       // mescola casualmente
int max = Collections.max(numeri); // elemento massimo
int min = Collections.min(numeri); // elemento minimo
```

Per ordinare tipi primitivi e `String`, `Collections.sort()` funziona direttamente perché questi tipi implementano già `Comparable`. Per ordinare oggetti personalizzati come `Personaggio`, devi implementare `Comparable` nella tua classe — lo vediamo nella sezione successiva.

```java
// Funziona subito - String implementa già Comparable
List<String> nomi = new ArrayList<>(Arrays.asList("Legolas", "Arthas", "Gandalf"));
Collections.sort(nomi);  // ordine alfabetico → [Arthas, Gandalf, Legolas]

// Funziona dopo aver implementato Comparable in Personaggio
List<Personaggio> squadra = new ArrayList<>();
squadra.add(new Personaggio("Arthas", 150, 30, 5));
squadra.add(new Personaggio("Gandalf", 100, 20, 7));
squadra.add(new Personaggio("Legolas", 120, 25, 3));

Collections.sort(squadra);  // ordina per livello crescente (definito in compareTo)
squadra.forEach(p -> System.out.println(p.getNome() + " Lv." + p.getLivello()));
```

Output:

```
Legolas Lv.3
Arthas Lv.5
Gandalf Lv.7
```

11.11 Comparable: definire l'ordine naturale

Nella sezione precedente hai visto che `Collections.sort()` ordina una lista. Ma come fa Java a sapere in che ordine mettere i tuoi oggetti personalizzati? Non lo sa — a meno che tu non

glielo dica.

Puoi codificare l'ordine "naturale" dei tuoi oggetti direttamente nella classe implementando l'interfaccia `Comparable`.

11.11.1 Implementare Comparable

`Comparable<T>` ha un solo metodo da implementare: `compareTo()`. Deve restituire:

- un numero **negativo** se `this` viene prima di `other`
- **zero** se sono equivalenti
- un numero **positivo** se `this` viene dopo di `other`

```
public class Personaggio implements Comparable<Personaggio> {

    private String nome;
    private int vita;
    private int vitaMax;
    private int attacco;
    private int livello;

    // ... costruttori, getter, setter, metodi ...

    // Ordine naturale: per livello crescente
    @Override
    public int compareTo(Personaggio altro) {
        return this.livello - altro.livello;
        // negativo → this viene prima (livello più basso)
        // zero     → stesso livello
        // positivo → this viene dopo (livello più alto)
    }
}
```

Ora `Collections.sort()` sa come ordinare i personaggi senza bisogno di istruzioni aggiuntive:

```
List<Personaggio> squadra = new ArrayList<>();
squadra.add(new Personaggio("Arthas", 150, 30, 5));
squadra.add(new Personaggio("Gandalf", 100, 20, 7));
squadra.add(new Personaggio("Legolas", 120, 25, 3));

// Funziona perché Personaggio implementa Comparable
```

```java
Collections.sort(squadra);

for (Personaggio p : squadra) {
    System.out.println(p.getNome() + " (Lv." + p.getLivello() + ")");
}
```

Output:

```
Legolas (Lv.3)
Arthas (Lv.5)
Gandalf (Lv.7)
```

`TreeSet` e `TreeMap` usano anch'essi `compareTo()` per mantenere gli elementi ordinati automaticamente:

```java
// Funziona perché Personaggio implementa Comparable
TreeSet<Personaggio> squadraOrdinata = new TreeSet<>();
squadraOrdinata.add(new Personaggio("Arthas", 150, 30, 5));
squadraOrdinata.add(new Personaggio("Gandalf", 100, 20, 7));
squadraOrdinata.add(new Personaggio("Legolas", 120, 25, 3));

// Già ordinati per livello, senza sort esplicito
squadraOrdinata.forEach(p ->
    System.out.println(p.getNome() + " Lv." + p.getLivello()));
```

La logica di compareTo()

Un trucco semplice per ricordare il segno:

```java
// Ordine crescente per un intero:
return this.livello - altro.livello;

// Ordine decrescente per un intero:
return altro.livello - this.livello;

// Per le stringhe, usa compareTo() già disponibile:
return this.nome.compareTo(altro.nome);  // ordine alfabetico
```

! Comparable e equals() devono essere coerenti

Se due oggetti sono uguali secondo `equals()`, `compareTo()` dovrebbe restituire 0. E viceversa — se `compareTo()` restituisce 0, in genere ci si aspetta che `equals()` sia `true`. Mantieni i due metodi allineati usando gli stessi campi.

11.12 Array vs ArrayList — quando usare cosa

	Array	ArrayList
Dimensione	Fissa	Dinamica
Tipi	Primitivi e oggetti	Solo oggetti (wrapper per primitivi)
Performance	Leggermente più veloce	Ottima per uso generale
Metodi utili	Solo `length`	`add`, `remove`, `contains`, `size`…
Quando usarlo	Dimensione nota e fissa	Dimensione variabile o sconosciuta

---

11.13 Esempio completo: gestione inventario

```
import java.util.*;

public class GestioneGioco {
    public static void main(String[] args) {

        // Lista giocatori
        List<Personaggio> giocatori = new ArrayList<>();
        giocatori.add(new Personaggio("Arthas", 150, 30, 1));
        giocatori.add(new Personaggio("Gandalf", 100, 20, 3));
        giocatori.add(new Personaggio("Legolas", 120, 25, 2));

        // Mappa punteggi
        Map<String, Integer> punteggi = new HashMap<>();
        for (Personaggio p : giocatori) {
```

```java
            punteggi.put(p.getNome(), 0);
        }

        // Simulazione: ogni personaggio guadagna punti per livello
        for (Personaggio p : giocatori) {
            int punti = p.getLivello() * 100;
            punteggi.put(p.getNome(), punti);
        }

        // Classifica ordinata per punteggio decrescente
        giocatori.sort((a, b) ->
            punteggi.get(b.getNome()) - punteggi.get(a.getNome()));

        System.out.println("=== CLASSIFICA ===");
        for (int i = 0; i < giocatori.size(); i++) {
            String nome = giocatori.get(i).getNome();
            System.out.println((i + 1) + ". " +
                nome + " → " + punteggi.get(nome) + " punti");
        }
    }
}
```

Output:

```
=== CLASSIFICA ===
1. Gandalf → 300 punti
2. Legolas → 200 punti
3. Arthas → 100 punti
```

11.14 Riepilogo

> **I concetti chiave di questa lezione**
>
> - Il **Collections Framework** offre strutture dati dinamiche e flessibili.
> - I **generics** (`<T>`) specificano il tipo degli elementi e rendono il codice type-safe.
> - **`ArrayList`** è una lista dinamica: elementi ordinati, duplicati ammessi, accesso per indice.
> - **`HashMap`** associa chiavi a valori: ricerca rapida per chiave, nessun ordine garantito.

- Programma verso l'**interfaccia** (`List`, `Map`) e istanzia l'**implementazione** (`ArrayList`, `HashMap`).
- La classe **`Collections`** offre metodi statici utili: `sort`, `reverse`, `shuffle`, `max`, `min`.
- Per ordinare oggetti personalizzati usa un **`Comparator`** con un lambda.

Nella prossima lezione approfondiamo le interfacce `List`, `Set` e `Queue` con le loro implementazioni e casi d'uso.

12 List, Set e Queue

12.1 Tre strutture, tre problemi diversi

Nella lezione precedente abbiamo visto `ArrayList` e `HashMap`. Il Collections Framework offre però molte altre strutture, ognuna progettata per un caso d'uso specifico.

La scelta della struttura giusta non è un dettaglio — influenza la leggibilità del codice, la correttezza logica e le performance. La domanda da farsi è sempre:

- Ho bisogno di **ordine** e **duplicati**? → **List**
- Ho bisogno di **unicità**? → **Set**
- Ho bisogno di processare elementi in **sequenza**? → **Queue**

12.2 List: sequenza ordinata

L'interfaccia `List` rappresenta una sequenza ordinata di elementi dove i **duplicati sono ammessi**. Ogni elemento ha un indice.

12.2.1 ArrayList vs LinkedList

Le due implementazioni principali hanno caratteristiche diverse:

	ArrayList	LinkedList
Accesso per indice	Veloce O(1)	Lento O(n)
Inserimento in mezzo	Lento O(n)	Veloce O(1)
Inserimento in fondo	Veloce	Veloce
Memoria	Meno overhead	Più overhead

	ArrayList	LinkedList
Quando usarla	Lettura frequente	Inserimenti/rimozioni frequenti

Per la maggior parte dei casi d'uso, `ArrayList` è la scelta migliore. `LinkedList` conviene solo quando fai molti inserimenti e rimozioni in posizioni intermedie.

```
import java.util.*;

// ArrayList - accesso rapido per indice
List<String> storico = new ArrayList<>();
storico.add("Arthas ha ucciso il Lich King");
storico.add("Gandalf ha trovato l'Anello");
storico.add("Legolas ha abbattuto l'elefante");

System.out.println(storico.get(1));  // accesso rapido per indice

// LinkedList - inserimenti rapidi
List<String> coda = new LinkedList<>();
coda.add("evento1");
coda.add(0, "evento0");  // inserimento in testa - efficiente con LinkedList
```

12.2.2 Metodi aggiuntivi di List

Oltre a quelli visti nella lezione precedente, `List` offre:

```
List<String> nomi = new ArrayList<>(Arrays.asList("Arthas", "Gandalf", "Legolas"));

// Sottolist (vista sulla lista originale)
List<String> primi = nomi.subList(0, 2);  // ["Arthas", "Gandalf"]

// Indice dell'ultima occorrenza
nomi.add("Arthas");
int ultimo = nomi.lastIndexOf("Arthas");  // 3

// Convertire in array
String[] arr = nomi.toArray(new String[0]);

// Sostituire tutti gli elementi
nomi.replaceAll(String::toUpperCase);  // ["ARTHAS", "GANDALF", "LEGOLAS", "ARTHAS"]
```

```
// Rimuovere con condizione
nomi.removeIf(n -> n.startsWith("A")); // rimuove tutti che iniziano con A
```

12.3 Set: nessun duplicato

L'interfaccia `Set` rappresenta una collezione **senza duplicati**. Perfetta quando vuoi garantire l'unicità degli elementi.

12.3.1 HashSet

`HashSet` è l'implementazione più comune — veloce ma senza ordine garantito:

```
import java.util.*;

Set<String> giocatoriConnessi = new HashSet<>();

giocatoriConnessi.add("Arthas");
giocatoriConnessi.add("Gandalf");
giocatoriConnessi.add("Arthas");  // ignorato - già presente!

System.out.println(giocatoriConnessi.size());           // 2, non 3
System.out.println(giocatoriConnessi.contains("Arthas")); // true

// Iterazione - l'ordine NON è garantito
for (String nome : giocatoriConnessi) {
    System.out.println(nome);
}
```

12.3.2 TreeSet

`TreeSet` mantiene gli elementi **ordinati** (ordine naturale o con Comparator):

```
Set<String> nomiOrdinati = new TreeSet<>();
nomiOrdinati.add("Legolas");
nomiOrdinati.add("Arthas");
nomiOrdinati.add("Gandalf");
```

```java
nomiOrdinati.add("Arthas");  // ignorato

// Stampa in ordine alfabetico: Arthas, Gandalf, Legolas
for (String nome : nomiOrdinati) {
    System.out.println(nome);
}

// Metodi specifici di TreeSet
TreeSet<Integer> punteggi = new TreeSet<>();
punteggi.add(150);
punteggi.add(320);
punteggi.add(80);
punteggi.add(210);

System.out.println(punteggi.first());  // 80 - minimo
System.out.println(punteggi.last());   // 320 - massimo
System.out.println(punteggi.headSet(200)); // elementi < 200: [80, 150]
System.out.println(punteggi.tailSet(200)); // elementi >= 200: [210, 320]
```

12.3.3 Operazioni tra insiemi

I `Set` supportano naturalmente le operazioni insiemistiche:

```java
Set<String> squadraA = new HashSet<>(Arrays.asList("Arthas", "Gandalf", "Legolas"));
Set<String> squadraB = new HashSet<>(Arrays.asList("Gandalf", "Aragorn", "Frodo"));

// Unione
Set<String> unione = new HashSet<>(squadraA);
unione.addAll(squadraB);  // tutti i giocatori unici

// Intersezione
Set<String> intersezione = new HashSet<>(squadraA);
intersezione.retainAll(squadraB);  // solo i giocatori in entrambe le squadre

// Differenza
Set<String> differenza = new HashSet<>(squadraA);
differenza.removeAll(squadraB);  // giocatori in A ma non in B

System.out.println("Unione: " + unione);             // [Arthas, Gandalf, Legolas, Aragorn, F
System.out.println("Intersezione: " + intersezione); // [Gandalf]
System.out.println("Differenza: " + differenza);     // [Arthas, Legolas]
```

HashSet vs TreeSet: quando usare quale

Usa **`HashSet`** quando: - non ti interessa l'ordine - vuoi la massima velocità (ricerca, inserimento, rimozione in O(1))
Usa **`TreeSet`** quando: - vuoi gli elementi sempre ordinati - hai bisogno di operazioni come "dammi tutti gli elementi tra X e Y"

12.4 Queue: la coda

L'interfaccia `Queue` rappresenta una struttura **FIFO** (First In, First Out) — il primo elemento inserito è il primo ad essere estratto. Come una fila al supermercato.

```
Inserimento → [C] [B] [A] → Estrazione
               ^             ^
             coda          testa
```

12.4.1 Metodi principali di Queue

Metodo	Comportamento se vuota
`add(e)` / `offer(e)`	Inserisce in coda (`add` lancia eccezione, `offer` no)
`remove()` / `poll()`	Estrae dalla testa (`remove` lancia eccezione, `poll` no)
`element()` / `peek()`	Legge la testa senza estrarre (`element` lancia eccezione, `peek` no)

La convenzione è usare `offer`, `poll` e `peek` — non lanciano eccezioni ma restituiscono `null`.

```java
import java.util.*;

Queue<String> codaEventi = new LinkedList<>();

// Inserimento in coda
codaEventi.offer("Arthas attacca");
codaEventi.offer("Gandalf lancia incantesimo");
```

```java
codaEventi.offer("Legolas scocca freccia");

System.out.println("Prossimo evento: " + codaEventi.peek()); // legge senza rimuovere

// Elaborazione degli eventi in ordine FIFO
while (!codaEventi.isEmpty()) {
    String evento = codaEventi.poll();  // estrae dalla testa
    System.out.println("Elaboro: " + evento);
}
```

12.4.2 PriorityQueue: coda con priorità

`PriorityQueue` elabora gli elementi non in ordine di inserimento, ma in base a una **priorità**:

```java
// PriorityQueue di default: ordine naturale (minimo prima)
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(50);
pq.offer(10);
pq.offer(30);

while (!pq.isEmpty()) {
    System.out.println(pq.poll());  // 10, 30, 50 - ordine crescente
}
```

Con oggetti personalizzati puoi definire la priorità con un `Comparator`:

```java
// Coda di personaggi ordinata per vita (prima i più feriti)
PriorityQueue<Personaggio> codaCura = new PriorityQueue<>(
    (a, b) -> a.getVita() - b.getVita()  // vita minore = priorità più alta
);

codaCura.offer(new Personaggio("Arthas", 150, 30, 1));
codaCura.offer(new Personaggio("Gandalf", 100, 20, 3));
codaCura.offer(new Personaggio("Legolas", 120, 25, 2));

// Il mago con 20 di vita viene curato per primo
while (!codaCura.isEmpty()) {
    Personaggio p = codaCura.poll();
    System.out.println("Curo " + p.getNome() + " (vita: " + p.getVita() + ")");
}
```

Output:

```
Curo Gandalf (vita: 20)
Curo Legolas (vita: 25)
Curo Arthas (vita: 30)
```

Queue come Deque

`LinkedList` implementa anche l'interfaccia `Deque` (Double Ended Queue) — una coda a doppia estremità da cui si può inserire ed estrarre sia dalla testa che dalla coda. Questo la rende utile anche come **stack** (LIFO):

```
Deque<String> stack = new LinkedList<>();
stack.push("primo");   // inserisce in testa
stack.push("secondo");
stack.push("terzo");

System.out.println(stack.pop());  // "terzo" - LIFO
System.out.println(stack.pop());  // "secondo"
```

12.5 Confronto finale: quale struttura scegliere?

Struttura	Ordine	Duplicati	Accesso	Caso d'uso tipico
`ArrayList`	Inserimento	Sì	Per indice	Lista generica, accesso frequente
`LinkedList`	Inserimento	Sì	Sequenziale	Molti inserimenti/rimozioni
`HashSet`	Nessuno	No	Per valore	Unicità, ricerca rapida
`TreeSet`	Ordinato	No	Per valore	Unicità + ordine
`HashMap`	Nessuno	Chiavi uniche	Per chiave	Dizionario, lookup rapido
`TreeMap`	Chiavi ordinate	Chiavi uniche	Per chiave	Dizionario + ordine

Struttura	Ordine	Duplicati	Accesso	Caso d'uso tipico
`LinkedList` (Queue)	FIFO	Sì	Testa/coda	Coda di eventi
`PriorityQueue`	Priorità	Sì	Per priorità	Scheduling, cura prioritaria

12.6 Esempio completo: sistema di turni

Un sistema di turni per un gioco — i personaggi agiscono in ordine di iniziativa (più alta prima):

```
import java.util.*;

public class SistemaTurni {
    public static void main(String[] args) {

        // Coda con priorità: più alta l'iniziativa, prima si agisce
        PriorityQueue<Personaggio> turni = new PriorityQueue<>(
            (a, b) -> b.getLivello() - a.getLivello()  // livello più alto = prima
        );

        turni.offer(new Personaggio("Arthas", 150, 30, 3));
        turni.offer(new Personaggio("Gandalf", 100, 20, 7));
        turni.offer(new Personaggio("Legolas", 120, 25, 5));

        Personaggio boss = new Personaggio("Lich King", 500, 40, 10);

        System.out.println("=== TURNO DI COMBATTIMENTO ===\n");

        // Set per tenere traccia dei nomi già estratti (solo per demo)
        Set<String> haAgito = new HashSet<>();

        while (!turni.isEmpty() && !boss.isMorto()) {
            Personaggio attaccante = turni.poll();
            System.out.print("[Lv." + attaccante.getLivello() + "] ");
            attaccante.attacca(boss);
            haAgito.add(attaccante.getNome());
```

```
        }

        System.out.println("\nHanno agito: " + haAgito);
        System.out.println("Vita del boss: " + boss.getVita() + "/" + boss.getVitaMax());
    }
}
```

12.7 Riepilogo

> I concetti chiave di questa lezione

- **`List`** — sequenza ordinata con duplicati. `ArrayList` per accesso rapido, `LinkedList` per inserimenti frequenti.
- **`Set`** — nessun duplicato. `HashSet` per velocità, `TreeSet` per ordine.
- **`Queue`** — elaborazione FIFO. `LinkedList` come coda semplice, `PriorityQueue` per priorità.
- Le operazioni insiemistiche (`addAll`, `retainAll`, `removeAll`) funzionano su tutti i `Set`.
- Preferisci i metodi "safe" (`offer`, `poll`, `peek`) a quelli che lanciano eccezioni (`add`, `remove`, `element`).
- La scelta della struttura giusta dipende dal **caso d'uso**: ordine, unicità, tipo di accesso.

13 Stream e pipeline

13.1 Un nuovo modo di pensare alle collections

Nelle lezioni precedenti abbiamo visto come gestire le collections con cicli `for`, `while` e metodi espliciti. Funziona, ma il codice può diventare verboso. Considera questo esempio — trovare i nomi dei personaggi di livello superiore a 3, in ordine alfabetico:

```
// Approccio tradizionale
// Approccio tradizionale
List<Personaggio> risultato = new ArrayList<>();
for (Personaggio p : squadra) {
    if (p.getLivello() > 3) {
        risultato.add(p);
    }
}
Collections.sort(risultato);  // usa l'ordine naturale definito in compareTo()

List<String> nomi = new ArrayList<>();
for (Personaggio p : risultato) {
    nomi.add(p.getNome());
}
```

Con gli **Stream** lo stesso risultato si scrive così:

```
// Approccio con Stream
List<String> nomi = squadra.stream()
    .filter(p -> p.getLivello() > 3)
    .sorted()
    .map(Personaggio::getNome)
    .collect(Collectors.toList());
```

Stesso risultato, meno codice, più leggibile. Vediamo come funziona.

13.2 Cos'è uno Stream

Uno **Stream** è una sequenza di elementi su cui si applicano operazioni in **pipeline**, una catena di trasformazioni che si eseguono una dopo l'altra.

Uno Stream non è una struttura dati, non memorizza elementi ma è un flusso di dati che viene elaborato e trasformato.

> **Le tre fasi di uno Stream**
>
> 1. **Sorgente** — da dove arrivano i dati (`List`, `Set`, array, file…)
> 2. **Operazioni intermedie** — trasformazioni che producono un nuovo Stream (`filter`, `map`, `sorted`…)
> 3. **Operazione terminale** — produce il risultato finale (`collect`, `count`, `forEach`, `reduce`…)

```
sorgente → [op. intermedia] → [op. intermedia] → [op. terminale] → risultato
squadra  →     filter()     →      map()        →    collect()    → List<String>
```

13.3 Creare uno Stream

```java
import java.util.stream.*;

List<Personaggio> squadra = new ArrayList<>();
// ...

// Da una List o Set
Stream<Personaggio> s1 = squadra.stream();

// Da un array
String[] nomi = {"Arthas", "Gandalf", "Legolas"};
Stream<String> s2 = Arrays.stream(nomi);

// Stream di valori diretti
```

```
Stream<String> s3 = Stream.of("uno", "due", "tre");

// Stream di interi
IntStream numeri = IntStream.range(1, 10);  // 1, 2, 3, ..., 9
IntStream numeri2 = IntStream.rangeClosed(1, 10); // 1, 2, 3, ..., 10
```

13.4 Lambda e method reference

Prima di vedere le operazioni, capiamo la sintassi che si usa con gli Stream.

Lambda — una funzione anonima compatta:

```
// Sintassi: (parametri) -> espressione
p -> p.getLivello() > 3           // predicate: prende un Personaggio, restituisce boolean
p -> p.getNome()                   // function: prende un Personaggio, restituisce String
```

Method reference — abbreviazione quando il lambda chiama solo un metodo:

```
// Invece di: p -> p.getNome()
Personaggio::getNome   // method reference su istanza

// Invece di: s -> System.out.println(s)
System.out::println    // method reference statico

// Invece di: s -> s.toUpperCase()
String::toUpperCase    // method reference su istanza
```

13.5 Operazioni intermedie

Le operazioni intermedie trasformano lo Stream in un altro Stream. Sono **lazy** cioè non vengono eseguite finché non arriva un'operazione terminale.

13.5.1 filter()

Mantiene solo gli elementi che soddisfano una condizione:

```
// Solo personaggi di livello superiore a 3
squadra.stream()
    .filter(p -> p.getLivello() > 3)
    ...

// Solo personaggi in vita
squadra.stream()
    .filter(p -> !p.isMorto())
    ...

// Solo personaggi con nome che inizia per A
squadra.stream()
    .filter(p -> p.getNome().startsWith("A"))
    ...
```

13.5.2 map()

Trasforma ogni elemento in qualcos'altro:

```
// Da Stream<Personaggio> a Stream<String> (solo i nomi)
squadra.stream()
    .map(p -> p.getNome())
    ...

// Equivalente con method reference
squadra.stream()
    .map(Personaggio::getNome)
    ...

// Da Stream<Personaggio> a Stream<Integer> (solo i livelli)
squadra.stream()
    .map(Personaggio::getLivello)
    ...

// Per ottenere IntStream (più efficiente con interi)
squadra.stream()
    .mapToInt(Personaggio::getLivello)
    ...
```

13.5.3 sorted()

Ordina gli elementi:

```
// Ordine naturale (richiede Comparable)
numeri.stream()
    .sorted()
    ...

// per oggetti che non implementano Comparable possiamo usare Comparator
squadra.stream()
    .sorted((a, b) -> a.getNome().compareTo(b.getNome()))
    ...

// Ordine inverso
squadra.stream()
    .sorted(Comparator.comparing(Personaggio::getLivello).reversed())
    ...
```

13.5.4 limit() e skip()

```
// Prendi solo i primi 3 elementi
squadra.stream()
    .limit(3)
    ...

// Salta i primi 2 elementi
squadra.stream()
    .skip(2)
    ...

// Paginazione: seconda pagina di 3 elementi
squadra.stream()
    .skip(3)
    .limit(3)
    ...
```

13.5.5 distinct() e peek()

```
// Rimuove i duplicati
List<Integer> livelli = squadra.stream()
    .map(Personaggio::getLivello)
    .distinct()
    .collect(Collectors.toList());

// peek() per debug - esegue un'azione senza modificare lo Stream
squadra.stream()
    .filter(p -> p.getLivello() > 2)
    .peek(p -> System.out.println("Dopo filter: " + p.getNome()))
    .map(Personaggio::getNome)
    .peek(n -> System.out.println("Dopo map: " + n))
    .collect(Collectors.toList());
```

13.6 Operazioni terminali

Le operazioni terminali consumano lo Stream e producono un risultato. Dopo un'operazione terminale, lo Stream non può essere riusato.

13.6.1 collect()

Raccoglie gli elementi in una struttura dati:

```
import java.util.stream.Collectors;

// In una List
List<String> nomi = squadra.stream()
    .map(Personaggio::getNome)
    .collect(Collectors.toList());

// In un Set (rimuove duplicati)
Set<Integer> livelli = squadra.stream()
    .map(Personaggio::getLivello)
    .collect(Collectors.toSet());
```

```java
// In una stringa con separatore
String nomiuniti = squadra.stream()
    .map(Personaggio::getNome)
    .collect(Collectors.joining(", "));  // "Arthas, Gandalf, Legolas"

// In una mappa nome → livello
Map<String, Integer> mappa = squadra.stream()
    .collect(Collectors.toMap(
        Personaggio::getNome,
        Personaggio::getLivello
    ));

// Raggruppati per livello
Map<Integer, List<Personaggio>> perLivello = squadra.stream()
    .collect(Collectors.groupingBy(Personaggio::getLivello));
```

13.6.2 forEach()

Esegue un'azione su ogni elemento — non restituisce niente:

```java
squadra.stream()
    .filter(p -> !p.isMorto())
    .forEach(p -> System.out.println(p.getNome() + " è in vita"));

// Equivalente con method reference
squadra.stream()
    .forEach(Personaggio::stampaScheda);
```

13.6.3 count(), sum(), average(), min(), max()

```java
// Conteggio
long numVivi = squadra.stream()
    .filter(p -> !p.isMorto())
    .count();

// Somma, media su interi
int vitaTotale = squadra.stream()
    .mapToInt(Personaggio::getVita)
    .sum();
```

```java
double mediaLivelli = squadra.stream()
    .mapToInt(Personaggio::getLivello)
    .average()
    .orElse(0.0);  // orElse perché average restituisce Optional

// Minimo e massimo
Optional<Personaggio> piuForte = squadra.stream()
    .max(Comparator.comparing(Personaggio::getAttacco));

piuForte.ifPresent(p ->
    System.out.println("Più forte: " + p.getNome()));
```

13.6.4 anyMatch(), allMatch(), noneMatch()

```java
// Almeno uno soddisfa la condizione?
boolean ceMagoVivo = squadra.stream()
    .anyMatch(p -> !p.isMorto());

// Tutti soddisfano la condizione?
boolean tuttiAlti = squadra.stream()
    .allMatch(p -> p.getLivello() >= 3);

// Nessuno soddisfa la condizione?
boolean nessunoMorto = squadra.stream()
    .noneMatch(Personaggio::isMorto);
```

13.6.5 findFirst() e findAny()

```java
// Primo personaggio di livello > 3
Optional<Personaggio> primo = squadra.stream()
    .filter(p -> p.getLivello() > 3)
    .findFirst();

primo.ifPresent(p -> System.out.println("Trovato: " + p.getNome()));

// Gestione esplicita dell'Optional
String nome = primo
```

```
    .map(Personaggio::getNome)
    .orElse("Nessuno trovato");
```

13.7 Optional: gestire l'assenza di valore

Molte operazioni terminali restituiscono `Optional<T>` — un contenitore che può contenere un valore oppure essere vuoto. Evita i `NullPointerException`.

```
Optional<Personaggio> forse = squadra.stream()
    .filter(p -> p.getNome().equals("Sauron"))
    .findFirst();

// Verifica se presente
if (forse.isPresent()) {
    System.out.println(forse.get().getNome());
}

// Versione compatta
forse.ifPresent(p -> System.out.println(p.getNome()));

// Valore di default se assente
Personaggio p = forse.orElse(new Personaggio("Default", 1, 1, 1));

// Eccezione se assente
Personaggio p2 = forse.orElseThrow(() ->
    new IllegalStateException("Personaggio non trovato!"));
```

13.8 Pipeline complete: esempi reali

13.8.1 Classifica dei sopravvissuti

```
List<String> classifica = squadra.stream()
    .filter(p -> !p.isMorto())
    .sorted(Comparator.comparing(Personaggio::getLivello).reversed())
    .map(p -> p.getNome() + " (Lv." + p.getLivello() + ")")
    .collect(Collectors.toList());

System.out.println("Sopravvissuti in ordine di livello:");
classifica.forEach(System.out::println);
```

13.8.2 Statistiche della squadra

```
IntSummaryStatistics stats = squadra.stream()
    .mapToInt(Personaggio::getVita)
    .summaryStatistics();

System.out.println("Vita minima:  " + stats.getMin());
System.out.println("Vita massima: " + stats.getMax());
System.out.println("Vita media:   " + stats.getAverage());
System.out.println("Vita totale:  " + stats.getSum());
System.out.println("Personaggi:   " + stats.getCount());
```

13.8.3 Dal file al report

Uno stream può elaborare dati da qualsiasi sorgente, anche righe di testo che rappresentano dati:

```
import java.util.stream.*;

// Simula righe lette da un file CSV: "nome,livello,vita"
List<String> righe = List.of(
    "Arthas,3,150",
    "Gandalf,7,100",
    "Legolas,5,120",
    "Sauron,10,500"
);

// Parse e filtra solo i personaggi di livello >= 5
Map<String, Integer> report = righe.stream()
    .map(riga -> riga.split(","))          // split ogni riga
```

```
    .filter(parti -> Integer.parseInt(parti[1]) >= 5) // livello >= 5
    .collect(Collectors.toMap(
        parti -> parti[0],                // chiave: nome
        parti -> Integer.parseInt(parti[2]) // valore: vita
    ));

System.out.println("Personaggi di alto livello:");
report.forEach((nome, vita) ->
    System.out.println("  " + nome + " → vita: " + vita));
```

Output:

```
Personaggi di alto livello:
  Gandalf → vita: 100
  Legolas → vita: 120
  Sauron → vita: 500
```

13.9 Riepilogo

> **I concetti chiave**
>
> - Uno **Stream** è un flusso di dati che si elabora in pipeline — non memorizza dati.
> - Le tre fasi: **sorgente** → **operazioni intermedie** → **operazione terminale**.
> - Le operazioni intermedie sono **lazy** — eseguite solo quando arriva la terminale.
> - `filter()` seleziona, `map()` trasforma, `sorted()` ordina, `limit()` tronca.
> - `collect()` raccoglie in una struttura, `forEach()` itera, `count()` conta.
> - `Optional` gestisce l'assenza di valore in modo sicuro, senza `null`.
> - I **lambda** e i **method reference** rendono il codice compatto e leggibile.

Part V

Stringhe avanzate

14 Stringhe avanzate

14.1 String è immutabile

Nella lezione sui tipi abbiamo visto che `String` è una classe. Ma ha una caratteristica fondamentale che spesso sorprende: le stringhe in Java sono **immutabili** — una volta create, non possono essere modificate.

Quando sembra che stai modificando una stringa, in realtà stai creando un nuovo oggetto:

```
String s = "ciao";
s = s + " mondo";
```

Cosa succede realmente: 1. viene creata una **nuova stringa** `"ciao mondo"` 2. i caratteri della vecchia stringa vengono copiati nel nuovo oggetto 3. la vecchia stringa `"ciao"` resta in memoria finché il garbage collector non la elimina

Ogni concatenazione crea quindi una nuova allocazione, una copia dei dati e uno spreco di tempo e memoria.

14.2 Il problema delle concatenazioni in ciclo

L'immutabilità diventa un problema serio quando concateni in un ciclo:

```
String testo = "";
for (int i = 0; i < 10000; i++) {
    testo = testo + i;  // crea un nuovo oggetto ad ogni iterazione!
}
```

Ad ogni iterazione viene creata una nuova stringa e vengono ricopiati tutti i caratteri precedenti. La complessità cresce rapidamente: **effetto valanga**.

Con 10.000 iterazioni questo codice crea 10.000 oggetti `String` intermedi, quasi tutti subito scartati. È lento e spreca memoria.

14.3 StringBuilder: il buffer delle stringhe

`StringBuilder` risolve questo problema. Funziona come un **buffer in memoria**:

- mantiene un array interno modificabile
- aggiunge caratteri senza ricreare tutto
- rialloca solo quando necessario

```
// Approccio SBAGLIATO
String testo = "";
for (int i = 0; i < 10000; i++) {
    testo = testo + i;
}

// Approccio CORRETTO
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);
}
String risultato = sb.toString();  // converte in String solo alla fine
```

Un solo oggetto, nessuna copia continua, prestazioni molto migliori.

14.4 Metodi principali di StringBuilder

```
StringBuilder sb = new StringBuilder("Arthas");

// append() - aggiunge in fondo
sb.append(" il Re dei Lich");     // "Arthas il Re dei Lich"
sb.append(42);                     // funziona con qualsiasi tipo
```

```java
// insert() - inserisce in posizione specifica
sb.insert(0, "Lord ");            // "Lord Arthas il Re dei Lich"

// delete() - rimuove una porzione
sb.delete(0, 5);                  // rimuove "Lord "

// replace() - sostituisce una porzione
sb.replace(0, 6, "Cavaliere");

// reverse() - inverte tutta la stringa
sb.reverse();

// length() e charAt()
int len = sb.length();
char c = sb.charAt(0);

// setCharAt() - modifica un carattere
sb.setCharAt(0, 'A');

// toString() - converte in String
String finale = sb.toString();
```

> **Quando usare StringBuilder, quando String**
>
> Usa **`StringBuilder`** quando concateni dentro cicli, crei testo lungo o fai parsing riga per riga.
> Usa **`String`** quando hai poche concatenazioni, lavori con stringhe costanti, o hai bisogno di codice semplice e leggibile.

14.5 Lo String Pool

Dopo aver capito `StringBuilder`, è utile capire un altro meccanismo interno di Java: lo **String Pool**.

Lo String Pool è una zona speciale della memoria in cui Java conserva le stringhe letterali per evitare duplicazioni inutili.

```
String a = "ciao";
String b = "ciao";
```

In questo caso Java **non** crea due oggetti distinti. Esiste un solo oggetto nello String Pool, e sia `a` che `b` puntano allo stesso riferimento:

```
System.out.println(a == b);      // true: stesso riferimento!
System.out.println(a.equals(b)); // true: stesso contenuto
```

Attenzione però a `new String()`:

```
String a = "ciao";
String b = new String("ciao");

System.out.println(a == b);      // false: oggetti diversi!
System.out.println(a.equals(b)); // true: contenuto uguale
```

`new String()` crea sempre un nuovo oggetto nell'heap, anche se il contenuto esiste già nel pool.

> **Perché esiste lo String Pool?**
>
> Per efficienza: riduce l'uso della memoria ed evita la creazione continua di oggetti identici. Immagina migliaia di stringhe `"OK"` o `"ERROR"` in un programma — senza pool avremmo migliaia di copie uguali in memoria.
> Lo String Pool funziona **solo perché le stringhe sono immutabili**. Se una stringa potesse cambiare, modificare `a` cambierebbe anche `b` dato che condividono lo stesso oggetto.

14.6 Stack, Heap e String Pool: dove vivono gli oggetti

Per capire davvero cosa succede con le stringhe, è utile vedere come Java organizza la memoria durante l'esecuzione.

Java utilizza tre zone principali:

```
                HEAP

            STRING POOL
      [ "java" ] [ "ciao" ] ...

  [ oggetti ]  [ array ]  [ StringBuilder]



                STACK
   a → riferimento
   b → riferimento
   sb → riferimento
```

Stack — contiene variabili locali dei metodi, riferimenti agli oggetti e parametri. È molto veloce, gestito automaticamente, segue la logica "entra/esci" dei metodi. Nota: nello stack non vive la stringa, ma solo il riferimento ad essa.

Heap — contiene oggetti, array, `StringBuilder` e tutto ciò che si crea con `new`. È più grande ma più lento dello stack.

String Pool — zona speciale dell'heap dedicata alle stringhe letterali.

14.7 Simulazione della memoria

```java
String a = "java";
String b = "java";
String c = new String("java");
```

Passo 1 — `String a = "java"`: Java controlla lo String Pool. Non c'è ancora `"java"`, quindi lo crea.

```
STRING POOL: [ "java" ]
STACK: a → pool["java"]
```

Passo 2 — `String b = "java"`: Java controlla lo String Pool. `"java"` esiste già — riusa lo stesso oggetto.

```
STRING POOL: [ "java" ]
STACK: a → pool["java"]
       b → pool["java"]   ← stesso riferimento!
```

Passo 3 — `String c = new String("java")`: `new` crea sempre un nuovo oggetto nell'heap, fuori dal pool.

```
STRING POOL: [ "java" ]
HEAP:        [ "java" ]   ← oggetto nuovo e separato
STACK: a → pool["java"]
       b → pool["java"]
       c → heap["java"]
```

Ecco perché `a == b` è `true` (stesso riferimento nel pool) ma `a == c` è `false` (oggetti diversi).

14.8 Esperimento guidato

Prevedi l'output prima di eseguire:

```java
// Caso 1: letterali
String a = "java";
String b = "ja" + "va";  // concatenazione di letterali → avviene a compile time!
String c = new String("java");

System.out.println(a == b);      // ?
System.out.println(a == c);      // ?
System.out.println(a.equals(c)); // ?
```

```java
// Caso 2: variabile + letterale
String a = "java";
String x = "ja";
String b = x + "va";  // concatenazione a runtime → crea un nuovo oggetto nell'heap!

System.out.println(a == b);      // ?
System.out.println(a.equals(b)); // ?
```

La chiave per rispondere

`"ja" + "va"` è una concatenazione di due **letterali** — il compilatore la risolve a compile time e produce un solo oggetto `"java"` nel pool.
`x + "va"` invece coinvolge una **variabile** — il compilatore non può prevederla, quindi la concatenazione avviene a runtime e crea un nuovo oggetto nell'heap.

14.9 Il metodo `intern()`

`intern()` forza una stringa a entrare nello String Pool (o restituisce il riferimento se esiste già):

```
String s = new String("ciao");  // nell'heap
String p = s.intern();          // restituisce il riferimento nel pool

String a = "ciao";
System.out.println(a == p);     // true - ora puntano allo stesso oggetto nel pool
```

Utile in rari casi in cui vuoi garantire il riuso della memoria per stringhe create dinamicamente.

14.10 Metodi avanzati di String

Alcuni metodi utili per l'elaborazione di testo:

```
String s = "  Arthas il Re dei Lich  ";

// Pulizia
s.trim();              // rimuove spazi iniziali e finali
s.strip();             // come trim ma gestisce anche spazi Unicode
s.stripLeading();      // solo spazi iniziali
s.stripTrailing();     // solo spazi finali

// Controllo
s.isBlank();           // true se vuota o solo spazi
```

```
s.isEmpty();            // true solo se lunghezza zero
"abc".repeat(3);        // "abcabcabc"

// Split
String csv = "Arthas,150,5";
String[] parti = csv.split(",");     // ["Arthas", "150", "5"]
String[] prime = csv.split(",", 2);  // ["Arthas", "150,5"] - max 2 parti

// Confronto
"abc".compareTo("abd");              // negativo - "abc" viene prima
"abc".compareToIgnoreCase("ABC");    // 0 - uguali ignorando maiuscole
```

14.10.1 Concatenare con join e formatted

```
// String.join - unisce elementi con un separatore
String nomi = String.join(", ", "Arthas", "Gandalf", "Legolas");
// "Arthas, Gandalf, Legolas"

// Con una lista
List<String> lista = List.of("Arthas", "Gandalf", "Legolas");
String uniti = String.join(" | ", lista);
```

14.11 Espressioni regolari

Le **espressioni regolari** (regex) sono un linguaggio per descrivere pattern di testo. Permettono di cercare, validare e trasformare stringhe in modo potente.

14.11.1 I caratteri speciali principali

Pattern	Significato	Esempio
`.`	Qualsiasi carattere	`a.c` → "abc", "a1c"
`*`	Zero o più del precedente	`ab*c` → "ac", "abc", "abbc"
`+`	Uno o più del precedente	`ab+c` → "abc", "abbc" (non "ac")
`?`	Zero o uno del precedente	`ab?c` → "ac", "abc"

Pattern	Significato	Esempio
`^`	Inizio della stringa	`^Arthas` → inizia con "Arthas"
`$`	Fine della stringa	`Lich$` → finisce con "Lich"
`\d`	Cifra (0-9)	`\d+` → uno o più cifre
`\w`	Carattere word (lettere, cifre, _)	`\w+` → una parola
`\s`	Spazio bianco	`\s+` → uno o più spazi
`[abc]`	Uno tra a, b, c	`[aeiou]` → una vocale
`[^abc]`	Nessuno tra a, b, c	`[^0-9]` → non una cifra
`[a-z]`	Range	`[a-zA-Z]` → qualsiasi lettera
`{n}`	Esattamente n volte	`\d{3}` → esattamente 3 cifre
`{n,m}`	Da n a m volte	`\d{2,4}` → da 2 a 4 cifre
`(abc)`	Gruppo	`(ab)+` → "ab", "abab"
`a|b`	a oppure b	`gatto|cane`

Escape in Java

In Java i caratteri speciali delle regex vanno scritti con doppio backslash: - `\d` in regex → `"\\d"` in Java - `\w+` in regex → `"\\w+"` in Java

14.11.2 matches(), replaceAll(), split()

```
// Validazione
"12345".matches("\\d+");                // true - solo cifre
"333-1234567".matches("\\d{3}-\\d{7}"); // true - formato telefono

// Sostituzione
String s = "Arthas ha 100 punti e Gandalf ha 250 punti";
s.replaceAll("\\d+", "###");            // "Arthas ha ### punti e Gandalf ha ### punti"
s.replaceFirst("\\d+", "###");          // solo la prima occorrenza
"Arthas   il    Re".replaceAll("\\s+", " "); // "Arthas il Re"

// Split
"Arthas   il Re".split("\\s+");         // ["Arthas", "il", "Re"]
"Arthas, Gandalf ,Legolas".split("\\s*,\\s*"); // ["Arthas", "Gandalf", "Legolas"]
```

14.11.3 Pattern e Matcher: ricerca avanzata

```
import java.util.regex.*;

String testo = "Arthas ha 150 vita e 30 di attacco";
Pattern pattern = Pattern.compile("\\d+");
Matcher matcher = pattern.matcher(testo);

while (matcher.find()) {
    System.out.println("Trovato: " + matcher.group() +
        " alla posizione " + matcher.start());
}
// Trovato: 150 alla posizione 10
// Trovato: 30 alla posizione 22
```

Con i **gruppi** puoi estrarre parti specifiche:

```
String dati = "Arthas:livello=5,vita=150";
Pattern p = Pattern.compile("(\\w+):livello=(\\d+),vita=(\\d+)");
Matcher m = p.matcher(dati);

if (m.matches()) {
    System.out.println("Nome:    " + m.group(1));  // Arthas
    System.out.println("Livello: " + m.group(2));  // 5
    System.out.println("Vita:    " + m.group(3));  // 150
}
```

14.12 Esempio completo: parser di un log di gioco

```
import java.util.*;
import java.util.regex.*;
import java.util.stream.*;

public class AnalisiLog {
    public static void main(String[] args) {
```

```java
        List<String> log = List.of(
            "[10:05] Arthas ha sconfitto Zombie (+50 punti)",
            "[10:07] Gandalf ha lanciato Fireball (-25 mana)",
            "[10:09] Arthas ha sconfitto Scheletro (+30 punti)",
            "[10:12] Legolas ha mancato il bersaglio",
            "[10:15] Gandalf ha sconfitto Drago (+200 punti)"
        );

        Pattern punteggio = Pattern.compile(
            "\\[(\\d+:\\d+)\\] (\\w+) ha sconfitto .+ \\(\\+(\\d+) punti\\)"
        );

        Map<String, Integer> totali = new HashMap<>();

        for (String riga : log) {
            Matcher m = punteggio.matcher(riga);
            if (m.matches()) {
                String nome = m.group(2);
                int punti = Integer.parseInt(m.group(3));
                totali.merge(nome, punti, Integer::sum);
            }
        }

        // Report finale con StringBuilder
        StringBuilder report = new StringBuilder();
        report.append("=== REPORT FINALE ===\n");

        totali.entrySet().stream()
            .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
            .forEach(e -> report
                .append(e.getKey())
                .append(": ")
                .append(e.getValue())
                .append(" punti\n"));

        System.out.println(report.toString());
    }
}
```

14.13 Riepilogo

I concetti chiave

- `String` è **immutabile** — ogni operazione crea un nuovo oggetto in memoria.
- La concatenazione in ciclo è inefficiente — ogni iterazione crea e scarta un oggetto.
- `StringBuilder` è **mutabile** — mantiene un array interno modificabile, molto più efficiente per costruzione incrementale.
- Lo **String Pool** è una zona speciale dell'heap dove Java conserva le stringhe letterali per evitare duplicazioni — funziona grazie all'immutabilità.
- Nello **Stack** vivono i riferimenti; nell'**Heap** vivono gli oggetti; nel **String Pool** vivono le stringhe letterali.
- `new String("...")` bypassa il pool e crea sempre un nuovo oggetto nell'heap — usa `==` restituisce `false`.
- Le **espressioni regolari** descrivono pattern di testo: `matches()` valida, `replaceAll()` sostituisce, `split()` divide.
- `Pattern` e `Matcher` permettono ricerche avanzate con estrazione di gruppi.

Part VI

Gestione file

15 Path, Files e Log

15.1 Lavorare con i file: una premessa

Finora i nostri programmi hanno lavorato con dati scritti direttamente nel codice. Quei dati sono sempre disponibili e sotto il nostro controllo.

Nel mondo reale le applicazioni lavorano quasi sempre con dati esterni: file di testo, log, configurazioni, CSV, dati prodotti da altri programmi. Un file non è mai garantito: può non esistere, può essere vuoto, può essere stato spostato, può non essere leggibile.

Per questo lavorare con i file significa accettare l'idea che qualcosa possa andare storto anche se il codice è corretto. Ed è qui che le **eccezioni checked** smettono di essere teoria e diventano necessarie.

15.2 Path: non è una stringa

In Java un file non si rappresenta con una semplice stringa, ma con un oggetto di tipo `Path`.

`Path` non è il file in sé — è il percorso che indica **dove si trova**. Possiamo pensarlo come un indirizzo o un puntatore a una posizione sul disco.

```
Path p = Path.of("input.txt");  // descrive una posizione, non legge nulla
```

Creare un `Path` non genera errori — anche se il file non esiste. L'errore arriva solo quando proviamo davvero ad accedere al file tramite `Files`.

> **!** Path NON è una stringa
>
> `Path` è un oggetto che rappresenta la posizione di una risorsa in modo **astratto e portabile**. A differenza di una stringa:

- conosce i separatori corretti per ogni sistema operativo
- permette di costruire percorsi in modo sicuro
- evita errori di concatenazione manuale
- non dipende da Windows, Linux o macOS

15.3 Percorsi relativi e assoluti

Un **percorso relativo** indica dove si trova un file a partire dalla cartella di lavoro del programma — non dal disco. Il programma non dipende dalla macchina e funziona anche se sposti il progetto.

Un **percorso assoluto** parte dalla radice del disco — è fragile e dipende dalla macchina.

```
// Percorso relativo - preferibile
Path p1 = Path.of("dati.txt");
Path p2 = Path.of("Resources", "dati.txt");  // cartella + file separati

// Percorso assoluto - da evitare nei progetti
Path p3 = Path.of("/home/utente/gioco/dati.txt");
```

Quando usi `Path.of("Resources", "dati.txt")` stai passando cartella e file come argomenti separati — Java costruisce il percorso corretto per il sistema operativo.

Quale sintassi preferire?

```
// Preferibile - Java gestisce i separatori
Path p = Path.of("Resources", "dati.txt");

// Funziona ma è meno portabile
Path p = Path.of("Resources/dati.txt");
```

Entrambe funzionano, ma la prima è più robusta se il programma gira su sistemi diversi.

15.4 Path vs Files: separazione delle responsabilità

Java separa nettamente due concetti:

- **`Path`** — descrive **dove** si trova una risorsa
- **`Files`** — descrive **cosa** vogliamo fare con quella risorsa

```
import java.nio.file.*;

Path file = Path.of("dati.txt");   // posizione
Files.readAllLines(file);          // operazione
```

`Files` è una classe di soli metodi statici — non si creano oggetti `Files`, si usano i suoi metodi passando un `Path`. Questo rende il codice molto leggibile: prima dichiari dove si trova il file, poi dici cosa vuoi farci.

Poiché `Files` lavora davvero con il filesystem, i suoi metodi possono fallire — e Java ci obbliga a gestire la `IOException`.

15.5 Operazioni di base su Path

```
Path file = Path.of("dati", "livello", "mappa.json");

file.getFileName();       // mappa.json
file.getParent();         // dati/livello
file.toAbsolutePath();    // percorso assoluto completo

// Combinare percorsi
Path base = Path.of("dati");
Path completo = base.resolve("livello/mappa.json");  // dati/livello/mappa.json

// Normalizzare (rimuove . e ..)
Path.of("dati/../dati/./mappa.json").normalize();    // dati/mappa.json
```

15.6 Verificare esistenza e tipo

```
Path file = Path.of("salvataggio.txt");

Files.exists(file);          // il file esiste?
Files.notExists(file);       // il file NON esiste?
Files.isRegularFile(file);   // è un file normale?
Files.isDirectory(file);     // è una directory?
Files.isReadable(file);      // è leggibile?
Files.isWritable(file);      // è scrivibile?
Files.size(file);            // dimensione in byte
```

15.7 Leggere un file: tre approcci

15.7.1 `readAllLines()` — tutto in memoria

```
try {
    List<String> righe = Files.readAllLines(Path.of("squadra.txt"));
    for (String riga : righe) {
        System.out.println(riga);
    }
} catch (IOException e) {
    System.out.println("Errore: " + e.getMessage());
}
```

Comodo e leggibile. Ma carica tutto il file in RAM — adatto solo per file piccoli.

15.7.2 `readString()` — come unica stringa

```
try {
    String contenuto = Files.readString(Path.of("config.txt"));
    System.out.println(contenuto);
} catch (IOException e) {
    System.out.println("Errore: " + e.getMessage());
}
```

15.7.3 `Files.lines()` — stream lazy

```
try (Stream<String> righe = Files.lines(Path.of("log.txt"))) {
    righe
        .filter(r -> !r.isBlank())
        .forEach(System.out::println);
} catch (IOException e) {
    System.out.println("Errore: " + e.getMessage());
}
```

Qui non stiamo dicendo "leggimi tutto" — stiamo dicendo "forniscimi una sequenza di righe". Il file viene letto un po' alla volta, man mano che lo stream avanza. Può elaborare un file enorme senza mai tenerlo tutto in memoria. È il primo ponte verso il concetto di buffer.

> **!** Lo stream collegato a un file è una risorsa
>
> `Files.lines()` restituisce uno stream che tiene aperto il file. Va chiuso quando hai finito. Per questo lo mettiamo tra le parentesi del `try-with-resources` — Java lo chiude automaticamente all'uscita dal blocco, anche in caso di errore.
> Inoltre: **uno stream si può usare una sola volta**. Se vuoi fare più analisi sullo stesso file, devi riaprirlo ogni volta.

15.8 Scrivere un file

```
// Scrittura semplice (crea o sovrascrive)
try {
    Files.writeString(Path.of("output.txt"), "Contenuto\n");
} catch (IOException e) {
    System.out.println("Errore: " + e.getMessage());
}

// Scrittura di più righe
try {
    Files.write(Path.of("output.txt"), List.of("riga 1", "riga 2", "riga 3"));
} catch (IOException e) {
    System.out.println("Errore: " + e.getMessage());
```

```
}

// Aggiunta in fondo (append)
try {
    Files.writeString(Path.of("log.txt"), "Nuova riga\n",
        StandardOpenOption.APPEND, StandardOpenOption.CREATE);
} catch (IOException e) {
    System.out.println("Errore: " + e.getMessage());
}
```

> **Sovrascrittura vs append**
>
> Quando scrivi su file, poniti sempre questa domanda: sto sostituendo il contenuto o lo sto aggiungendo?
>
> - **Sovrascrittura** — il file viene ricreato ad ogni esecuzione. Tipico dei **report** (voglio l'ultimo risultato).
> - **Append** — il nuovo testo si aggiunge al contenuto esistente. Tipico dei **log** (voglio una cronologia).
>
> Questa scelta è importante perché cambia il significato del file prodotto.

15.9 Cartelle di output: buona pratica

È buona pratica separare i file di input da quelli di output usando cartelle dedicate (`Reports/`, `Output/`, `Logs/`). Un programma robusto controlla che la cartella esista e la crea se non c'è:

```
private static final Path INPUT       = Path.of("accessi.txt");
private static final Path REPORTS_DIR = Path.of("Reports");
private static final Path REPORT_FILE = REPORTS_DIR.resolve("report.txt");

// Prima creo la cartella, poi scrivo
Files.createDirectories(REPORTS_DIR);
Files.writeString(REPORT_FILE, contenuto, StandardCharsets.UTF_8);
```

15.10 Esempio completo: analisi log di accessi

Un programma che legge un file di log, calcola statistiche e genera un report.

Il file `accessi.txt`:

```
2025-12-18 08:01;alice;LOGIN
2025-12-18 08:05;bob;LOGIN
2025-12-18 08:20;alice;LOGOUT
2025-12-18 08:45;carla;LOGIN
2025-12-18 09:10;bob;LOGOUT
2025-12-18 09:30;alice;LOGIN
```

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.time.LocalDateTime;
import java.util.stream.Stream;

public class AnalisiAccessi {

    private static final Path INPUT       = Path.of("accessi.txt");
    private static final Path REPORTS_DIR = Path.of("Reports");
    private static final Path REPORT_FILE = REPORTS_DIR.resolve("report.txt");

    public static void main(String[] args) {

        if (!Files.exists(INPUT)) {
            System.out.println("Errore: file non trovato → " + INPUT);
            return;
        }

        // Ogni calcolo riapre il file - uno stream non si riusa
        long righeTotali    = contaRighe(INPUT);
        long righeValide    = contaRigheValide(INPUT);
        long login          = contaEvento(INPUT, "LOGIN");
        long logout         = contaEvento(INPUT, "LOGOUT");
        long utenti         = contaUtentiDistinti(INPUT);
        long malformate     = righeTotali - righeValide;

        // Prima costruisco il report in memoria, poi lo scrivo su file
        String report = buildReport(righeTotali, righeValide,
```

```java
            malformate, login, logout, utenti);

        try {
            Files.createDirectories(REPORTS_DIR);
            Files.writeString(REPORT_FILE, report, StandardCharsets.UTF_8);
            System.out.println("Report salvato in: " + REPORT_FILE);
        } catch (IOException e) {
            System.out.println("Errore scrittura: " + e.getMessage());
        }
    }

    private static long contaRighe(Path p) {
        try (Stream<String> righe = Files.lines(p, StandardCharsets.UTF_8)) {
            return righe.count();
        } catch (IOException e) { return 0; }
    }

    private static long contaRigheValide(Path p) {
        try (Stream<String> righe = Files.lines(p, StandardCharsets.UTF_8)) {
            return righe.map(r -> r.split(";"))
                        .filter(c -> c.length == 3).count();
        } catch (IOException e) { return 0; }
    }

    private static long contaEvento(Path p, String evento) {
        try (Stream<String> righe = Files.lines(p, StandardCharsets.UTF_8)) {
            return righe.map(r -> r.split(";"))
                        .filter(c -> c.length == 3)
                        .filter(c -> c[2].equals(evento)).count();
        } catch (IOException e) { return 0; }
    }

    private static long contaUtentiDistinti(Path p) {
        try (Stream<String> righe = Files.lines(p, StandardCharsets.UTF_8)) {
            return righe.map(r -> r.split(";"))
                        .filter(c -> c.length == 3)
                        .map(c -> c[1].trim())
                        .filter(s -> !s.isEmpty())
                        .distinct().count();
        } catch (IOException e) { return 0; }
    }
```

```
    private static String buildReport(long tot, long valid, long bad,
                                      long login, long logout, long utenti) {
        return "REPORT ACCESSI\n"
             + "==============\n"
             + "Generato: " + LocalDateTime.now() + "\n\n"
             + "Righe totali:     " + tot    + "\n"
             + "Righe valide:     " + valid  + "\n"
             + "Righe malformate: " + bad    + "\n\n"
             + "LOGIN:            " + login  + "\n"
             + "LOGOUT:           " + logout + "\n\n"
             + "Utenti distinti:  " + utenti + "\n";
    }
}
```

15.11 Log: registrare gli eventi

```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.time.*;
import java.time.format.*;
import java.io.IOException;

public class Logger {

    private final Path fileLog;
    private final DateTimeFormatter formatter =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    public Logger(String percorso) {
        this.fileLog = Path.of(percorso);
        try {
            Files.createDirectories(fileLog.getParent());
        } catch (IOException e) {
            System.out.println("Impossibile creare cartella log: " + e.getMessage());
        }
    }
```

```java
    public void log(String livello, String messaggio) {
        String riga = "[" + LocalDateTime.now().format(formatter) + "] "
                    + "[" + livello + "] " + messaggio + "\n";
        try {
            Files.writeString(fileLog, riga,
                StandardCharsets.UTF_8,
                StandardOpenOption.CREATE,
                StandardOpenOption.APPEND);
        } catch (IOException e) {
            System.out.println("Errore log: " + e.getMessage());
        }
    }

    public void info(String msg)  { log("INFO ", msg); }
    public void warn(String msg)  { log("WARN ", msg); }
    public void error(String msg) { log("ERROR", msg); }
}
```

15.12 Gestione delle date con `java.time`

```java
import java.time.*;
import java.time.format.*;
import java.time.temporal.ChronoUnit;

LocalDate oggi       = LocalDate.now();
LocalTime ora        = LocalTime.now();
LocalDateTime adesso = LocalDateTime.now();

LocalDate natale = LocalDate.of(2025, 12, 25);

LocalDate domani = oggi.plusDays(1);
boolean passato  = natale.isBefore(oggi);

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
String formattata = adesso.format(fmt);

long giorni = ChronoUnit.DAYS.between(oggi, natale);
```

Perché `java.time` e non `Date`?

La vecchia classe `java.util.Date` è mutabile, con mesi che partono da 0 e gestione del fuso orario confusa. `java.time` (Java 8+) è immutabile, chiara e corretta. Usa sempre `java.time`.

15.13 Riepilogo

I concetti chiave

- **`Path`** rappresenta **dove** si trova una risorsa — non verifica nulla, non legge nulla.
- **`Files`** rappresenta **cosa** fare con quella risorsa — può fallire con `IOException`.
- Usa `Path.of("cartella", "file.txt")` — Java gestisce i separatori in modo portabile.
- I percorsi **relativi** sono preferibili — il codice funziona su qualsiasi macchina.
- `Files.readAllLines()` carica tutto in RAM — adatto per file piccoli.
- `Files.lines()` è **lazy** — legge man mano, adatto per file grandi. Va chiuso con try-with-resources.
- Uno stream collegato a un file si usa **una sola volta** — riaprilo per ogni analisi.
- Separa la **costruzione dei dati** dalla loro **scrittura su file**.
- Usa **`java.time`** per gestire date e ore — mai `java.util.Date`.

16 Buffer e scrittura

16.1 Il principio del buffer

Nella lezione precedente abbiamo usato `Files.writeString()` e `Files.readAllLines()` — comode e leggibili. Ma c'è un principio importante che vale sia per i file che per le stringhe:

> **Evitare tante operazioni piccole e costose, accumulando i dati prima di scrivere o elaborare.**

Accedere al disco è costoso. Farlo molte volte per piccoli pezzi di dati è lento. Il **buffer** è una zona di memoria RAM usata come zona di appoggio tra il programma e il file: accumula i dati e li scarica sul disco in un'unica operazione, riducendo drasticamente il numero di accessi.

Quello che cambia tra le diverse API è **chi gestisce il buffer** e **quanto è visibile**.

16.2 Buffer impliciti: le API di alto livello

Quando usi metodi come:

```
Files.readString(p);
Files.readAllLines(p);
Files.writeString(p, testo);
```

il buffering esiste, ma è completamente **nascosto**. Java legge o scrive il file usando buffer interni, poi ti restituisce il risultato finito.

Questi metodi sono comodi, sicuri e leggibili. Ma implicano una scelta precisa: **tutto il contenuto viene caricato in memoria**. Sono indicati per file piccoli — configurazioni, testi brevi, output di report.

> **L'idea di base in Java moderno**
>
> Non si parte dai buffer. Si parte da API di alto livello, già ottimizzate, e si scende solo quando serve — cioè quando il file è grande, quando si lavora riga per riga con logica complessa, o quando si vuole controllo fine sul flusso dei dati.

16.3 Il primo passo verso il buffer: `Files.lines()`

`Files.lines()` è già un passo verso il buffering: non carica tutto, consuma i dati mentre arrivano.

```
try (Stream<String> righe = Files.lines(p)) {
    righe.forEach(System.out::println);
}
```

Questo codice può stampare un file enorme senza mai tenerlo tutto in memoria. Il buffer c'è, ma non lo vediamo.

16.4 Buffer espliciti su testo: BufferedReader e BufferedWriter

Quando vogliamo controllo esplicito, usiamo `BufferedReader` e `BufferedWriter`. Il buffer non è più un dettaglio tecnico nascosto — diventa parte del nostro modello.

16.4.1 BufferedReader: leggere riga per riga

```
import java.nio.file.*;
import java.io.*;

Path p = Path.of("nomi.txt");

try (BufferedReader br = Files.newBufferedReader(p)) {
    String riga;
```

```
    while ((riga = br.readLine()) != null) {
        System.out.println("Letto: " + riga);
    }
}
```

Dichiarando `BufferedReader` stai dicendo esplicitamente tre cose: 1. il file viene letto **a righe**, non come blocco unico 2. il file viene **consumato progressivamente** 3. c'è un'area di memoria che accumula dati prima di consegnarteli — il buffer è parte del tuo modello

Il `try-with-resources` garantisce che il file venga aperto una sola volta, letto progressivamente, e **chiuso automaticamente** alla fine — anche se scatta un'eccezione nel mezzo.

16.4.2 Gestione delle eccezioni dentro il try

```
Path p = Path.of("numeri.txt");

try (BufferedReader br = Files.newBufferedReader(p)) {
    String riga;
    while ((riga = br.readLine()) != null) {
        int n = Integer.parseInt(riga);  // può lanciare NumberFormatException
        System.out.println(n * 2);
    }
} catch (IOException e) {
    System.out.println("Errore di lettura file");
}
```

Se una riga non è un numero, il programma va in errore. Ma il file viene comunque chiuso. Questo è il punto chiave: **la chiusura della risorsa non dipende dal fatto che il codice finisca bene**. La gestione delle risorse è separata dalla gestione degli errori.

16.4.3 Due risorse nello stesso try

```
Path src = Path.of("input.txt");
Path dst = Path.of("output.txt");

try (
    BufferedReader br = Files.newBufferedReader(src);
    BufferedWriter bw = Files.newBufferedWriter(dst)
```

```
) {
    String riga;
    while ((riga = br.readLine()) != null) {
        bw.write(riga.toUpperCase());
        bw.newLine();
    }
}
```

Java garantisce che prima venga chiuso il writer (svuotando i buffer di scrittura), poi il reader. Questo ordine è importante — e tu non devi ricordartelo: è parte del contratto del try-with-resources.

16.4.4 BufferedWriter: scrivere con buffer

```
Path file = Path.of("output.txt");

try (BufferedWriter bw = Files.newBufferedWriter(file)) {
    for (int i = 1; i <= 1000; i++) {
        bw.write("Riga " + i);
        bw.newLine();  // usa il separatore corretto per il sistema operativo
    }
}
```

Invece di mille accessi al disco, il buffer accumula le righe in memoria e le scarica in poche operazioni. Molto più efficiente.

> **`newLine()` vs `"\n"`**
>
> Usa sempre `bw.newLine()` invece di `"\n"`. Il metodo scrive il carattere di a capo corretto per il sistema operativo: `\n` su Linux/Mac, `\r\n` su Windows.

16.4.5 Append con BufferedWriter

```
try (BufferedWriter bw = Files.newBufferedWriter(file,
        StandardOpenOption.APPEND,
        StandardOpenOption.CREATE)) {
    bw.write("Riga aggiunta");
    bw.newLine();
}
```

16.4.6 PrintWriter: testo formattato

`PrintWriter` aggiunge `println()` e `printf()` — ideale per report tabellari:

```
try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(file))) {
    pw.println("=== REPORT ===");
    pw.printf("%-15s %5s %5s%n", "Nome", "Vita", "Lv");
    pw.printf("%-15s %5d %5d%n", "Arthas", 150, 5);
    pw.printf("%-15s %5d %5d%n", "Gandalf", 100, 7);
}
```

16.5 L'analogia tra file e stringhe

Il principio del buffer vale in entrambi i contesti:

Situazione	Senza buffer	Con buffer
File	scrittura continua byte per byte	`BufferedWriter`
Stringhe	concatenazione in ciclo con `+`	`StringBuilder`

`BufferedWriter` bufferizza il **disco**. `StringBuilder` bufferizza la **memoria**. Stessa idea, contesti diversi.

16.6 Buffer su flussi binari

Per i file di testo ha senso parlare di righe e caratteri. Per i **file binari** — immagini, audio, zip, file `.class` — no: sono solo sequenze di byte, senza significato testuale.

In quel caso si usano `BufferedInputStream` e `BufferedOutputStream`:

```
import java.io.*;
import java.nio.file.*;

Path src = Path.of("immagine.png");
Path dst = Path.of("copia.png");
```

```java
try (
    BufferedInputStream in  = new BufferedInputStream(Files.newInputStream(src));
    BufferedOutputStream out = new BufferedOutputStream(Files.newOutputStream(dst))
) {
    byte[] buffer = new byte[4096];  // il buffer è concreto e visibile
    int letti;
    while ((letti = in.read(buffer)) != -1) {
        out.write(buffer, 0, letti);
    }
}
```

Qui il buffer non è più astratto: è un array di byte che rappresenta fisicamente quanti dati leggi alla volta. `read()` restituisce il numero di byte effettivamente letti — l'ultima lettura può essere parziale, quindi si scrive solo `letti` byte, non tutti i 4096.

> **Perché 4096?**
>
> 4096 byte (4 KB) è una dimensione comune perché corrisponde tipicamente alla dimensione di un blocco del filesystem. Leggere o scrivere blocchi allineati al filesystem è più efficiente.

16.7 Buffer specializzati: DataInputStream e DataOutputStream

`DataInputStream` e `DataOutputStream` non introducono un nuovo tipo di buffer — si appoggiano a uno già esistente. La loro specialità è leggere e scrivere **tipi primitivi in formato binario**: `int`, `double`, `boolean`, `long` e così via.

Senza questi stream, per salvare un intero dovresti convertirlo in stringa, scriverlo, poi rileggerlo e convertirlo di nuovo. Con `DataOutputStream` scrivi direttamente il valore binario — più compatto e più veloce.

```java
import java.io.*;
import java.nio.file.*;

Path file = Path.of("dati.bin");

// Scrittura - avvolge un BufferedOutputStream
try (DataOutputStream out = new DataOutputStream(
        new BufferedOutputStream(Files.newOutputStream(file)))) {
```

```
    out.writeInt(42);           // scrive 4 byte
    out.writeDouble(3.14);      // scrive 8 byte
    out.writeBoolean(true);     // scrive 1 byte
    out.writeUTF("Arthas");     // scrive la stringa in formato UTF-8 binario

}

// Lettura - avvolge un BufferedInputStream
try (DataInputStream in = new DataInputStream(
        new BufferedInputStream(Files.newInputStream(file)))) {

    int n       = in.readInt();
    double d    = in.readDouble();
    boolean b   = in.readBoolean();
    String nome = in.readUTF();

    System.out.println(n + " " + d + " " + b + " " + nome);
    // 42 3.14 true Arthas
}
```

> **!** L'ordine di lettura deve essere identico all'ordine di scrittura
>
> Il file binario non ha metadati — è solo una sequenza di byte. Java non sa cosa rappresenta ogni byte finché non glielo dici tu. Se scrivi `int`, `double`, `boolean` in quest'ordine, devi leggere esattamente nello stesso ordine. Un errore di ordine produce valori senza senso, senza nessun messaggio di errore.

16.7.1 Quando usarli

`DataInputStream`/`DataOutputStream` sono utili quando: - vuoi salvare dati strutturati in formato **compatto e non leggibile** da un editor di testo - stai implementando un **formato di salvataggio binario** (es. file di salvataggio di un gioco) - lavori con **protocolli di rete** che trasmettono dati primitivi in formato binario

Non usarli quando i dati devono essere leggibili da un essere umano o da altri programmi — in quel caso usa CSV o JSON in formato testo.

16.8 Il quadro completo dei buffer

Ricapitolando tutti i livelli che abbiamo visto:

Strumento	Gestisce	Visibilità buffer	Quando usarlo
`Files.readString()` / `writeString()`	char	Implicito, nascosto	File piccoli, semplicità
`Files.lines()`	char	Implicito, lazy	File grandi con Stream
`BufferedReader` / `BufferedWriter`	char	Esplicito, riga per riga	File di testo grandi
`BufferedInputStream` / `BufferedOutputStream`	byte	Esplicito, array di byte	File binari, copie
`DataInputStream` / `DataOutputStream`	tipi primitivi	Eredita da Buffered*	Salvataggi binari strutturati

> **ℹ La progressione concettuale**
>
> ```
> Files.readString()              → buffer implicito, tutto in RAM
> Files.lines()                   → buffer implicito, lazy
> BufferedReader / Writer         → buffer esplicito, testo (char)
> BufferedInputStream / Output    → buffer esplicito, binario (byte)
> DataInputStream / Output        → buffer ereditato, tipi primitivi
> ```
>
> Si scende di livello solo quando serve.

16.9 La codifica dei caratteri

Quando lavori con file di testo, specifica sempre **UTF-8** esplicitamente per evitare problemi con caratteri speciali su sistemi diversi:

```
import java.nio.charset.StandardCharsets;

try (BufferedWriter bw = Files.newBufferedWriter(
        file, StandardCharsets.UTF_8)) {
    bw.write("Testo con accenti: à è ì ò ù");
    bw.newLine();
}
```

```
try (BufferedReader br = Files.newBufferedReader(
        file, StandardCharsets.UTF_8)) {
    System.out.println(br.readLine());
}
```

16.10 Flush: svuotare il buffer manualmente

Normalmente il buffer viene svuotato automaticamente alla chiusura. Ma in alcuni casi, come un log in tempo reale, vuoi che i dati vengano scritti subito:

```
try (BufferedWriter bw = Files.newBufferedWriter(
        Path.of("live.log"), StandardOpenOption.APPEND)) {

    for (int i = 0; i < 5; i++) {
        bw.write("Evento " + i);
        bw.newLine();
        bw.flush();  // scrive immediatamente sul disco

        Thread.sleep(1000);
    }
} catch (IOException | InterruptedException e) {
    System.out.println("Errore: " + e.getMessage());
}
```

Senza `flush()`, gli eventi rimarrebbero nel buffer e verrebbero scritti tutti insieme alla chiusura.

16.11 Il quadro completo: quando usare cosa

Strumento	Tipo	Quando usarlo
`Files.readString()`	testo, implicito	file piccoli, lettura rapida
`Files.readAllLines()`	testo, implicito	file piccoli, serve una List
`Files.writeString()`	testo, implicito	scrittura semplice

Strumento	Tipo	Quando usarlo
`Files.lines()`	testo, lazy	file grandi con Stream
`BufferedReader`	testo, esplicito	lettura riga per riga, file grandi
`BufferedWriter`	testo, esplicito	scrittura incrementale, molte righe
`PrintWriter`	testo, formattato	report con `printf`
`BufferedInputStream`	binario	copia file, immagini, audio
`BufferedOutputStream`	binario	scrittura file binari

> **La progressione concettuale**
>
> ```
> Files.readString()         → buffer implicito, tutto in RAM
> Files.lines()              → buffer implicito, lazy (primo passo)
> BufferedReader/Writer      → buffer esplicito su testo
> BufferedInputStream/Output → buffer esplicito su byte
> ```
>
> Si scende di livello solo quando serve: file grandi, prestazioni, dati binari.

16.12 Esempio completo: logger con buffer e rotazione giornaliera

```java
import java.nio.file.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.time.*;
import java.time.format.*;

public class Logger {

    private final Path cartella;
    private final DateTimeFormatter fmtData =
        DateTimeFormatter.ofPattern("yyyy-MM-dd");
    private final DateTimeFormatter fmtOra =
        DateTimeFormatter.ofPattern("HH:mm:ss");

    public Logger(String cartella) {
```

```java
        this.cartella = Path.of(cartella);
        try {
            Files.createDirectories(this.cartella);
        } catch (IOException e) {
            System.out.println("Impossibile creare cartella log: " + e.getMessage());
        }
    }

    // Un file di log diverso per ogni giorno
    private Path fileOggi() {
        return cartella.resolve("gioco-" + LocalDate.now().format(fmtData) + ".log");
    }

    private void scrivi(String livello, String messaggio) {
        String riga = "[" + LocalTime.now().format(fmtOra) + "] "
                    + "[" + livello + "] " + messaggio;

        try (BufferedWriter bw = Files.newBufferedWriter(
                fileOggi(),
                StandardCharsets.UTF_8,
                StandardOpenOption.CREATE,
                StandardOpenOption.APPEND)) {

            bw.write(riga);
            bw.newLine();

        } catch (IOException e) {
            System.err.println("Errore log: " + e.getMessage());
        }
    }

    public void info(String msg)  { scrivi("INFO ", msg); }
    public void warn(String msg)  { scrivi("WARN ", msg); }
    public void error(String msg) { scrivi("ERROR", msg); }

    public void stampaLog() {
        Path file = fileOggi();
        if (!Files.exists(file)) {
            System.out.println("Nessun log per oggi.");
            return;
        }
```

```
        System.out.println("=== LOG " + LocalDate.now() + " ===");
        try (BufferedReader br = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
            String riga;
            while ((riga = br.readLine()) != null) {
                System.out.println(riga);
            }
        } catch (IOException e) {
            System.out.println("Errore lettura: " + e.getMessage());
        }
    }
}
```

16.13 Riepilogo

I concetti chiave

- Il **buffer** è una zona di RAM usata come appoggio tra il programma e il disco — riduce il numero di accessi.
- Le API di alto livello (`Files.readString`, `Files.writeString`) usano buffer **impliciti** — tutto in memoria, adatte per file piccoli.
- `Files.lines()` è **lazy** — primo passo verso il buffer, non carica tutto.
- **`BufferedReader`** e **`BufferedWriter`** rendono il buffer **esplicito** — per testo, riga per riga.
- Il **try-with-resources** chiude le risorse automaticamente, nell'ordine corretto, anche in caso di eccezione.
- **`BufferedInputStream`** e **`BufferedOutputStream`** per file **binari** — il buffer diventa un array di byte concreto e misurabile.
- Specifica sempre **`StandardCharsets.UTF_8`** per evitare problemi con caratteri speciali.
- Usa `flush()` per scrivere immediatamente senza aspettare la chiusura.
- Si parte dall'**alto livello** e si scende solo quando serve.

Part VII

Qualità del codice

17 Testing con JUnit

17.1 Perché testare il codice?

Hai scritto una classe `Personaggio` con setter che controllano i valori. Come sai che funzionano davvero? Provi a mano ogni volta che modifichi qualcosa?

Il problema del testing manuale è che è: - **lento** — devi ricordarti cosa testare ogni volta - **inaffidabile** — dimentichi casi limite - **non ripetibile** — non puoi rieseguirlo automaticamente dopo ogni modifica

I **test automatici** risolvono tutti questi problemi: li scrivi una volta, li esegui in un secondo ogni volta che modifichi il codice. Se qualcosa si rompe, lo scopri subito.

> **Il principio di base**
>
> Un test automatico è un pezzo di codice che verifica che un altro pezzo di codice si comporti come previsto. Se il comportamento cambia inaspettatamente, il test fallisce e ti avvisa.

17.2 JUnit 5: il framework di testing

JUnit è il framework di testing più usato in Java. La versione attuale è JUnit 5. Permette di scrivere test in modo semplice e leggibile.

Per usarlo in un progetto Maven, aggiungi al `pom.xml`:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
</dependency>
```

Per un progetto semplice senza Maven, puoi scaricare il jar da junit.org.

17.3 Struttura di una classe di test

Per convenzione, la classe di test ha lo stesso nome della classe testata con suffisso `Test`, e si trova nella cartella `src/test/java`:

```
src/
  main/java/
      Personaggio.java       ← classe da testare
  test/java/
      PersonaggioTest.java   ← classe di test
```

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class PersonaggioTest {

    private Personaggio eroe;

    // Eseguito prima di OGNI test - ricrea l'oggetto pulito
    @BeforeEach
    void setUp() {
        eroe = new Personaggio("Arthas", 150, 30, 1);
    }

    // Eseguito dopo OGNI test - pulizia
    @AfterEach
    void tearDown() {
        // eventuali risorse da chiudere
    }
```

```java
    // Eseguito una volta sola prima di tutti i test della classe
    @BeforeAll
    static void setUpClass() {
        System.out.println("Inizio test PersonaggioTest");
    }

    // Eseguito una volta sola dopo tutti i test
    @AfterAll
    static void tearDownClass() {
        System.out.println("Fine test PersonaggioTest");
    }

    @Test
    void testCreazione() {
        assertEquals("Arthas", eroe.getNome());
        assertEquals(150, eroe.getVitaMax());
        assertEquals(150, eroe.getVita());  // parte con vita piena
        assertEquals(30, eroe.getAttacco());
        assertEquals(1, eroe.getLivello());
    }
}
```

17.4 Le asserzioni principali

Il cuore di ogni test è l'**asserzione** — una verifica che deve essere vera. Se è falsa, il test fallisce.

```java
// Uguaglianza
assertEquals(150, eroe.getVita());          // expected, actual
assertEquals("Arthas", eroe.getNome());

// Uguaglianza con delta (per i double)
assertEquals(3.14, Math.PI, 0.01);          // tolleranza di 0.01

// Disuguaglianza
assertNotEquals(0, eroe.getVita());

// Booleani
```

```
assertTrue(eroe.getVita() > 0);
assertFalse(eroe.isMorto());

// Null
assertNull(eroe.getArma());          // se Personaggio ha getArma()
assertNotNull(eroe.getNome());

// Stesso oggetto in memoria
assertSame(eroe, eroe);
assertNotSame(eroe, new Personaggio("Arthas", 150, 30, 1));

// Verifica che un'eccezione venga lanciata
assertThrows(IllegalArgumentException.class, () -> {
    eroe.setVita(-100);
});

// Tutti i test in un blocco (tutti eseguiti anche se uno fallisce)
assertAll("statistiche",
    () -> assertEquals("Arthas", eroe.getNome()),
    () -> assertEquals(150, eroe.getVitaMax()),
    () -> assertFalse(eroe.isMorto())
);
```

17.5 Scrivere test significativi

Un buon test verifica **un comportamento specifico** — non tutto insieme. Il nome del metodo deve descrivere esattamente cosa si sta testando.

```
class PersonaggioTest {

    private Personaggio eroe;

    @BeforeEach
    void setUp() {
        eroe = new Personaggio("Arthas", 150, 30, 1);
    }

    // --- Test costruttore ---
```

```java
    @Test
    void nuovoPersonaggioParteConVitaPiena() {
        assertEquals(eroe.getVitaMax(), eroe.getVita());
    }

    @Test
    void vitaMaxNegativaVieneNormalizzata() {
        Personaggio p = new Personaggio("Test", -50, 10, 1);
        assertTrue(p.getVitaMax() > 0);
    }

    // --- Test subirDanno ---

    @Test
    void subirDannoRiduceVita() {
        eroe.subirDanno(30);
        assertEquals(120, eroe.getVita());
    }

    @Test
    void subirDannoMaggioreDellaVitaPortaAZero() {
        eroe.subirDanno(9999);
        assertEquals(0, eroe.getVita());
    }

    @Test
    void subirDannoZeroNonModificaVita() {
        int vitaPrima = eroe.getVita();
        eroe.subirDanno(0);
        assertEquals(vitaPrima, eroe.getVita());
    }

    @Test
    void dopoMortaIsMortoRestituisceTrue() {
        eroe.subirDanno(9999);
        assertTrue(eroe.isMorto());
    }

    // --- Test salaDiLivello ---

    @Test
    void salaDiLivelloAumentaIlLivello() {
```

```java
        eroe.salaDiLivello();
        assertEquals(2, eroe.getLivello());
    }

    @Test
    void salaDiLivelloRipristinaVitaPiena() {
        eroe.subirDanno(50);
        eroe.salaDiLivello();
        assertEquals(eroe.getVitaMax(), eroe.getVita());
    }

    @Test
    void salaDiLivelloAumentaAttacco() {
        int attaccoPrima = eroe.getAttacco();
        eroe.salaDiLivello();
        assertTrue(eroe.getAttacco() > attaccoPrima);
    }

    // --- Test setter ---

    @Test
    void setVitaConValoreNegativoPortaAZero() {
        eroe.setVita(-10);
        assertEquals(0, eroe.getVita());
    }

    @Test
    void setVitaSopraVitaMaxVieneClampata() {
        eroe.setVita(9999);
        assertEquals(eroe.getVitaMax(), eroe.getVita());
    }

    @Test
    void setAttaccoNegativoNonModificaAttacco() {
        int attaccoPrima = eroe.getAttacco();
        eroe.setAttacco(-5);
        assertEquals(attaccoPrima, eroe.getAttacco());
    }
}
```

17.6 Test delle eccezioni

Verificare che il codice lanci le eccezioni giuste è fondamentale:

```
@Test
void setVitaConValoreNegativoLanciaEccezione() {
    // Se il setter lancia IllegalArgumentException con valori negativi:
    IllegalArgumentException ex = assertThrows(
        IllegalArgumentException.class,
        () -> eroe.setVita(-100)
    );

    // Puoi anche verificare il messaggio dell'eccezione
    assertTrue(ex.getMessage().contains("negativa") ||
               ex.getMessage().contains("negativo"));
}

@Test
void costruttoreConNomeNulloLanciaEccezione() {
    assertThrows(IllegalArgumentException.class, () ->
        new Personaggio(null, 150, 30, 1)
    );
}
```

17.7 Test parametrizzati

Quando vuoi testare lo stesso comportamento con valori diversi, i **test parametrizzati** evitano la duplicazione:

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;

class PersonaggioTest {

    private Personaggio eroe;

    @BeforeEach
    void setUp() {
```

```
        eroe = new Personaggio("Arthas", 150, 30, 1);
    }

    // Test con valori singoli
    @ParameterizedTest
    @ValueSource(ints = {0, -1, -100, Integer.MIN_VALUE})
    void setVitaNegativaPortaSempreAZero(int vitaNegativa) {
        eroe.setVita(vitaNegativa);
        assertEquals(0, eroe.getVita());
    }

    // Test con coppie di valori (danno, vitaAttesa)
    @ParameterizedTest
    @CsvSource({
        "0, 150",    // nessun danno → vita rimane 150
        "30, 120",   // danno 30 → vita 120
        "150, 0",    // danno = vita → vita 0
        "200, 0"     // danno > vita → vita 0
    })
    void subirDannoCalcolaVitaCorretta(int danno, int vitaAttesa) {
        eroe.subirDanno(danno);
        assertEquals(vitaAttesa, eroe.getVita());
    }
}
```

17.8 Piano di test

Prima di scrivere i test, conviene pianificarli. Un **piano di test** elenca cosa verificare per ogni metodo, includendo i casi normali e i casi limite.

Per la classe `Personaggio`:

Metodo	Caso da testare	Risultato atteso
Costruttore	Valori validi	Oggetto creato con vita = vitaMax
Costruttore	`vitaMax` negativo	`vitaMax` normalizzato a valore positivo
Costruttore	Nome null/vuoto	Eccezione o nome di default

Metodo	Caso da testare	Risultato atteso
`subirDanno()`	Danno < vita	Vita diminuisce esattamente di `danno`
`subirDanno()`	Danno = vita	Vita = 0, `isMorto()` = true
`subirDanno()`	Danno > vita	Vita = 0, non negativa
`subirDanno()`	Danno = 0	Vita invariata
`setVita()`	Valore valido	Vita aggiornata
`setVita()`	Valore negativo	Vita = 0
`setVita()`	Valore > vitaMax	Vita = vitaMax
`salaDiLivello()`	Prima chiamata	Livello = 2, vita piena
`isMorto()`	Vita > 0	false
`isMorto()`	Vita = 0	true

I casi limite sono i più importanti

I test più utili sono quelli sui **casi limite** (edge cases): valori a zero, negativi, null, stringhe vuote, liste vuote. Questi sono i valori con cui il codice si comporta spesso in modo inatteso.

17.9 Test-Driven Development (TDD)

Il **TDD** è un approccio in cui scrivi i test **prima** del codice di produzione:

1. **Red** — scrivi un test che fallisce (il codice non esiste ancora)
2. **Green** — scrivi il minimo codice necessario per far passare il test
3. **Refactor** — migliora il codice senza rompere i test

```java
// 1. RED - scrivo il test prima di avere il metodo
@Test
void puntiEsperienzaBasatiSuLivello() {
    Personaggio p = new Personaggio("Arthas", 150, 30, 3);
    assertEquals(300, p.getPuntiEsperienza());  // il metodo non esiste ancora!
}

// 2. GREEN - aggiungo il metodo minimo per far passare il test
public int getPuntiEsperienza() {
    return livello * 100;
```

```
}

// 3. REFACTOR - miglioro se necessario
public int getPuntiEsperienza() {
    return livello * livello * 50;  // formula più interessante
}
// il test mi dice subito se ho rotto qualcosa
```

> **Vantaggi del TDD**
>
> - Forza a pensare all'interfaccia prima dell'implementazione
> - Garantisce che ogni riga di codice sia coperta da almeno un test
> - Il refactoring diventa sicuro — i test ti dicono se hai rotto qualcosa

---

17.10 Esempio completo: test della classe Arma

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.*;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;

class ArmaTest {

    private Arma spada;

    @BeforeEach
    void setUp() {
        spada = new Arma("Spada del Caos", 50);
    }

    @Test
    void nuovaArmaParte conDurabilita100() {
        assertEquals(100, spada.getDurabilita());
    }

    @Test
    void usaRiduceDurabilita() {
```

```
        spada.usa();
        assertEquals(99, spada.getDurabilita());
    }

    @Test
    void dannoCalaCon Durabilita() {
        int dannoIniziale = spada.getDanno();
        // Simula usura (99 utilizzi → durabilità 1)
        for (int i = 0; i < 99; i++) spada.usa();
        assertTrue(spada.getDanno() < dannoIniziale);
    }

    @Test
    void armaRottaNonInfliggeDanno() {
        for (int i = 0; i < 100; i++) spada.usa();
        assertTrue(spada.isRotta());
        // Con durabilità 0, il danno dovrebbe essere 0
        assertEquals(0, spada.getDanno());
    }

    @Test
    void riparaRipristinaDurabilita() {
        spada.usa();
        spada.usa();
        spada.ripara();
        assertEquals(100, spada.getDurabilita());
    }

    @ParameterizedTest
    @ValueSource(ints = {0, -1, -100})
    void dannoBas eNonPositivoVieneNormalizzato(int dannoInvalido) {
        Arma arma = new Arma("Test", dannoInvalido);
        assertTrue(arma.getDanno() > 0);
    }
}
```

17.11 Riepilogo

I concetti chiave di questa lezione

- I **test automatici** verificano il comportamento del codice in modo rapido e ripetibile.
- **JUnit 5** è il framework standard per il testing in Java.
- Le **asserzioni** (`assertEquals`, `assertTrue`, `assertThrows`…) verificano i risultati attesi.
- `@BeforeEach` ricrea l'oggetto pulito prima di ogni test — fondamentale per l'isolamento.
- I **casi limite** (zero, negativo, null, stringa vuota) sono i test più importanti.
- I **test parametrizzati** evitano la duplicazione quando testi lo stesso comportamento con valori diversi.
- Un **piano di test** elenca sistematicamente cosa verificare prima di scrivere i test.
- Il **TDD** inverte l'ordine: test prima, codice dopo — porta a design più puliti.

18 SOLID

18.1 Cos'è SOLID

Quando un programma è piccolo, quasi qualsiasi approccio funziona. Ma quando cresce — più classi, più funzionalità, più persone che ci lavorano, il codice scritto senza criteri diventa difficile da capire, da modificare e da testare.

SOLID è un insieme di cinque principi di progettazione che aiutano a scrivere codice orientato agli oggetti che sia: - facile da leggere - facile da modificare senza rompere quello che già funziona - facile da estendere con nuove funzionalità

Ogni lettera di SOLID corrisponde a un principio.

Lettera	Nome	In breve
S	Single Responsibility	una classe, un solo compito
O	Open/Closed	aperto alle estensioni, chiuso alle modifiche
L	Liskov Substitution	le sottoclassi devono poter sostituire la madre
I	Interface Segregation	interfacce piccole e specifiche
D	Dependency Inversion	dipendi dalle astrazioni, non dalle implementazioni

Non devi memorizzare le definizioni. Devi capire il problema che ogni principio risolve.

18.2 S - Single Responsibility Principle

> Una classe deve avere un solo motivo per cambiare.

In altre parole: una classe fa una cosa sola e la fa bene.

18.2.1 Il problema

```
public class Personaggio {
    private String nome;
    private int vita;
    private int livello;

    // logica del personaggio
    public void subirDanno(int danno) { ... }
    public void salaDiLivello() { ... }

    // logica di salvataggio -- non c'entra con il personaggio!
    public void salvaSuFile(String percorso) { ... }
    public void caricaDaFile(String percorso) { ... }

    // logica di stampa -- non c'entra con il personaggio!
    public void stampaScheda() { ... }
    public void stampaStatistiche() { ... }
}
```

`Personaggio` fa troppe cose: gestisce lo stato del personaggio, salva su file e stampa. Se cambia il formato del file di salvataggio, devi modificare `Personaggio` – ma il personaggio non c'entra nulla con il file.

18.2.2 La soluzione

Separa le responsabilità in classi diverse:

```
public class Personaggio {
    private String nome;
    private int vita;
    private int livello;

    public void subirDanno(int danno) { ... }
    public void salaDiLivello() { ... }
    // solo la logica del personaggio
}

public class PersonaggioRepository {
    public void salva(Personaggio p, String percorso) { ... }
```

```
    public Personaggio carica(String percorso) { ... }
    // solo la logica di persistenza
}

public class PersonaggioView {
    public void stampaScheda(Personaggio p) { ... }
    public void stampaStatistiche(Personaggio p) { ... }
    // solo la logica di presentazione
}
```

> **Schema mentale**
>
> Quando scrivi una classe, chiediti: "quanti motivi ho per modificare questa classe in futuro?". Se la risposta e piu di uno, probabilmente sta facendo troppe cose.

18.3 O - Open/Closed Principle

Una classe deve essere aperta alle estensioni ma chiusa alle modifiche.

In altre parole: aggiungi funzionalita nuove senza toccare il codice che gia funziona.

18.3.1 Il problema

```
public class CalcolatoreTariffa {

    public double calcola(Personaggio p, String tipo) {
        if (tipo.equals("normale")) {
            return p.getLivello() * 10.0;
        } else if (tipo.equals("premium")) {
            return p.getLivello() * 10.0 * 0.8;
        } else if (tipo.equals("vip")) {
            return p.getLivello() * 10.0 * 0.5;
        }
        // ogni volta che aggiungi un tipo, devi modificare questo metodo
        return 0;
    }
}
```

Ogni nuovo tipo di tariffa richiede di aprire e modificare `CalcolatoreTariffa`. Rischi di rompere i tipi che gia funzionano.

18.3.2 La soluzione

Usa un'interfaccia per definire il contratto, e una classe separata per ogni variante:

```
public interface Tariffa {
    double calcola(Personaggio p);
}

public class TariffaNormale implements Tariffa {
    @Override
    public double calcola(Personaggio p) {
        return p.getLivello() * 10.0;
    }
}

public class TariffaPremium implements Tariffa {
    @Override
    public double calcola(Personaggio p) {
        return p.getLivello() * 10.0 * 0.8;
    }
}

public class TariffaVip implements Tariffa {
    @Override
    public double calcola(Personaggio p) {
        return p.getLivello() * 10.0 * 0.5;
    }
}
```

Per aggiungere un nuovo tipo di tariffa, crei una nuova classe che implementa `Tariffa` – senza toccare nulla di quello che esiste gia.

> **Schema mentale**
>
> Quando ti trovi ad aggiungere un `else if` a una catena gia lunga, fermati. Probabilmente stai violando Open/Closed. Chiediti: posso risolvere questo con una nuova classe invece di modificare quella esistente?

18.4 L - Liskov Substitution Principle

> Se B estende A, allora ovunque si usa A si deve poter usare B senza che il programma si comporti in modo inatteso.

In altre parole: una sottoclasse deve poter sostituire la classe madre senza sorprese.

18.4.1 Il problema

```
public class Rettangolo {
    protected int base;
    protected int altezza;

    public void setBase(int base)       { this.base = base; }
    public void setAltezza(int altezza) { this.altezza = altezza; }
    public int area()                   { return base * altezza; }
}

public class Quadrato extends Rettangolo {
    @Override
    public void setBase(int lato) {
        this.base = lato;
        this.altezza = lato;   // il quadrato ha i lati uguali
    }

    @Override
    public void setAltezza(int lato) {
        this.base = lato;
        this.altezza = lato;
    }
}
```

In apparenza sembra corretto. Ma:

```
Rettangolo r = new Quadrato();
r.setBase(5);
r.setAltezza(3);
System.out.println(r.area());   // ci aspettiamo 15, otteniamo 9
```

`Quadrato` si comporta in modo diverso da `Rettangolo` – viola il principio di Liskov.

18.4.2 La soluzione

Se una sottoclasse non puo rispettare il contratto della madre, ripensa la gerarchia. In questo caso, `Quadrato` e `Rettangolo` non hanno un rapporto "e un" corretto – entrambi possono estendere una classe `Figura` piu generica.

```
public abstract class Figura {
    public abstract int area();
}

public class Rettangolo extends Figura {
    private int base, altezza;
    public Rettangolo(int base, int altezza) { ... }
    public int area() { return base * altezza; }
}

public class Quadrato extends Figura {
    private int lato;
    public Quadrato(int lato) { ... }
    public int area() { return lato * lato; }
}
```

> **Schema mentale**
>
> Se una sottoclasse deve "sabotare" un metodo della madre – svuotarlo, lanciare un'eccezione, cambiarne il significato – stai probabilmente violando Liskov. Ripensa la gerarchia.

18.5 I - Interface Segregation Principle

> Non costringere una classe a implementare metodi che non usa.

In altre parole: e meglio avere tante interfacce piccole e specifiche che una sola grande e generica.

18.5.1 Il problema

```java
public interface Personaggio {
    void attacca();
    void difendi();
    void lanciaIncantesimo();
    void cavalca();
    void nuota();
}

public class Guerriero implements Personaggio {
    public void attacca() { ... }
    public void difendi() { ... }
    public void lanciaIncantesimo() {
        throw new UnsupportedOperationException(); // il guerriero non lancia incantesimi!
    }
    public void cavalca() { ... }
    public void nuota() {
        throw new UnsupportedOperationException(); // il guerriero non sa nuotare!
    }
}
```

`Guerriero` e costretto a implementare metodi che non ha senso abbia. L'interfaccia e troppo grande.

18.5.2 La soluzione

Dividi l'interfaccia in parti piu piccole e specifiche:

```java
public interface Combattente {
    void attacca();
    void difendi();
}

public interface Mago {
    void lanciaIncantesimo();
}

public interface Cavaliere {
    void cavalca();
}

public interface Nuotatore {
```

```
    void nuota();
}

// Il guerriero implementa solo quello che sa fare
public class Guerriero implements Combattente, Cavaliere {
    public void attacca() { ... }
    public void difendi() { ... }
    public void cavalca() { ... }
}

// Il mago implementa quello che sa fare lui
public class Mago implements Combattente, Mago {
    public void attacca() { ... }
    public void difendi() { ... }
    public void lanciaIncantesimo() { ... }
}
```

> **Schema mentale**
>
> Se una classe implementa un'interfaccia ma alcuni metodi li lascia vuoti o lancia eccezioni, l'interfaccia e troppo grande. Spezzala.

18.6 D - Dependency Inversion Principle

> I moduli di alto livello non devono dipendere dai moduli di basso livello. Entrambi devono dipendere dalle astrazioni.

In altre parole: dipendi dalle interfacce, non dalle implementazioni concrete.

18.6.1 Il problema

```
public class GestorePartita {

    private SalvataggioSuFile salvataggio;   // dipende da una classe concreta!

    public GestorePartita() {
```

```
        this.salvataggio = new SalvataggioSuFile();
    }

    public void salva(Personaggio p) {
        salvataggio.salva(p);
    }
}
```

`GestorePartita` e legato a `SalvataggioSuFile`. Se vuoi cambiare il sistema di salvataggio – per esempio salvare su database o sul cloud – devi modificare `GestorePartita`.

18.6.2 La soluzione

Introduce un'interfaccia e passa l'implementazione dall'esterno:

```
public interface Salvataggio {
    void salva(Personaggio p);
    Personaggio carica();
}

public class SalvataggioSuFile implements Salvataggio {
    public void salva(Personaggio p) { ... }
    public Personaggio carica() { ... }
}

public class SalvataggioSuDatabase implements Salvataggio {
    public void salva(Personaggio p) { ... }
    public Personaggio carica() { ... }
}

public class GestorePartita {

    private Salvataggio salvataggio;   // dipende dall'interfaccia

    public GestorePartita(Salvataggio salvataggio) {
        this.salvataggio = salvataggio;   // riceve l'implementazione dall'esterno
    }

    public void salva(Personaggio p) {
        salvataggio.salva(p);
```

```
    }
}
```

Ora puoi cambiare il sistema di salvataggio senza toccare `GestorePartita`:

```
GestorePartita g1 = new GestorePartita(new SalvataggioSuFile());
GestorePartita g2 = new GestorePartita(new SalvataggioSuDatabase());
```

> **Schema mentale**
>
> Se dentro una classe scrivi `new NomeClasseConcreta()` per creare una dipendenza, chiediti: questa classe deve davvero sapere quale implementazione usare? Spesso la risposta e no – ricevila dall'esterno tramite il costruttore.

18.7 I cinque principi insieme

SOLID non e una lista di regole da seguire ciecamente. E un insieme di domande da farti mentre progetti:

Principio	La domanda da farti
S	Questa classe ha piu di un motivo per cambiare?
O	Posso aggiungere questa funzionalita senza modificare codice esistente?
L	Le mie sottoclassi si comportano come ci si aspetta dalla madre?
I	Le mie interfacce hanno metodi che alcune classi non useranno mai?
D	Le mie classi dipendono da implementazioni concrete invece che da astrazioni?

Se la risposta a queste domande e si, c'e spazio per migliorare il design.

18.8 Riepilogo

I concetti chiave di questa lezione

- **S** - Single Responsibility: una classe, una sola responsabilita. Se fa troppe cose, spezzala.
- **O** - Open/Closed: estendi con nuove classi, non modificare quelle esistenti. Usa interfacce e polimorfismo.
- **L** - Liskov Substitution: una sottoclasse deve comportarsi come la madre. Se non puo, ripensa la gerarchia.
- **I** - Interface Segregation: interfacce piccole e specifiche. Nessuna classe deve implementare metodi inutili.
- **D** - Dependency Inversion: dipendi dalle interfacce, non dalle classi concrete. Passa le dipendenze dall'esterno.

19